

به نام خدا



دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس مدار های مجتمع خیلی فشرده تمرین کامپیوتری سوم

نام و نام خانوادگی	بهزاد جنتی
شماره دانشجویی	810103098
تاریخ ارسال گزارش	1404/03/11
استاد	دکتر شقایق وحدت
ترم	بهار 1404

فهرست

9	بخش اول تمرین
9	بخش الف) پیاده سازی CARRY PROPAGATION ADDER
10	پیاده سازی گیت های پایه:
10	گیت NAND2:
11	تعریف گیت NOR2:
12	تعریف گیت INVERTER:
13	پیاده سازی گیت XOR:
14	پیاده سازی گیت های AND2 و OR2:
14	پیاده سازی AND2:
15	پیاده سازی FA:
16	پیاده سازی جمع کننده 8 بیتی CRA:
18	تست جمع کننده 8 بیتی RIPPLE با مقداردهی اولیه ورودی ها:
19	انجام شبیه سازی:
20	رسم شکل موج خروجی برای جمع کننده RIPPLE
23	بخش ب) پیاده سازی جمع کننده KOGGE STONE
23	مفهوم و پیاده سازی PROPAGATE:
24	پیاده سازی بلاک های جمع کننده kogge stone:
24	بلاک pre processing:
25	بلاک Black Cell:
25	بلاک Grey Cell:
25	پیاده سازی SUM block:
26	تعریف جمع کننده KOGGE STONE:
26	مرحله اول pre processing:
27	مرحله دوم KOGGE STONE:
27	مرحله یا استیج سوم پیاده سازی KOGGE STONE:

28	مرحله چهارم پیاده سازی KOGGE STONE
29	محاسبه SUM ها در جمع کننده KOGGE STONE
29	تست جمع کننده KOGGE STONE با ورودی های دستی
30	نتایج انجام شبیه سازی برای تست ورودی های دستی
32	ج) بررسی صحت عملکرد دو جمع کننده با وکتور
35	تست جمع کننده CRA با وکتور فایل
38	تست جمع کننده KOGGE STONE با وکتور فایل
40	د) محاسبه وکتور برای تحریک مسیر بحرانی و اندازی گیری تاخیر بیشینه جمع کننده ها
44	بخش ه) محاسبه توان دینامیک برای دو جمع کننده
45	خروجی توان دینامیک برای جمع کننده CRA
45	توان دینامیک برای جمع کننده KOGGE STONE
46	بخش و) محاسبه توان استاتیک برای دو جمع کننده
48	نتیجه شبیه سازی برای توان استاتیک جمع کننده CRA
49	نتیجه شبیه سازی برای توان استاتیک جمع کننده KOGGE STONE
50	بخش دوم تمرین
51	بررسی پارامترهای تاخیر و توان در گوشه طراحی FF
51	بررسی تاخیر بیشینه در کرنر FF برای جمع کننده KOGGE STONE
53	بررسی توان دینامیک برای جمع کننده KOGGE STONE در گوشه طراحی FF
54	بررسی توان استاتیک برای جمع کننده KOGGE STONE در گوشه FF
55	بررسی پارامترهای تاخیر و توان در گوشه طراحی SS
55	بررسی تاخیر بیشینه در کرنر SS برای جمع کننده KOGGE STONE
56	بررسی توان دینامیک برای جمع کننده KOGGE STONE در گوشه طراحی SS
56	بررسی توان استاتیک برای جمع کننده KOGGE STONE در گوشه SS
57	بررسی پارامترهای تاخیر و توان در گوشه طراحی FS
58	بررسی تاخیر بیشینه در کرنر FS برای جمع کننده KOGGE STONE
58	بررسی توان دینامیک برای جمع کننده KOGGE STONE در گوشه طراحی FS

59		بررسی توان استاتیک برای جمع کننده KOGGE STONE در گوشه طراحی FS:
60		بررسی پارامترهای تاخیر و توان در گوشه طراحی SF:
60		بررسی تاخیر بیشینه در کرنر SF برای جمع کننده KOGGE STONE:
61		بررسی توان دینامیک برای جمع کننده KOGGE STONE در گوشه طراحی SF:
61		بررسی توان استاتیک برای جمع کننده KOGGE STONE در گوشه طراحی SF:
63		بخش سوم تمرین
63		بخش الف)
65		بخش امتیازی:
65		انجام شبیه سازی:
68		کد پایتون برای استخراج داده ها از فایل ca3_koggestone.mt0
69		نمودار توان دینامیک بر حسب ولتاژ Vdd:
69		هیستوگرام تأخیر بر حسب ولتاژ:
70		هیستوگرام تأخیر و توان:
72		بخش ب سوال)
75		بررسی نمودار تاخیر بر حسب دما:
76		نمودار توان دینامیک بر حسب دما:
76		جمع بندی بخش ب تمرین)

شکل‌ها

- شکل 1- ایمپورت کردن لایبرری 90 نانومتر و تعریف پارامترها برای پیاده سازی..... 10
- شکل 2- تصویر شماتیک NAND دو ورودی 10
- شکل 3- تعریف subcircuit ، NAND2 در کد 11
- شکل 4- نحوه تعریف ترانزیستورها در کد..... 11
- شکل 5- تصویر شماتیک NOR2 11
- شکل 6- تعریف NOR2 به عنوان subcircuit در کد 12
- شکل 7- شماتیک گیت inverter 12
- شکل 8- تعریف گیت INVERTER به عنوان subcircuit در کد HSPICE 13
- شکل 9- شماتیک گیت xor 13
- شکل 10- تعریف گیت XOR به عنوان subcircuit در کد 14
- شکل 11- پیاده سازی AND2 در کد به کمک inverter و NAND2 14
- شکل 12- تعریف گیت OR2 به عنوان سابسرکت به کمک گیت های NOR2 و inverter در کد 15
- 15
- شکل 13- شماتیک FA 15
- شکل 14- بلوک دیاگرام مربوط به FA 16
- شکل 15- تعریف FA به عنوان یک subcircuit در کد 16
- شکل 16- تصویر جمع کننده 4 بیتی به روش RIPPLE ADDER 17
- شکل 17- تعریف جمع کننده 8 بیتی CRA در کد HSPICE 17
- شکل 18- نمونه گیری از جمع کننده و مقدار دهی اولیه به سیگنالهای ورودی جهت بررسی صحت کارکرد مدار..... 18
- شکل 19- انجام شبیه سازی زمانی..... 19
- شکل 20- خروجی مورد انتظار برای ورودی های اعمال شده به جمع کننده به شکل دستی . 19
- شکل 21- دستور option post =2 19
- شکل 22- انجام شبیه سازی در نرم افزار HSPICE 20
- شکل 23- پیام job concluded به نشانه ی انجام شبیه سازی به شکل موفق..... 20
- شکل 24- شکل موج مربوط به sum اول 21
- شکل 25- شکل موج مربوط به sum دوم 21
- شکل 26- تصویر شکل موج مربوط به sum سوم 22
- شکل 27- تصویر شکل موج خروجی S4 22
- شکل 28- مفهوم propagate و generate 24

شکل 29- بلاک محاسبه P و G برای همه‌ی ورودی‌ها در اولین مرحله جمع‌کننده kogge stone	24
شکل 30- تعریف بلوک Black Cell به شکل سابسرکت در کد	25
شکل 31- تعریف Grey Cell به شکل سابسرکت در کد	25
شکل 32- تعریف sum block به عنوان سابسرکت در کد	26
شکل 33- مرحله pre processing در جمع‌کننده kogge stone adder	26
شکل 34- مرحله دوم پیاده‌سازی KOGGE STONE ADDER	27
شکل 35- مرحله سوم پیاده‌سازی KOGGE STONE	27
شکل 36- مرحله چهارم پیاده‌سازی KOGGE STONE	28
شکل 37- تولید کری خروجی در جمع‌کننده KOGGE STONE ADDER	28
شکل 38- محاسبه SUM ها در جمع‌کننده KOGGE STONE	29
شکل 39- تعریف main circuit برای kogge stone adder در فایل CA3_koggestone.sp	29
شکل 40- خروجی sum اول برای ورودی دستی داده شده به جمع‌کننده kogge stone	30
شکل 41- خروجی sum دوم برای ورودی دستی داده شده به جمع‌کننده kogge stone	30
شکل 42- تصویر مربوط به sum سوم جمع‌کننده kogge stone	31
شکل 43- خروجی مربوط به sum چهارم جمع‌کننده kogge stone برای ورودی دستی داده شده	31
شکل 44- فایل پایتون برای ایجاد وکتور فایل جهت ایجاد تست کیس‌های تصادفی	33
شکل 45- تعریف Radix و سیگنال‌های ورودی در فایل وکتور	34
شکل 46- تعریف مقادیر پریود و موارد مربوط به تاخیر سیگنال‌ها در فایل وکتور	34
شکل 47- نمونه‌ای از 1000 تست کیس تولید شده برای جمع‌کننده CRA	35
شکل 48- انجام شبیه‌سازی برای CRA ADDER با دستور tran	36
شکل 49- خروجی فایل ca3.err پس از انجام شبیه‌سازی روی 1000 نمونه تست تصادفی	36
شکل 50- تغییر به شکل دستی یکی از تست کیس‌ها	36
شکل 51- تغییر بیت کری خروجی برای اولین تست کیس فایل DVF1.vec	37
شکل 52- خروجی فایل ca3.err پس از انجام شبیه‌سازی و تغییر دستی یک بیت تست کیس	37
شکل 53- دستور tran برای انجام شبیه‌سازی زمانی در فایل ca3_koggestone.sp	38
شکل 54- خروجی فایل ca3_koggestone پس از انجام شبیه‌سازی روی وکتورها	38
شکل 55- تغییر مقدار Cout برای اولین تست وکتور در فایل DVF.vec برای جمع‌کننده KOGGE STONE	39
شکل 56- خروجی فایل ارور پس از تغییر تست وکتور به شکل دستی	39
شکل 57- فایل وکتور جدید برای مسیر بحرانی	41

- شکل 58- نحوه محاسبه وکتورها برای اندازه گیری تاخیر بیشینه و مسیر بحرانی 41
- شکل 59- تاخیر ماکسیمم برای SUM جمع کننده CRA 42
- شکل 60- نحوه اندازه گیری تاخیر ها با دستور MEASURE 42
- شکل 61- خروجی تاخیر برای وکتور دوم مربوط به کری جمع کننده CRA 42
- شکل 62- تاخیر بیشینه برای KOGGE STONE 43
- شکل 63- تنظیمات در فایل وکتور رندوم برای محاسبه توان دینامیک برای 1000 تست ورودی 44
- شکل 64- تنظیمات برای انجام شبیه سازی برای توان دینامیک 45
- شکل 65- توان دینامیک برای جمع کننده CRA 45
- شکل 66- محاسبه توان دینامیک برای KOGGE STONE به ازای 1000 تست وکتور رندوم ورودی 45
- شکل 67- روال محاسبه توان استاتیک از روی توان میانگین 47
- شکل 68- مقدار توان میانگین اندازه گیری شده برای CRA پس از شبیه سازی برای 1000 وکتور رندوم ورودی 48
- شکل 69- انجام محاسبات برای بدست آوردن توان استاتیک جمع کننده CRA 48
- شکل 70- مقدار توان متوسط اندازه گیری شده برای جمع کننده KOGGE STONE روی 1000 وکتور رندوم ورودی 49
- شکل 71- انجام محاسبات برای بدست آوردن توان استاتیک خالص برای جمع کننده KOGGE STONE 49
- شکل 72- اضافه کردن کتابخانه 90 نانومتر با پسوند FF 51
- شکل 73- نتیجه شبیه سازی برای تاخیر بیشینه در کرنر FF 52
- شکل 74- توان دینامیک جمع کننده KOGGE STONE در کرنر FF و برای 1000 تست وکتور رندوم 53
- شکل 75- توان میانگین اندازه گیری شده در کرنر FF 54
- شکل 76- توان استاتیک محاسبه شده در گوشه FF 54
- شکل 77- ایمپورت لایبری با پسوند SS 55
- شکل 78- تاخیر بیشینه جمع کننده KOGGE STONE در کرنر SS 55
- شکل 79- توان دینامیک جمع کننده KOGGE STONE در کرنر SS 56
- شکل 80- توان میانگین در گوشه SS برای جمع کننده KOGGE STONE 57
- شکل 81- توان استاتیک محاسبه شده برای جمع کننده KOGGE STONE در کرنر SS 57
- شکل 82- تاخیر بیشینه برای جمع کننده در کرنر FS 58
- شکل 83- توان دینامیک برای جمع کننده در گوشه FS 59
- شکل 84- توان میانگین اندازه گیری شده برای گوشه FS 59

- شکل 85- توان استاتیک جمع کننده در گوشه FS..... 60
- شکل 86- ایمپورت لایبرری با پسوند SF..... 60
- شکل 87- تاخیر بیشینه در گوشه SF..... 61
- شکل 88- توان دینامیک اندازه گیری شده در گوشه SF..... 61
- شکل 89- توان میانگین محاسبه شده در گوشه SF برای 5000 نانو ثانیه شبیه سازی 62
- شکل 90- توان استاتیک برای گوشه SF..... 62
- شکل 91- انجام شبیه سازی در HSPICE..... 66
- شکل 92- خروجی شبیه سازی..... 67
- شکل 93- کد پایتون برای فایل با پسوند mt0..... 68
- شکل 94- نمودار توان دینامیک بر حسب منبع ولتاژ 69
- شکل 95- نمودار تاخیر بر حسب ولتاژ..... 70
- شکل 96- انجام شبیه سازی با دکاهای مختلف با اضافه کردن دستور temp..... 73
- شکل 97- خروجی فایل lis. برای شبیه سازی در دماهای مختلف 73
- شکل 98- فایل های ایجاد شده پس از شبیه سازی با پسوند mtX..... 73
- شکل 99- برنامه پایتون برای استخراج اطلاعات از فایلای با پسوند mtX..... 74
- شکل 100- نمودار تغییرات تاخیر بر حسب دما 75
- شکل 101- نمودار توان بر حسب دما..... 76

جدول‌ها

- جدول 1- مقایسه تاخیر بیشینه جمع کننده ها 43
- جدول 2- مقایسه توان دینامیک مصرفی برای دو جمع کننده به ازای 1000 وکتور ورودی 46
- جدول 3- مقایسه توان استاتیک برای دو جمع کننده 50
- جدول 4- مقایسه تاخیر ، توان دینامیک و استاتیک جمع کننده KOGGE STONE در گوشه های مختلف 63

بخش اول تمرین



دانشکده مهندسی برق و کامپیوتر
نیمسال دوم ۱۴۰۳-۱۴۰۴

دانشگاه تهران
VLSI پیشرفته - دکتر وحدت
تمرین کامپیوتری سوم



مقدمه:

در این تمرین کامپیوتری مدارهای مربوط به دو جمع‌کننده‌ی مختلف را پیاده‌سازی کرده و سپس به کمک نرم‌افزار HSPICE از نظر توان و تأخیر آنها را با یکدیگر مقایسه خواهیم کرد. درگام بعدی تأثیر نوسانات بر عملکرد جمع‌کننده‌ها به کمک تحلیل Monte Carlo بررسی خواهد شد.

بخش اول: (۷۵٪)

در این بخش دو جمع‌کننده‌ی ۸ بیتی Carry Propagation و Kogge Stone را پیاده‌سازی خواهیم کرد. سپس از صحت عملکرد هر دوی آنها مطمئن شده و تأخیر بیشینه و توان استاتیک و دینامیک را برای هر دو جمع‌کننده به دست خواهیم آورد. در نهایت به مقایسه‌ی نتایج به دست آمده خواهیم پرداخت. در طراحی جمع‌کننده‌ها توجه داشته باشید که حتماً سیگنال‌های C_{in} و C_{out} را علاوه بر دو ورودی ۸ بیتی و یک خروجی ۸ بیتی در نظر بگیرید.

در بخش اول این تمرین قرار است هر دو ساختار جمع‌کننده خواسته شده را در محیط HSPICE پیاده‌سازی کنیم و سپس به انجام تست‌های خواسته شده بپردازیم. برای انجام این تمرین و نوشتن کدها، جهت سادگی و دیباگ راحت‌تر از نرم‌افزار Notepad++ استفاده کردیم. حال به بررسی بخش الف می‌پردازیم.

بخش الف) پیاده‌سازی CARRY PROPAGATION ADDER

الف) (۱۵٪) ساختار مربوط به یک Carry Propagation Adder (CPA) ۸ بیتی را در HSPICE پیاده‌سازی کنید. در پیاده‌سازی خود حتماً به ماحول محور (Modular) بودن توصیف‌ها اهمیت دهید، زیرمدار (subcircuit)‌های مورد استفاده را بیان کرده و نحوه‌ی قرارگیری آنها را شرح دهید.

در این بخش از ما خواسته شده تا یک جمع‌کننده ۸ بیتی CARRY PROPAGATION ADDER پیاده‌سازی کنیم و طراحی هم باید ماژولار باشد.

خب در اینجا یک فایل با نام CA3.sp ساختیم و سپس در ابتدای فایل پس از نوشتن صورت سوال در خط اول به ایمپورت لایبرری 90 نانومتر که در مورد نیاز تمرین است پرداختیم.

```

D:\vlsi vahdat\HWs&CAs\CA3_810103098\ca3.sp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF2.vec
1 *****carry ripple adder*****
2 .lib "crn90g_2d5_1k_v1d2p1.1" TT
3
4 * Technology Parameters
5 .param Lmin=180n
6 .param Wmin=180n
7 .param VDD = 1.0
8

```

شکل 1- ایمپورت کردن لایبری 90 نانومتر و تعریف پارامترها برای پیاده سازی

همان طور که در تصویر بالا میبینید پارامترها شامل حداقل طول و عرض ترانزیستور هاست که در صورت سوال گفته شده بود که باید 180n در نظر گرفته شوند.

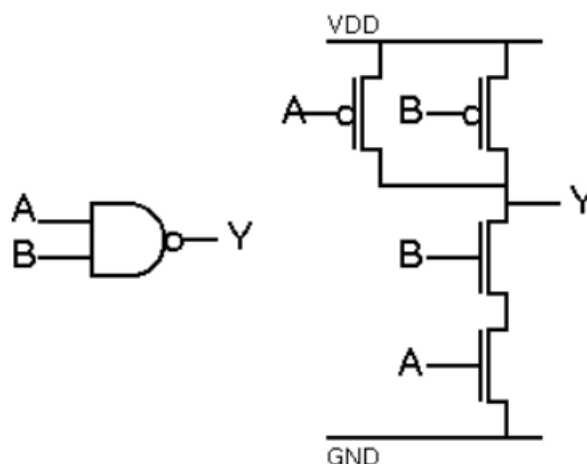
همچنین یه پارامتر هم برای منبع ولتاژ مدارمون در نظر گرفتیم که مقدار 1 ولت را طبق صورت سوال برای آن در نظر گرفتیم.

پیاده سازی گیت های پایه:

در این بخش قرار است گیت های پایه ای که در رسم مدار و تعریف بخش های مختلف به آنها نیاز داریم را تعریف کنیم.

گیت NAND2:

در زیر تصویری از NAND2 گذاشته شده است.



شکل 2- تصویر شماتیک NAND دو ورودی

همان طور که در تصویر می بینید NAND2 از دو ترانزیستور NMOS و دو ترانزیستور PMOS تشکیل شده که PMOS ها با هم موازی و NMOS ها با هم سری هستند.

در زیر هم تصویری از تعریف NAND2 در نرم افزار HSPICE رو مشاهده میکنید:

```
D:\vlsi vahdat\HWs&CAs\CA3_810103098\ca3.sp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF2.vec
10
11 ***** defining subcircuit for NAND2 gate *****
12 .SUBCKT NAND2 A B out vdd vss
13 M1 out A net1 vss nch L=Lmin W=Wmin
14 M2 net1 B vss vss nch L=Lmin W=Wmin
15 M3 out A vdd vdd pch L=Lmin W='2*Wmin'
16 M4 out B vdd vdd pch L=Lmin W='2*Wmin'
17 .ENDS NAND2
18
```

شکل 3- تعریف subcircuit ، NAND2 در کد

در ضمن اندازه W رو برای ترانزیستورهای PMOS دو برابر NMOS گرفتیم تا تاخیر شبکه بالا بر و پایین بر یکسان شود.

برای تعریف ترانزیستورها هم از فرمتی که در صورت سوال قرار داده شده بود استفاده کردیم.

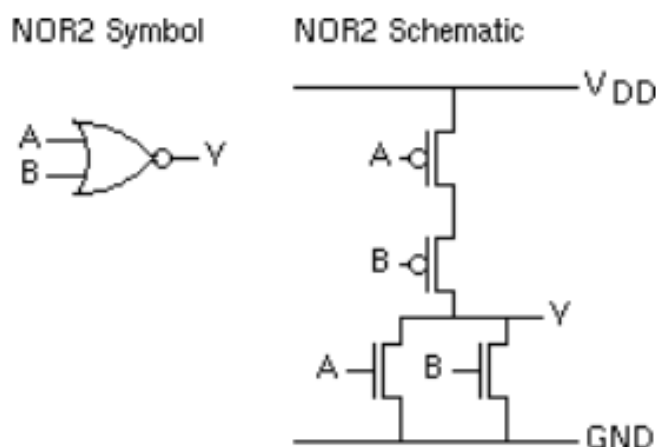
- برای نمونه گیری از ترانزیستورها از دستور زیر به عنوان نمونه استفاده نمایید:

```
M1 out in1 0 0 nch L=Lmin W=Wmin
M2 out in1 Vdin Vdin pch L=Lmin W='Wmin*2'
```

شکل 4- نحوه تعریف ترانزیستورها در کد

تعریف گیت NOR2:

در ابتدا تصویری از گیت NOR2 در زیر قرار داده شده است:



شکل 5- تصویر شماتیک NOR2

همان طور که می بینید این گیت هم از چهار ترانزیستور ساخته می شود که برعکس NAND2 دو ترانزیستور PMOS در شبکه بالابر با هم سری و دو ترانزیستور NMOS در شبکه پایین بر با هم موازی هستند.

در ادامه تصویر تعریف این گیت در نرم افزار HSPICE هم قرار داده شده است.

```

19 ***** defining subcircuit for NOR2 gate *****
20 .SUBCKT NOR2 A B out vdd vss
21 M1 out A vss vss nch L=Lmin W=Wmin
22 M2 out B vss vss nch L=Lmin W=Wmin
23 M3 out A net1 vdd pch L=Lmin W='2*Wmin'
24 M4 net1 B vdd vdd pch L=Lmin W='2*Wmin'
25 .ENDS NOR2
26

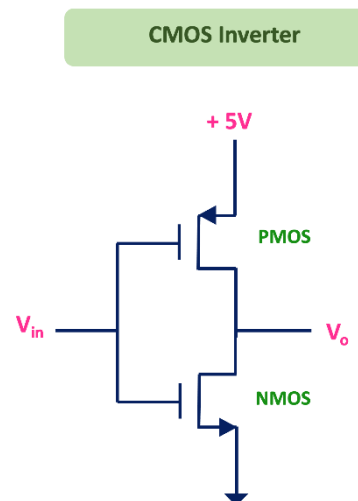
```

شکل 6- تعریف NOR2 به عنوان subcircuit در کد

در اینجا منظور از VSS همان زمین یا GND است که در ادامه به عنوان منبع ولتاژ 0 ولت تعریف شده است و در تعریف گیت ها و مدارها به کار میرود.

تعریف گیت INVERTER:

در ابتدا شماتیک این گیت قرار داده شده است:



شکل 7- شماتیک گیت inverter

همان طور که می بینید این گیت از دو ترانزیستور ساخته شده که شامل یک PMOS در بالابر و یک NMOS در شبکه پایین بر است.

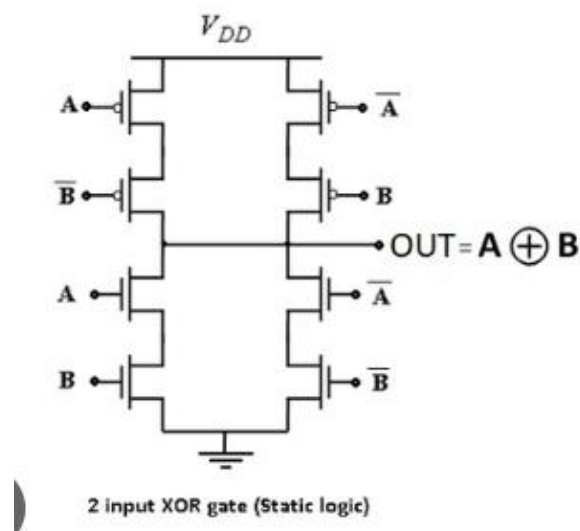
در ادامه تصویر تعریف این گیت به عنوان subcircuit در کد قرار داده شده است:

```
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec
26
27 ***** defining subcircuit for inverter gate *****
28 .SUBCKT INV in out vdd vss
29 M1 out in vss vss nch L=Lmin W=Wmin
30 M2 out in vdd vdd pch L=Lmin W='2*Wmin'
31 .ENDS INV
32
```

شکل 8- تعریف گیت INVERTER به عنوان subcircuit در کد HSPICE

پیاده سازی گیت XOR:

در ابتدا مثل موارد قبل اول شماتیکی از این گیت قرار داده شده است که بر اساس آن این گیت در کد تعریف شده است.



شکل 9- شماتیک گیت xor

همان طور که در تصویر نیز مشاهده می‌کنید برای پیاده سازی و تعریف این گیت به 8 ترانزیستور نیاز است که نحوه چیدمان آنها در تصویر بالا مشخص است، همچنین نیاز داریم تا معکوس یکسری ورودی ها رو نیز داشته باشیم که برای این کار باید از subcircuit گیت inverter که پیش تر تعریف کرده بودیم نمونه بگیریم.

در زیر تصویر پیاده سازی و تعریف این گیت در کد را مشاهده میکنید:

همچنین برای مناطق بین دو ترانزیستور از net استفاده شده است.

```

D:\vlsi vahdat\HWs&CAs\CA3_810103098\ca3.sp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec
32
33 ***** defining subcircuit for XOR2 gate *****
34 .SUBCKT XOR2 A B out vdd vss
35 Xin1 A Abar vdd vss INV
36 Xin2 B Bbar vdd vss INV
37 M1 net1 A Vdd Vdd pch L=Lmin W='2*Wmin'
38 M2 out Bbar net1 Vdd pch L=Lmin W='2*Wmin'
39 M3 net2 Abar Vdd Vdd pch L=Lmin W='2*Wmin'
40 M4 out B net2 Vdd pch L=Lmin W='2*Wmin'
41 M5 out A net3 vss nch L=Lmin W='Wmin'
42 M6 net3 B vss vss nch L=Lmin W='Wmin'
43 M7 out Abar net4 vss nch L=Lmin W='Wmin'
44 M8 net4 Bbar vss vss nch L=Lmin W='Wmin'
45 .ends XOR2

```

شکل 10- تعریف گیت XOR به عنوان subcircuit در کد

خب تا اینجا تمام گیت های پایه‌ی مورد نیاز رو به شکل subcircuit تعریف کردیم که بعداً بتونیم برای تعریف بخش های مختلف جمع کننده 8 بیتی مون ازشون نمونه بگیریم.

پیاده سازی گیت های AND2 و OR2:

در این بخش قرار است با استفاده از NAND و NOR که پیش تر تعریف کرده بودیم این دو گیت رو نیز پیاده سازی کنیم:

پیاده سازی AND2:

خب برای پیاده سازی این مورد کافیست تا خروجی NAND2 که قبلاً تعریف کرده بودیم رو نات کنیم و تعریف این مورد در کد رو میتونید در زیر مشاهده کنید:

```

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec
46
47 *****
48 * defining subcircuit for AND2 gate
49 .SUBCKT AND2 A B out vdd vss
50 X1 A B net1 vdd vss NAND2
51 X2 net1 out vdd vss INV
52 .ENDS AND2
53

```

شکل 11- پیاده سازی AND2 در کد به کمک inverter و NAND2

در بخش بعد به پیاده سازی OR2 ورودی میپردازیم.

پیاده سازی گیت OR2:

در پیاده سازی OR2 در کدمون از گیت های تعریف شده NOR2 و INVERTER استفاده می‌کنیم و همچنین میتونید تصویر مربوط به این مورد رو هم در صفحه بعد مشاهده کنید.

```

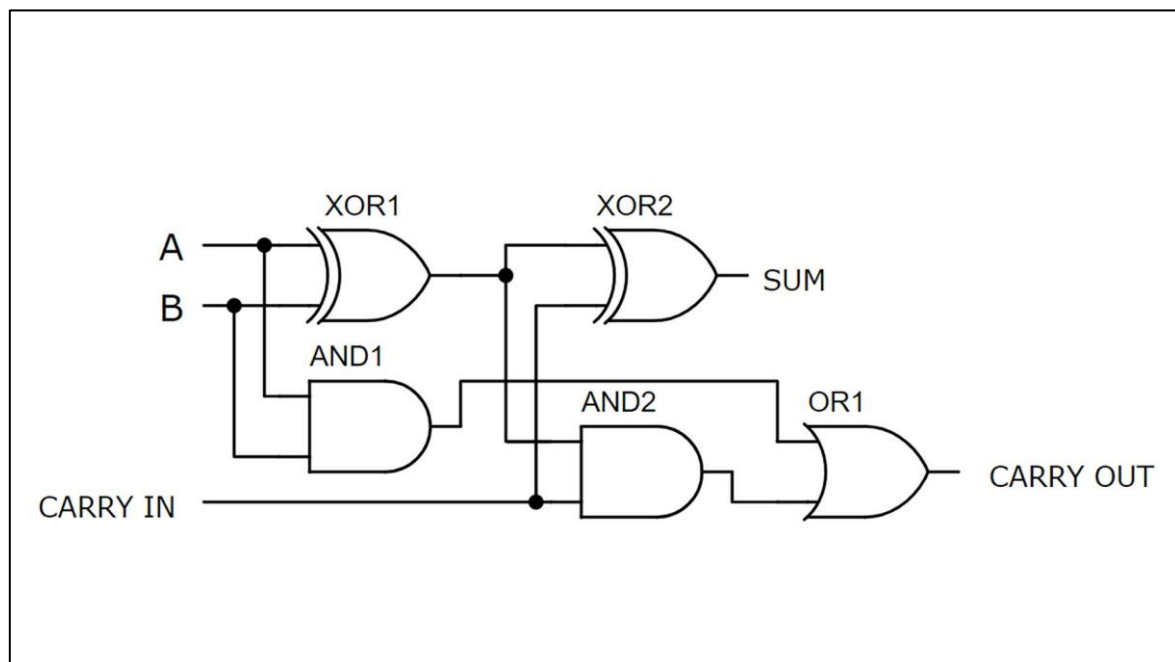
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec
53
54 *****
55 * defining subcircuit for OR2 GATE
56 .SUBCKT OR2 A B out vdd vss
57 X1 A B net1 vdd vss NOR2
58 X2 net1 out vdd vss INV
59 .ENDS OR2
60

```

شکل 12- تعریف گیت OR2 به عنوان سابسرکت به کمک گیت های NOR2 و inverter در کد
 حال تمام گیت های مدنظرمون برای پیاده سازی جمع کننده رو ساختیم.

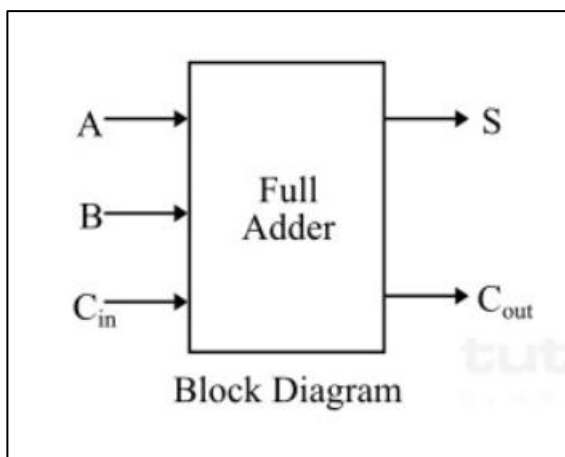
پیاده سازی FA:

خب همان طور که از قبل هم میدانیم FA یک جمع کننده تک بیتی است که دو بیت ورودی رو
 میگیره و به همراه Carry ورودی با هم جمع میکنه و در خروجی یک بیت Sum و یک بیت Cout
 تولید میکنه ، در مجموع 3 ورودی و 2 خروجی داریم.
 برای تولید Sum در FA باید هر سه ورودی گفته شده رو با هم XOR کنیم.
 در زیر شماتیکی از FA قرار داده شده است:



شکل 13- شماتیک FA

در زیر هم بلوک دیاگرام مربوط به FA قرار داده شده است:



شکل 14- بلوک دیاگرام مربوط به FA

حال با توجه به توضیحات و تصاویر قرار داده شده پیاده سازی مربوط به FA و تعریف آن در کدمون رو انجام می‌دهیم:

```
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF2.vec
60
61 *****
62 ***** defining subcircuit for Full Adder *****
63 .SUBCKT FA a b cin sum cout vdd vss
64 * Generate sum: a XOR b XOR cin
65 X1 a b net1 vdd vss XOR2
66 X2 net1 cin sum vdd vss XOR2
67
68 * Generate Cout(carry) = AB + Cin(A XOR B)
69 X3 a b net2 vdd vss AND2
70 X4 net1 cin net3 vdd vss AND2
71 X5 net2 net3 cout vdd vss OR2
72 .ENDS FA
73
```

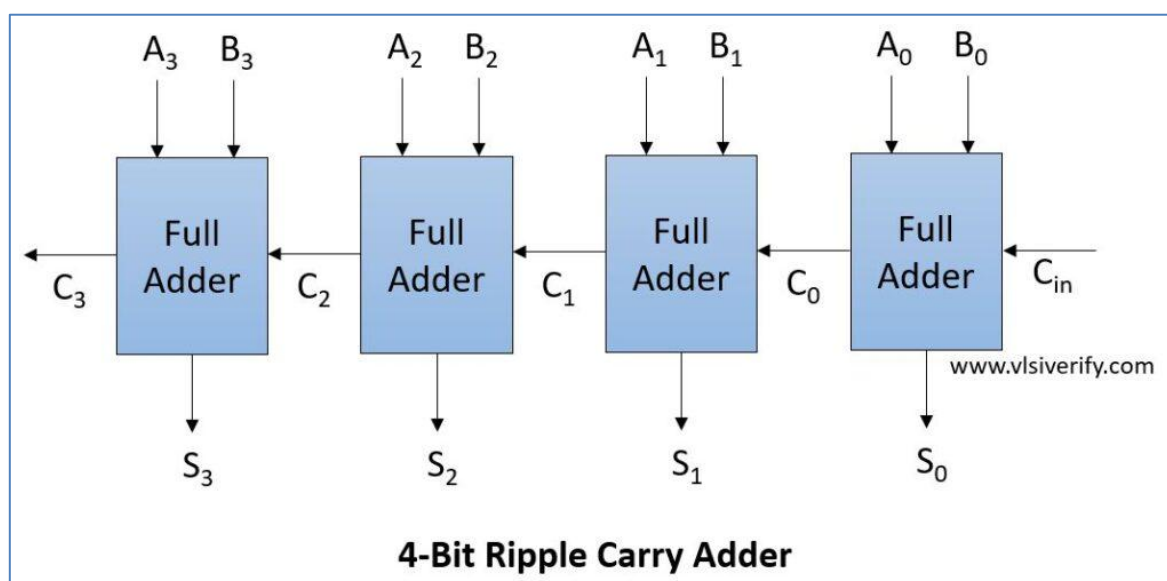
شکل 15- تعریف FA به عنوان یک subcircuit در کد

نکته) خوبی طراحی به روش subcircuit این است که هر وقت بخواهیم میتونیم ازش نمونه بگیریم برای تعریف بقیه بخش ها و ماژولاریتی طراحی رو بالاتر می‌برد.
در ادامه به جزئیات طراحی خود جمع کننده 8 بیتی مون می‌پردازیم و توضیحی در مورد کارکرد آن خواهیم داد.

پیاده سازی جمع کننده 8 بیتی CRA:

خب در این قسمت قرار است به پیاده سازی CARRY RIPPLE ADDER در کدمون بپردازیم. همان طور که مشخص است این جمع کننده از 8 تا FA تشکیل شده است که با هم ripple شده اند یعنی carry تولید شده توسط FA اول میشه کری ورودی FA دوم و غیره.

برای فهم ساده تر تصویری از این جمع کننده در زیر قرار داده شده است:



شکل 16- تصویر جمع کننده 4 بیتی به روش RIPPLE ADDER

در ادامه به تعریف این جمع کننده 8 بیتی در خود کد میپردازیم:

```

73
74 *****
75 * 8-bit Ripple-Carry Adder
76 .SUBCKT RCA8 A8 A7 A6 A5 A4 A3 A2 A1 B8 B7 B6 B5 B4 B3 B2 B1 Cin S8 S7 S6 S5 S4 S3 S2 S1 Cout vdd vss
77 XFA1 A1 B1 Cin S1 C2 vdd vss FA
78 XFA2 A2 B2 C2 S2 C3 vdd vss FA
79 XFA3 A3 B3 C3 S3 C4 vdd vss FA
80 XFA4 A4 B4 C4 S4 C5 vdd vss FA
81 XFA5 A5 B5 C5 S5 C6 vdd vss FA
82 XFA6 A6 B6 C6 S6 C7 vdd vss FA
83 XFA7 A7 B7 C7 S7 C8 vdd vss FA
84 XFA8 A8 B8 C8 S8 Cout vdd vss FA
85 .ENDS RCA8

```

شکل 17- تعریف جمع کننده 8 بیتی CRA در کد HSPICE

همان طور که میبینید برای تعریف این جمع کننده به عنوان subcircuit در کدمون اومدیم و از 8 تا FA نمونه گرفتیم و کری های خروجی رو به ورودی FA بعدی وصل کردیم. تا اینجا نحوه تعریف گیت ها و پیاده سازی اولین جمع کننده رو بیا کردیم. در ادامه به ورودی ها به شکل دستی مقدار اولیه میدهم و یک تست ساده از جمع کننده مون میگیریم تا مطمئن شویم به درستی کار میکند.

تست جمع کننده 8 بیتی RIPPLE با مقداردهی اولیه ورودی ها:

```

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec
73
74 *****
75 * 8-bit Ripple-Carry Adder
76 .SUBCKT RCA8 A8 A7 A6 A5 A4 A3 A2 A1 B8 B7 B6 B5 B4 B3 B2 B1 Cin S8 S7 S6 S5 S4 S3 S2 S1 Cout vdd vss
77 XFA1 A1 B1 Cin S1 C2 vdd vss FA
78 XFA2 A2 B2 C2 S2 C3 vdd vss FA
79 XFA3 A3 B3 C3 S3 C4 vdd vss FA
80 XFA4 A4 B4 C4 S4 C5 vdd vss FA
81 XFA5 A5 B5 C5 S5 C6 vdd vss FA
82 XFA6 A6 B6 C6 S6 C7 vdd vss FA
83 XFA7 A7 B7 C7 S7 C8 vdd vss FA
84 XFA8 A8 B8 C8 S8 Cout vdd vss FA
85 .ENDS RCA8
86
87 *****
88 * Testbench
89 Vdd vdd 0 DC 'VDD'
90 Vss vss 0 DC 0
91
92 .VEC 'DVF1.vec'
93
94 XRCA8 A8 A7 A6 A5 A4 A3 A2 A1 B8 B7 B6 B5 B4 B3 B2 B1 Cin S8 S7 S6 S5 S4 S3 S2 S1 Cout vdd vss RCA8
95
96 * Input signals
97 VA1 A1 0 DC 1
98 VA2 A2 0 DC 0
99 VA3 A3 0 DC 1
100 VA4 A4 0 DC 1
101 VA5 A5 0 DC 0
102 VA6 A6 0 DC 0
103 VA7 A7 0 DC 1
104 VA8 A8 0 DC 1
105
106 VB1 B1 0 DC 0
107 VB2 B2 0 DC 1
108 VB3 B3 0 DC 0
109 VB4 B4 0 DC 1
110 VB5 B5 0 DC 0
111 VB6 B6 0 DC 1
112 VB7 B7 0 DC 0
113 VB8 B8 0 DC 0
114
115 VCin Cin 0 DC 0
116

```

شکل 18- نمونه گیری از جمع کننده و مقدار دهی اولیه به سیگنالهای ورودی جهت بررسی صحت کارکرد مدار

خب همان طور که در تصویر بالا مشاهده میکنید ابتدا از جمع کننده 8 بیتی خودمون نمونه گرفتیم و سپس اومدیم و به هر کدوم از سیگنالهای ورودی یک مقدار اولیه دادیم. برای تعریف ولتاژ و ولتاژ ورودی ها از سینتکسی که در جزوه جناب دیوسالار وجود داشت استفاده کردیم.

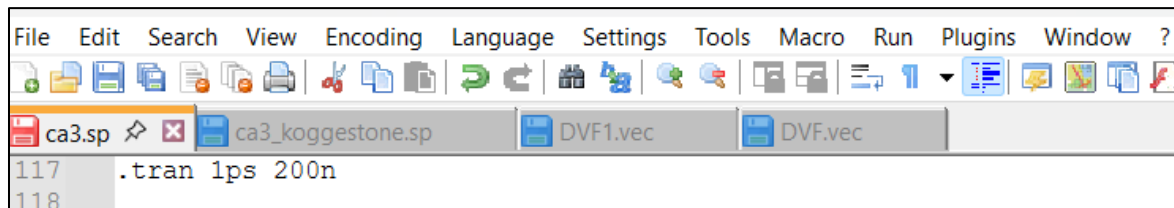
منبع ولتاژ و جریان مستقل و dc:

مقدار	N+	N-	Vx	یا	مقدار	N+	N-	Vx
مقدار	N+	N-	Ix	یا	مقدار	N+	N-	Ix

دقت کنید برای مقداردهی اولیه میتونستیم از VPULSE هم استفاده کنیم.

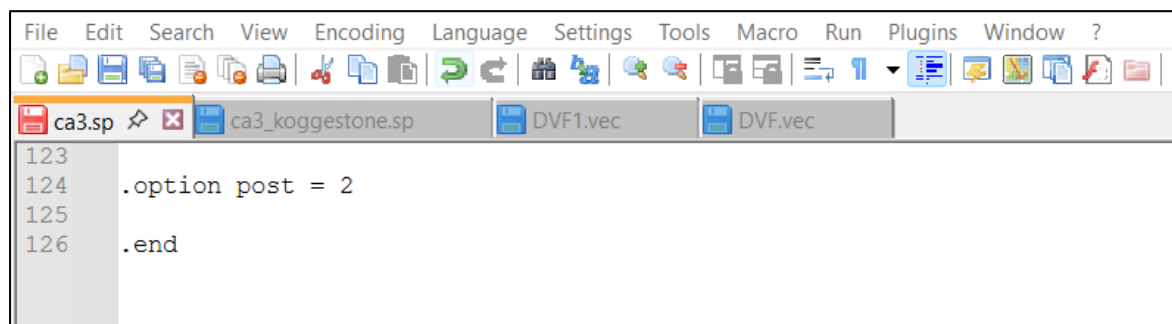
انجام شبیه سازی:

با استفاده از دستور tran. به شبیه سازی از نوع زمانی انجام میدهیم.

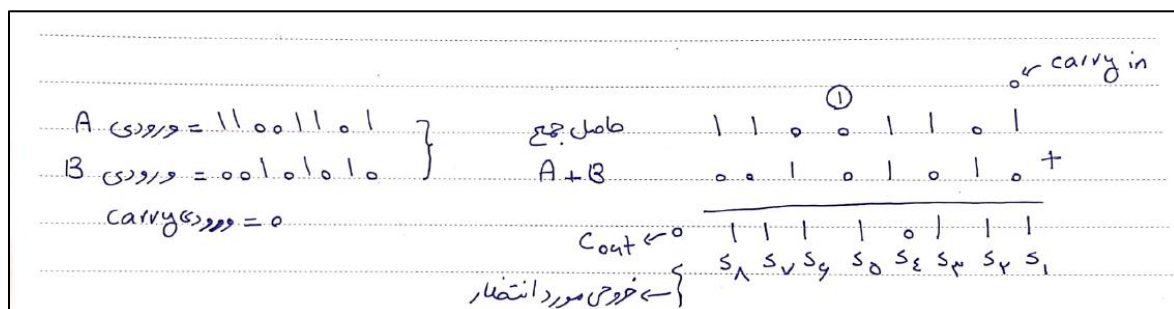


شکل 19- انجام شبیه سازی زمانی

حال باید خروجی شبیه سازی رو که در فایل CA3.tr ذخیره شده در نرم افزار CosmosScope باز کنیم و به بررسی سیگنالهای خروجی و مطابقت با خروجی مورد انتظار بپردازیم. برای اینکه خروجی شبیه سازی قابل اجرا در این نرم افزار شود از دستور option post به شکل زیر استفاده میکنیم:

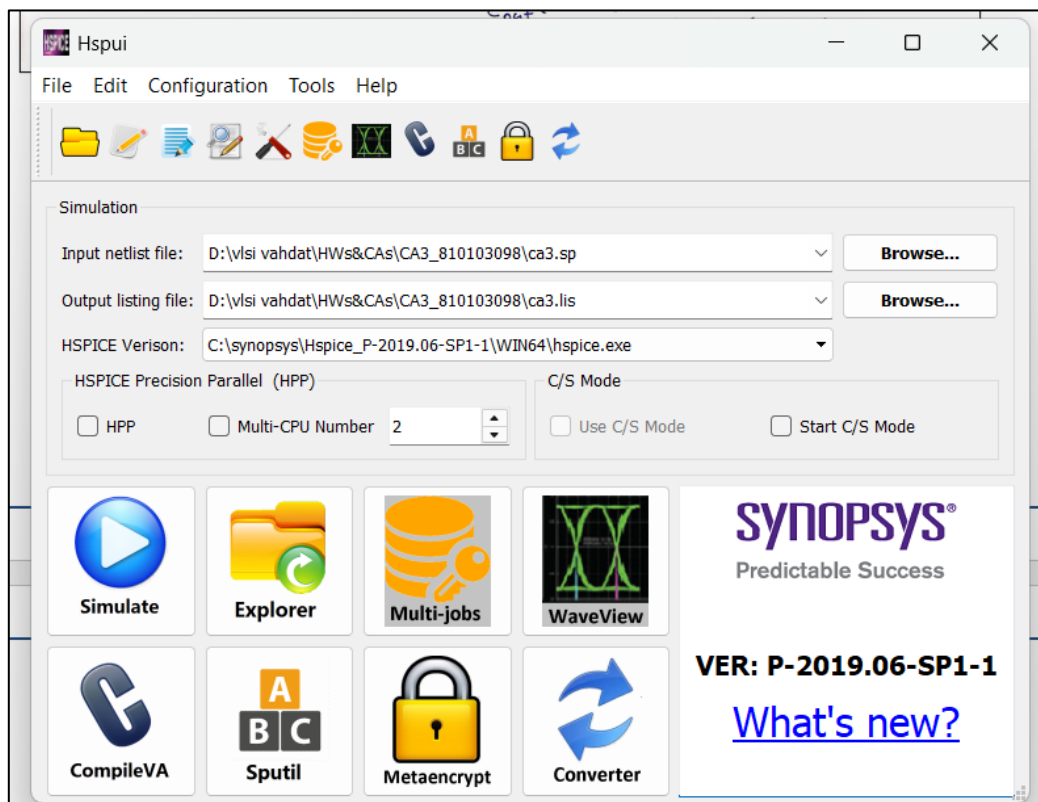


شکل 21- دستور =2 option post.



شکل 20- خروجی مورد انتظار برای ورودی های اعمال شده به جمع کننده به شکل دستی

حال شکل موج خروجی ها رو در نرم افزار CosmosScope بررسی خواهیم کرد. در صفحه بعد نتایج به دست آمده قرار داده شده است.



شکل 22- انجام شبیه سازی در نرم افزار HSPICE

```

***** job concluded
*****
*****carry ripple adder*****
***** job statistics summary tnom= 25.000 temp= 25.000 *****
***** HSPICE Threads Information *****

```

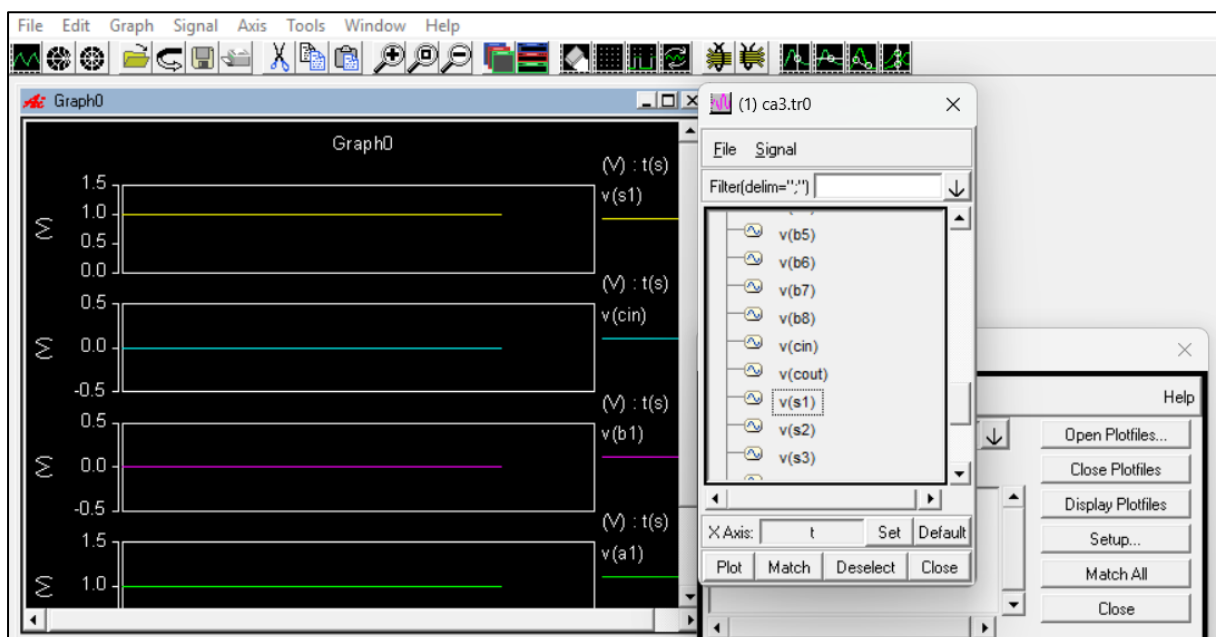
در زیر نیز خروجی فایل CA3.lis قرار داده شده است:

شکل 23- پیام job concluded به نشانه ی انجام شبیه سازی به شکل موفق

در صفحه بعد نیز شکل موج مربوط به شبیه سازی قرار داده شده است:

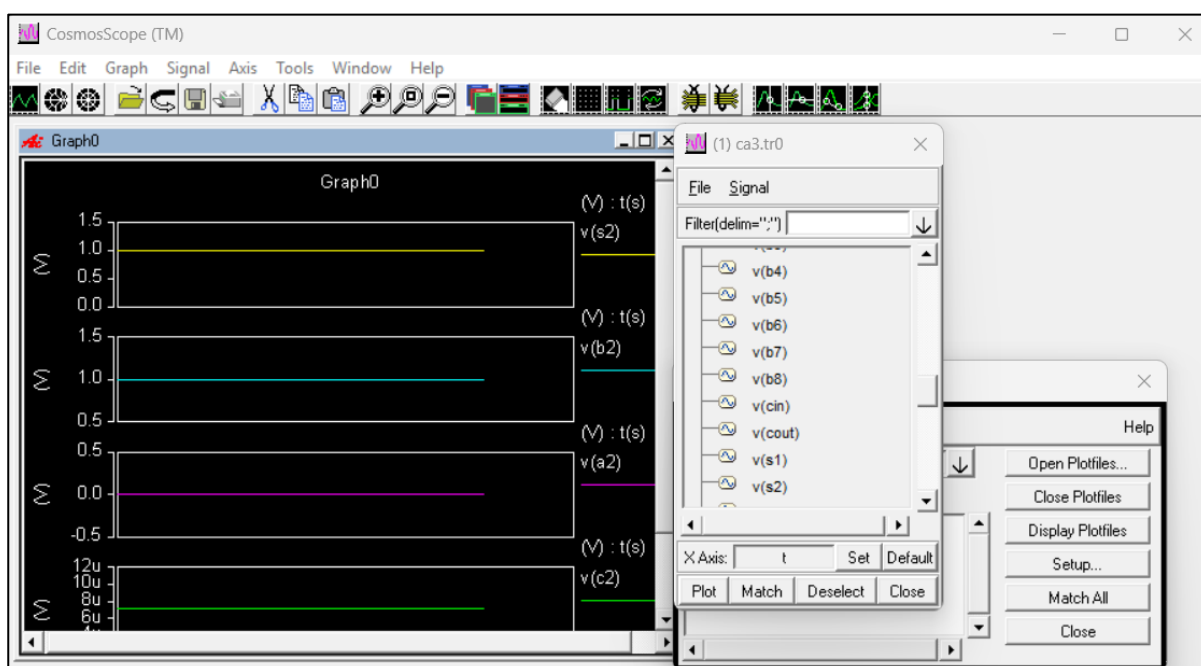
رسم شکل موج خروجی برای جمع کننده RIPPLE

برای رسم موج اول باید از گزینه CA3.tr -> plot file -> open رو انتخاب کنیم تا بتونیم ولتاژها و جریان های مختلف برای نودهای مختلف مدار رو حساب کنیم.
(نکته) ترتیب کری ها Cin و C2 و ... میباشد.



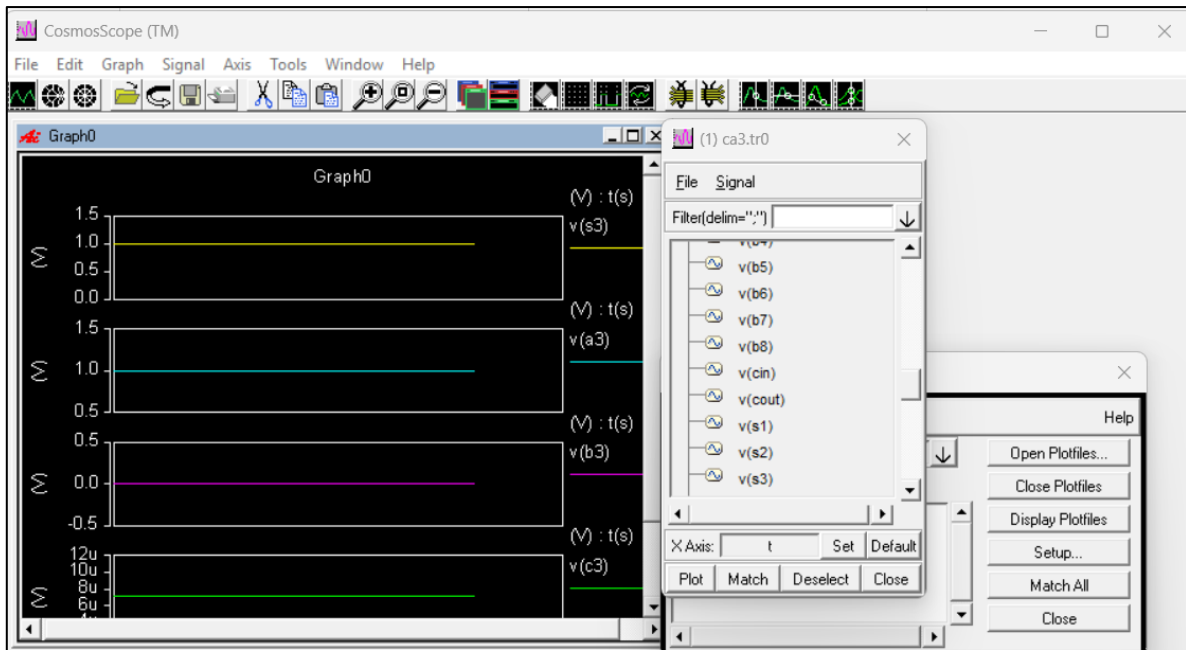
شکل 24- شکل موج مربوط به sum اول

همان طور که در تصویر بالا میبینید خروجی دقیقا مطابق انتظار شد.



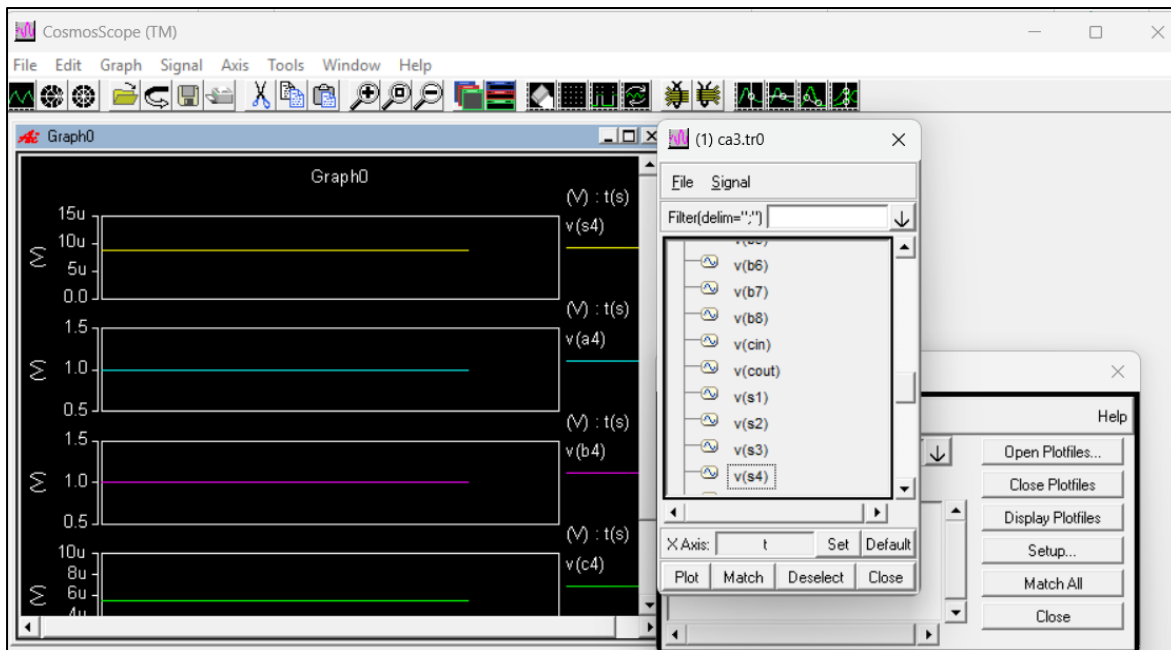
شکل 25- شکل موج مربوط به sum دوم

کری خروجی در تصویر بالا در حد میکرو ولت در اومده که همون نزدیک صفره.



شکل 26- تصویر شکل موج مربوط به سوم sum

در تصور زیر نیز خروجی sum4 قرار داده شده است:



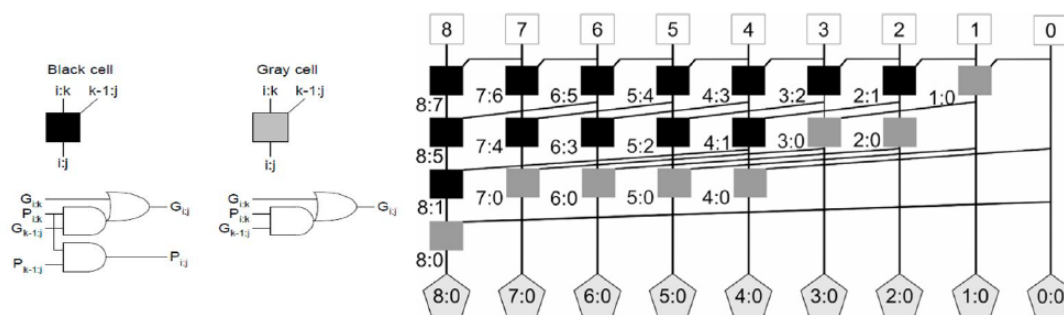
شکل 27- تصویر شکل موج خروجی S4

بقیه شکلها جهت خلاصه بودن گزارش در اینجا قرار داده نشده است.

البته لازم به ذکر است در ادامه و در بخش ج به بررسی صحت جمع کننده ها با وکتور میپردازیم.

بخش ب) پیاده سازی جمع کننده KOGGE STONE

ب) (۱۵٪) ساختار مربوط به Kogge Stone Adder ۸ بیتی را در HSPICE پیاده سازی کنید. در پیاده سازی خود حتماً به ماجول محور (ماجولار) بودن توصیف ها اهمیت دهید، زیرمدارهای مورد استفاده را بیان کرده و نحوه ی قرارگیری آنها را شرح دهید. شکل ۱ شماتیک مدار این جمع کننده را نمایش می دهد.



شکل ۱ : شماتیک مدار Kogge Stone Adder ۸ بیتی

در ابتدا برای پیاده سازی و در ادامه تست این جمع کننده یک فایل جدا با نام CA3_koggestone.sp ساختیم که کد این جمع کننده در آن قرار دارد.

خب در این فایل هم ایمپورت لایبرری و تعریف گیت های پایه مثل قبل انجام میشود لذا از آن گذر میکنیم.

همان طور که در شکل بالا هم میبینید دو بخش اصلی این جمع کننده یکی Black Box و دیگری Grey Box میباشد.

مفهوم و پیاده سازی PROPAGATE:

در اینجا propagate یا همان P به این معنی است که کری ورودی هر چی باشد عیناً به کری خروجی منتقل میشود. (دلیل: وقتی XOR دو ورودی برابر با 1 باشد یعنی یکی 1 و دیگری 0، کری ورودی هر چی باشه به خروجی منتقل میشه).

در ضمن لازم به ذکر است که مفهوم PROPAGATE و GENERATE روی کری تعریف میشن.

در زیر تصویر از ساختار propagate نیز قرار داده شده است:

در تصویر صفحه بعد همچنین میتونین مفهوم generate رو هم مشاهده کنید.

$$P_i = A_i \oplus B_i, \quad G_i = A_i B_i$$

$$S_i = P_i \oplus C_i, \quad C_{i+1} = G_i + P_i C_i$$

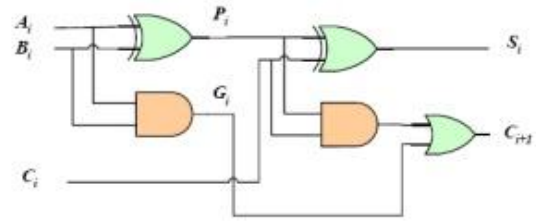
Therefore:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1(G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$$

Similarly:

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$



شکل 28- مفهوم generate و propagate

خب حالا Generate چی می‌گه؟ می‌گه اگه AND هر دو ورودی برابر 1 بود کاری به کری مرحله قبل نداشته باش و کری خروجی برابر با یک میشه.

پیاده سازی بلاک های جمع کننده kogge stone:

در این قسمت به پیاده سازی و توضیح بلاک های مختلف برای پیاده سازی این جمع کننده میپردازیم:

بلاک pre processing:

در ابتدا و قبل از انجام هر محاسبه ای باید مقدار P و G رو برای همه ی ورودی ها محاسبه کنیم جهت اینکار اومدیم و در کد یک سابسرکت تعریف کردیم که بعدا ازش نمونه بگیریم:

```
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec
78
79 * Pre-Processing Block - Generates initial P and G signals
80 .SUBCKT PRE_PROCESS A B P G vdd gnd
81 XU1 A B P vdd gnd XOR2
82 XU2 A B G vdd gnd AND2
83 .ENDS PRE_PROCESS
84
```

شکل 29- بلاک محاسبه P و G برای همه ی ورودی ها در اولین مرحله جمع کننده kogge stone

بلاک Black Cell:

خب تصویر این بلوک در صورت سوال نیز موجود است لذا به شماتیک آن نمی‌پردازیم و به پیاده سازی آن در کد می‌پردازیم، تصویر پیاده سازی در زیر قرار داده شده است:

```
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec
63 * Black Cell (BC) - Group PG Logic
64 .SUBCKT BLACK_CELL Gi_j Pi_j Gk_i_1 Pk_i_1 Gi_k Pi_k vdd gnd
65 * Gi_j = Gi_k + (Pi_k * Gk_i_1)
66 * Pi_j = Pi_k * Pk_i_1
67 XU1 Pi_k Gk_i_1 and_out vdd gnd AND2
68 XU2 Gi_k and_out Gi_j vdd gnd OR2
69 XU3 Pi_k Pk_i_1 Pi_j vdd gnd AND2
70 .ENDS BLACK_CELL
```

شکل 30- تعریف بلوک Black Cell به شکل سابسرکت در کد

به شکل کوتاه برای ورودی های این سل ما P و G همین level رو می‌خوایم و P و G استیج قبل تا بتونیم خروجی که ترکیب شده هر دو هست رو تولید کنیم.

بلاک Grey Cell:

در پیاده سازی این بلوک به نحوه زیر و طبق تصویر قرار داده شده در صورت سوال می‌پردازیم:

```
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec
72 * Grey Cell (GC) - Only generates group generate signals
73 .SUBCKT GREY_CELL Gi_j Gk_i_1 Pk_i Gi_k vdd gnd
74 * Gi_j = Gi_k + (Pk_i * Gk_i_1)
75 XU1 Pk_i Gk_i_1 and_out vdd gnd AND2
76 XU2 Gi_k and_out Gi_j vdd gnd OR2
77 .ENDS GREY_CELL
```

شکل 31- تعریف Grey Cell به شکل سابسرکت در کد

خب برای محاسبه این مورد به G همون مرحله و مرحله قبل و P همون مرحله نیاز داریم تا حاصل بشه G تجميع شده مثلا:

$$G_{3:2} = G_3 + P_3 G_2$$

از این بلوک بعدا برای پیاده سازی طبقات مختلف جمع کننده مون استفاده خواهیم کرد.

پیاده سازی SUM block:

در این بخش می‌خوایم نحوه محاسبه sum های انتهایی در جمع کننده kogge stone رو بررسی کنیم و سابسرکتی بنویسیم که بعدا ازش نمونه بگیریم برای محاسبه sum ها.

```

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec
84
85 * Sum Block - Computes final sum bit
86 .SUBCKT SUM_BLOCK P C S vdd gnd
87 XU1 P C S vdd gnd XOR2
88 .ENDS SUM_BLOCK
89

```

شکل 32- تعریف sum block به عنوان سابسرکت در کد

برای محاسبه sum از فرمول زیر همان طور که به یاد داریم استفاده میکنیم:

$$S = A \oplus B \oplus C = P \oplus C$$

خب میدونستیم که sum از XOR سه ورودی بدست میاد حالا XOR دو بیت A و B که همون بیت های ورودی ما هستند همون مقدار P رو به ما میده و با جایگذاری در فرمول بالا به جواب آخر رسیدیم و در بلوک تعریف شده در کد هم دو بیت ورودی A و B که XOR شدن (همون P) رو با کری ورودی XOR میکنیم تا SUM بدست بیاد.

نکته مهم: در ضمن در پیاده سازی جمع کننده باید بدونیم که G3:0 همون C3 یا کری واقعی تولید شده میباشد.

تعریف جمع کننده KOGGE STONE:

خب در این مرحله قرار است از بلوک هایی که در مرحله قبل تعریف کردیم استفاده کنیم و جمع کننده KOGGE STONE خودمون رو بسازیم.

مرحله اول pre processing:

در ابتدا باید مقادیر P و G برای هر دو جفت از ورودی ها حساب شود، در زیر کد مربوط به آن را میتوانید مشاهده کنید:

```

93 * Pre-Processing Stage - Generate initial P and G signals
94 XPP1 A1 B1 P1 G1 vdd gnd PRE_PROCESS
95 XPP2 A2 B2 P2 G2 vdd gnd PRE_PROCESS
96 XPP3 A3 B3 P3 G3 vdd gnd PRE_PROCESS
97 XPP4 A4 B4 P4 G4 vdd gnd PRE_PROCESS
98 XPP5 A5 B5 P5 G5 vdd gnd PRE_PROCESS
99 XPP6 A6 B6 P6 G6 vdd gnd PRE_PROCESS
100 XPP7 A7 B7 P7 G7 vdd gnd PRE_PROCESS
101 XPP8 A8 B8 P8 G8 vdd gnd PRE_PROCESS

```

شکل 33- مرحله pre processing در جمع کننده kogge stone adder

مرحله دوم KOGGE STONE:

در این مرحله ترکیب دو تایی جایگاه های ورودی رو داریم و از یک GREY CELL و 7 تا BLACK CELL طبق چیدمان تصویر قرار داده شده برای پیاده سازی در سوال استفاده میکنیم، نحوه تعریف و ورودی های هر cell طبق صورت سوال میباشد و میتونید در تصویر زیر مشاهده کنید:

```
103 * Handle Cin by creating G0 = Cin
104 * Grey Cell to handle Cin for position 1
105 XGC0 C1 Cin P1 G1 vdd gnd GREY_CELL
106
107 * Stage 1: Compute spans of 2
108 * Black Cells for stage 1
109 XBC1_1 G2_1 P2_1 G1 P1 G2 P2 vdd gnd BLACK_CELL
110 XBC1_2 G3_2 P3_2 G2 P2 G3 P3 vdd gnd BLACK_CELL
111 XBC1_3 G4_3 P4_3 G3 P3 G4 P4 vdd gnd BLACK_CELL
112 XBC1_4 G5_4 P5_4 G4 P4 G5 P5 vdd gnd BLACK_CELL
113 XBC1_5 G6_5 P6_5 G5 P5 G6 P6 vdd gnd BLACK_CELL
114 XBC1_6 G7_6 P7_6 G6 P6 G7 P7 vdd gnd BLACK_CELL
115 XBC1_7 G8_7 P8_7 G7 P7 G8 P8 vdd gnd BLACK_CELL
116
```

شکل 34- مرحله دوم پیاده سازی KOGGE STONE ADDER

دقت کنید که تا اینجا کری C1 و Cin تولید شده اند.

مرحله یا استیج سوم پیاده سازی KOGGE STONE:

در این قسمت ترکیب چهارتایی بخش ها رو داریم که برای پیاده سازی طبق تصویر داده شده به دو GREY CELL و پنج تا BLACK CELL استفاده می کنیم و کد مربوط به این بخش رو در زیر میتونید مشاهده کنید:

در واقع در اینجا خروجی های دوتایی مرحله قبل گرفته شده و با هم ادغام میشن.

```
117 * Stage 2: Compute spans of 4
118 * Grey Cell for C2
119 XGC2_1 C2 G1 P2_1 G2_1 vdd gnd GREY_CELL
120 * Grey Cell for C3
121 XGC2_2 C3 G2_1 P3_2 G3_2 vdd gnd GREY_CELL
122 * Black Cells for stage 2
123 XBC2_3 G4_1 P4_1 G2_1 P2_1 G4_3 P4_3 vdd gnd BLACK_CELL
124 XBC2_4 G5_2 P5_2 G3_2 P3_2 G5_4 P5_4 vdd gnd BLACK_CELL
125 XBC2_5 G6_3 P6_3 G4_3 P4_3 G6_5 P6_5 vdd gnd BLACK_CELL
126 XBC2_6 G7_4 P7_4 G5_4 P5_4 G7_6 P7_6 vdd gnd BLACK_CELL
127 XBC2_7 G8_5 P8_5 G6_5 P6_5 G8_7 P8_7 vdd gnd BLACK_CELL
```

شکل 35- مرحله سوم پیاده سازی KOGGE STONE

در خروجی این بخش کری های C2 و C3 هم تولید می شوند.

مرحله چهارم پیاده سازی KOGGE STONE:

در این بخش خروجی های مرحله قبل که ترکیبات چهارتایی از استیج ها بود با هم ترکیب و به شکل هشت تایی در میان.

برای پیاده سازی این بخش هم طبق صورت سوال به 4 تا GREY CELL و یک BLACK CELL نیاز داریم. لازم به ذکر است که خروجی های GREY CELL ها کری های واقعی مدار ما میباشند.

خروجی این مرحله کری های C4 و C5 و C6 و C7 میباشند.

تصویر مربوط به این مرحله از کد رو نیز میتونید در زیر مشاهده کنید:

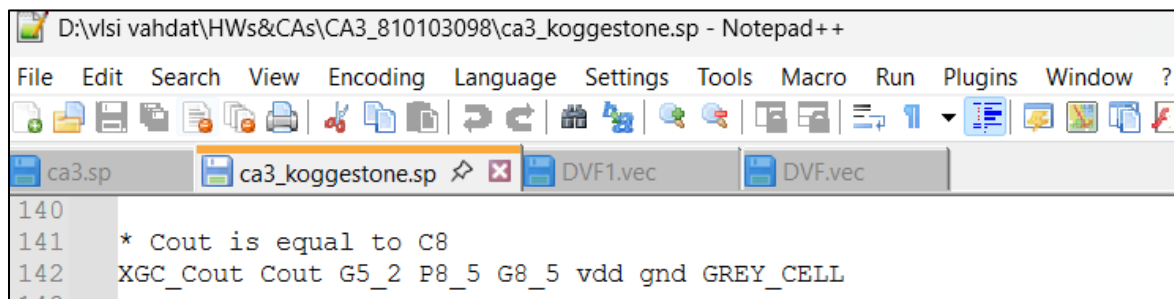
```
28
29 * Stage 3: Compute spans of 8 and final carries
30 * Grey Cell for C4
31 XGC3_1 C4 G1 P4_1 G4_1 vdd gnd GREY_CELL
32 * Grey Cell for C5
33 XGC3_2 C5 G2_1 P5_2 G5_2 vdd gnd GREY_CELL
34 * Grey Cell for C6
35 XGC3_3 C6 G3_2 P6_3 G6_3 vdd gnd GREY_CELL
36 * Grey Cell for C7
37 XGC3_4 C7 G4_1 P7_4 G7_4 vdd gnd GREY_CELL
38 * Grey Cell for C8
39 XGC3_5 C8 G5_2 P8_5 G8_5 vdd gnd GREY_CELL
40
```

شکل 36- مرحله چهارم پیاده سازی KOGGE STONE

تولید کری خروجی در KOGGE STONE:

در مرحله پنجم پیاده سازی طبق شکل سوال نیز به کری خروجی میرسیم که توسط یک GREY CELL پیاده سازی شده است.

تصویر مربوط به این مورد رو هم میتونید در زیر مشاهده کنید:



```
D:\vlsi vahdat\HWs&CAs\CA3_810103098\ca3_koggestone.sp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec
140
141 * Cout is equal to C8
142 XGC_Cout Cout G5_2 P8_5 G8_5 vdd gnd GREY_CELL
143
```

شکل 37- تولید کری خروجی در جمع کننده KOGGE STONE ADDER

در مرحله آخر پیاده سازی جمع کننده به محاسبه SUM ها میپردازیم.

محاسبه SUM ها در جمع کننده KOGGE STONE:

در این بخش به محاسبه SUM ها توسط نمونه گرفتن از سابسرکت تعریف شده و طبق تصویر جمع کننده سوال میپردازیم:

```
D:\vlsi vahdat\HWS&CAs\CA3_810103098\ca3_koggestone.sp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec
143
144 * Calculate sum bits
145 * S_i = P_i ⊕ C_(i-1)
146 XS1 P1 Cin S1 vdd gnd SUM_BLOCK
147 XS2 P2 C1 S2 vdd gnd SUM_BLOCK
148 XS3 P3 C2 S3 vdd gnd SUM_BLOCK
149 XS4 P4 C3 S4 vdd gnd SUM_BLOCK
150 XS5 P5 C4 S5 vdd gnd SUM_BLOCK
151 XS6 P6 C5 S6 vdd gnd SUM_BLOCK
152 XS7 P7 C6 S7 vdd gnd SUM_BLOCK
153 XS8 P8 C7 S8 vdd gnd SUM_BLOCK
154
155 .ENDS KSA8
156
```

شکل 38- محاسبه SUM ها در جمع کننده KOGGE STONE

خب تا اینجا تمامی بخش های جمع کننده KOGGE STONE رو طبق شکل سوال پیاده سازی کردیم و تعریف جمع کننده اینجا به پایان میرسد.

در ادامه و قبل از تست این جمع کننده باید از آن نمونه بگیریم و مقادیر vdd و gnd رو هم تعریف کرد.

```
156
157 * Main circuit - Power supplies and connections
158 Vdd vdd 0 DC Vdd
159 Vgnd gnd 0 DC 0
160
161 .VEC 'DVF.vec'
162
163 * Instantiate the 8-bit Kogge Stone Adder
164 XKSA A8 A7 A6 A5 A4 A3 A2 A1 B8 B7 B6 B5 B4 B3 B2 B1 Cin S8 S7 S6 S5 S4 S3 S2 S1 Cout vdd gnd KSA8
165
```

شکل 39- تعریف main circuit برای kogge stone adder در فایل CA3_koggestone.sp

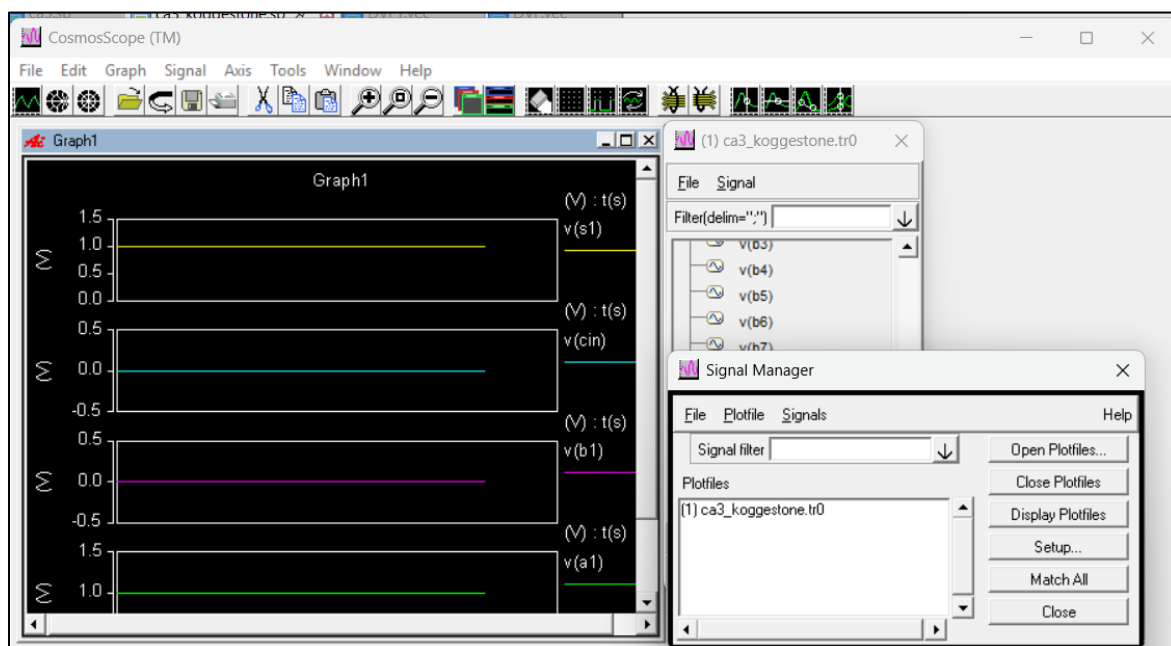
تست جمع کننده KOGGE STONE با ورودی های دستی:

به مانند جمع کننده قبلی و قبل از تست آن ها از طریق وکتور میایم و خروجی جمع کننده رو برای یک ورودی ثابت دستی بررسی میکنیم و از طریق نرم افزار CosmosScope به بررسی شکل موج خروجی ها میپردازیم.

در صفحه بعد میتونید شکل موج خروجی ها رو مشاهده کنید.

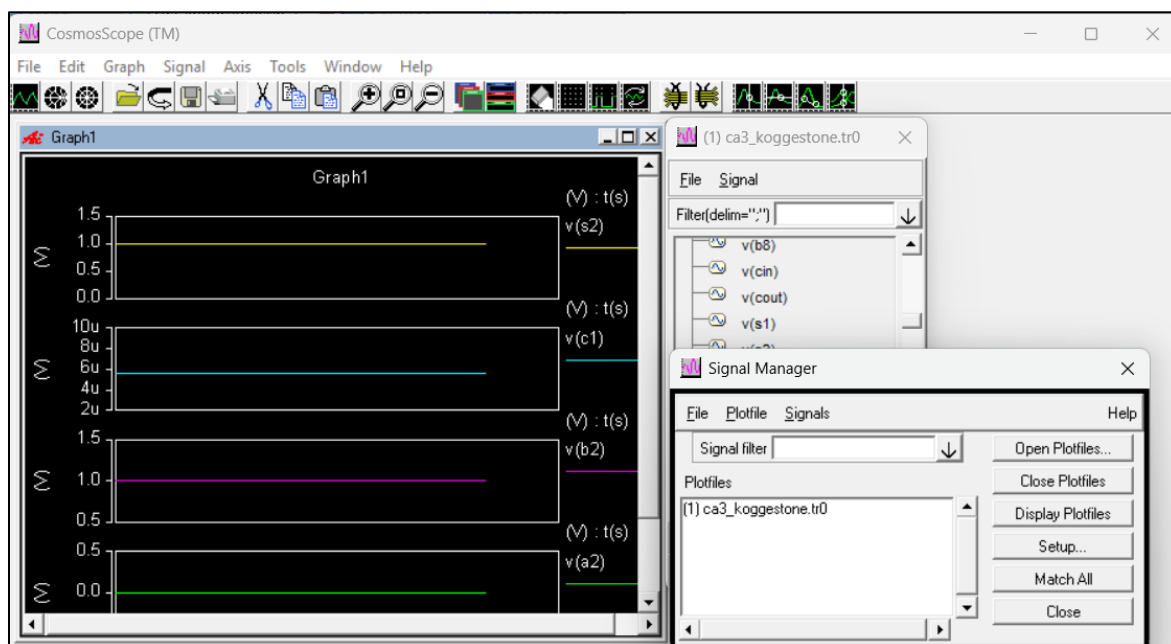
نتایج انجام شبیه سازی برای تست ورودی های دستی:

در زیر تصاویر خروجی شکل موج برای هر کدام از خروجی های sum برحسب ورودی ها رو میتونید مشاهده کنید:



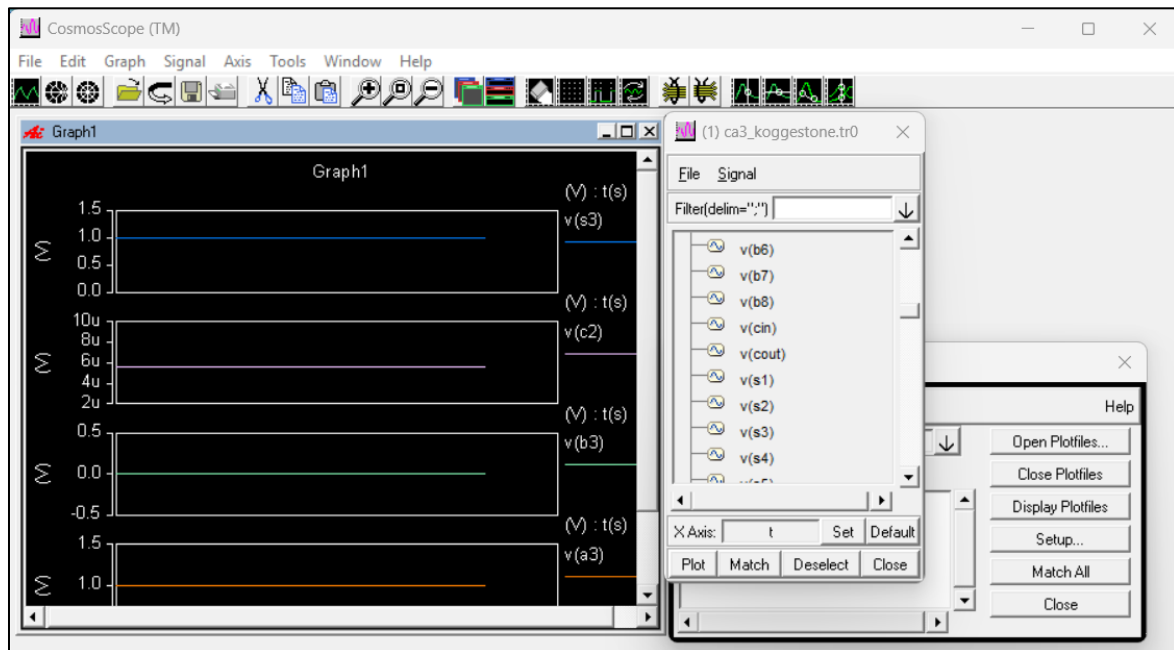
شکل 40- خروجی sum اول برای ورودی دستی داده شده به جمع کننده kogge stone

مثالی که در اینجا مورد بررسی قرار گرفته همان مثال جمع کننده قبل است که البته در کد HSPICE این جمع کننده هم موجود است.



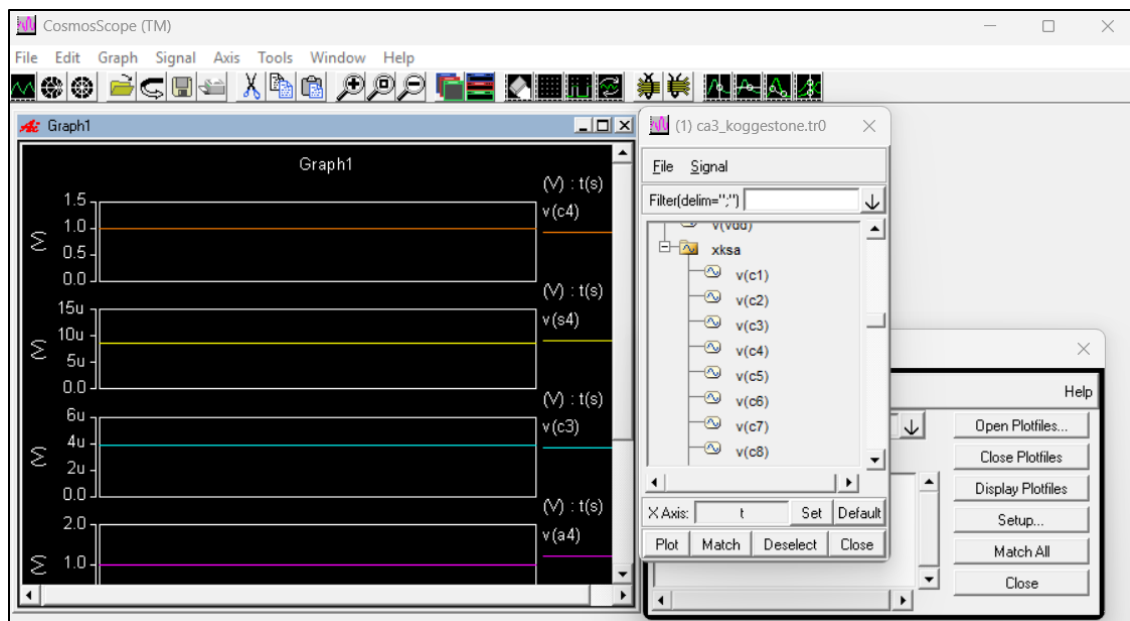
شکل 41- خروجی sum دوم برای ورودی دستی داده شده به جمع کننده kogge stone

دقت کنید که در اینجا برعکس جمع کننده قبل کری ها به ترتیب $C_{in}, C_1, C_2, \dots, C_8$ هستند در حالی که جمع کننده قبلی $C_{in}, C_2, \dots, C_{out}$ داشتیم.



شکل 42- تصویر مربوط به sum سوم جمع کننده kogge stone

در ادامه به آخرین شکل موج از این جمع کننده میپردازیم که قصد داریم در این گزارش به آن بپردازیم و بقیه خروجی ها بررسی شده ولی جهت خلاصه کردن گزارش کار از قرار دادن آنها خودداری کردیم.



شکل 43- خروجی مربوط به sum چهارم جمع کننده kogge stone برای ورودی دستی داده شده

تا اینجا به شکل دستی به سیگنالهای ورودی هر دو جمع کننده ورودی دادیم و صحت عملکرد آنها را با بررسی از روی شکل موج بررسی کردیم. در قسمت بعد قرار است با استفاده از وکتور به بررسی صحت عملکرد دو جمع کننده بپردازیم.

ج) بررسی صحت عملکرد دو جمع کننده با وکتور

ج) (۲۰٪) پس از توصیف دو جمع کننده ۸ بیتی، می‌خواهیم از صحت عملکرد آنها مطمئن شویم. برای این امر، نیازمند کار با Digital Vector File (DVF) هستیم. به کمک این فایل برداری می‌توان ورودی‌های مدار را در طول زمان مشخص کرد و همچنین نتیجه‌ی مورد انتظار از مدار را بررسی کرد. در ادامه توضیح مختصری در مورد این نوع فایل ارائه می‌شود. DVF شامل سه بخش اصلی است:

۱. تعریف الگوی بردار (Vector Pattern Definition): در این بخش باید نام بردارها، نوع آنها (ورودی یا خروجی) و همچنین اینکه مقدار این بردارها بر چه مبنای عددی (radix) محاسبه گردد، مشخص شود. این بخش در ابتدای فایل قرار می‌گیرد. در این بخش می‌توان به صورت برداری نام سیگنال‌ها را مشخص کرد. به بیان دیگر اگر در نتلیست HPSICE خود S1, S7, ..., S8 داشته باشید، در اینجا می‌توانید به صورت S[8:1] به آن اشاره نمایید. یک مثال برای این بخش به صورت زیر است:

```
; 8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout
radix      44  44  1   44  1
; define name for each vector
vname      A[8:1]  B[8:1]  cin S[8:1]  cout
; define IO
IO  II  II  I   00  0
```

۲. مشخصه‌های شکل موج (Characteristic Waveform): در این بخش، ویژگی‌های مختلف سیگنال، از جمله واحد زمانی مورد استفاده در فایل، rise time و fall time، آستانه‌های ولتاژی منطق high و low و موارد دیگری تعیین می‌شوند. یکی از مهم‌ترین ویژگی‌های قابل تنظیم در این بخش، تأخیر است. همانطور که در مثال زیر مشخص شده است، این مورد به کمک یک mask عمل می‌کند که در صورت یک بودن مقدار متناظر با یکی از سیگنال‌ها، تأخیر مشخص شده برای این سیگنال اعمال می‌شود. این کار می‌تواند باعث شود که بررسی مقدار خروجی‌ها با تأخیر نسبت به اعمال ورودی‌ها انجام پذیرد.

```
; define waveform characteristics
tunit      ps
Period      600
tdelay      588.0      00  00  0   FF  1
Slope       50
VOH         '0.8*Vdd'
```

خب در ابتدا به فایل پایتون با نام generate_DVF.py میسازیم که بر اساس پارامتر num_samples بیاد و طبق الگوی داده شده در صفحه قبل برای ما فایل DVF1.vec رو با تعداد 1000 تست کیس به شکل تصادفی بسازد که از این فایل بعدا برای تست هر دو جمع کننده استفاده خواهیم کرد.

```

1 def generate_dvf(num_samples=1000, filename="DVF1.vec"):
2
3     # بخش 1: تعریف الگوی بردار
4     with open(filename, 'w') as f:
5         f.write(";8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout \n")
6         f.write("radix 44 44 1 44 1 \n")
7         f.write(";define name for each vector \n")
8         f.write("vname A[8:1] B[8:1] cin S[8:1] cout \n")
9         f.write(";define IO \n")
10        f.write("IO II II I 00 0 \n\n")
11
12    # بخش 2: مشخصات شکل موج
13    f.write(";define waveform characteristics \n")
14    f.write("tunit ps \n")
15    f.write("Period 5000 \n")
16    f.write("tdelay 4900.0 00 00 0 FF 1\n")
17    f.write("Slope 50 \n")
18    f.write("VOH '0.8*vdd'\n")
19    f.write("VOL '0.2*vdd'\n\n")
20
21    # بخش 3: داده‌های جدولی
22    f.write("; tabular data\n")
23
24    # تولید داده‌های تصادفی
25    for _ in range(num_samples):
26        a = random.randint(0, 255) # عدد تصادفی 8 بیتی برای A
27        b = random.randint(0, 255) # عدد تصادفی 8 بیتی برای B
28        cin = random.randint(0, 1) # نقی ورودی تصادفی
29
30        # محاسبه خروجی مورد انتظار
31        sum_result = (a + b + cin) & 0xFF # 8 بیت کم ارزش جمع
32        cout = 1 if (a + b + cin) > 255 else 0 # نقی خروجی
33
34        # نوشتن داده در فایل (هگزادسیمال)
35        f.write(f"{a:02x} {b:02x} {cin} {sum_result:02x} {cout}\n")
36
37    print(f"فایل DVF با {num_samples} نمونه در {filename} ایجاد شد.")
38    # تولید فایل با 1000 نمونه
39    generate_dvf(1000)
  
```

Interactive - generate_DVF.py X

Connected to Python 3.12.7

```
def generate_dvf(num_samples=1000, filename="DVF1.vec"): ...
```

```
def generate_dvf(num_samples=1000, filename="DVF1.vec"): ...
```

ایجاد شد DVF1.vec با 1000 نمونه در DVF فایل.

No kernel connected

No kernel connected

Press **Enter** to execute.

شکل 44- فایل پایتون برای ایجاد وکتور فایل جهت ایجاد تست کیس های تصادفی

حال بعد از اجرای این فایل در محیط VScode یک فایل وکتور برای ما ساخته میشود که در ادامه به جزئیات آن خواهیم پرداخت.

خب در ابتدای فایل DVF.vec باید radix قرار داشته باشد که با می‌گوید که فرمت هر یک از سیگنال ها به چه شکله الان برای سیگنال ورودی A ما 44 رو داریم که نشون میده این ورودی 8 بیتی بوده ولی در اینجا با فرمت Hexadecimal نشان داده می‌شود. مثلا 1 نشان دهنده کی بیت باینری میباشد. بعد از نوبت به تعریف کردن اسم سیگنال ها میشود.

```

1 ;8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout
2 radix 44 44 1 44 1
3 ;define name for each vector
4 vname A[8:1] B[8:1] Cin S[8:1] Cout
5 ;define IO
6 IO II II I OO O

```

شکل 45- تعریف Radix و سیگنالهای ورودی در فایل وکتور

همان طور که در تصویر بالا میبینید با دستور vname ترتیب ورودی ها معین شده که بعداً در ادامه برای هر کدام از این سیگنالهای ورودی نوع ورودی یا خروجی بودن تعریف شده است. مثلاً چون سیگنال A از 8 بیت و به شکل هگزادسیمال است و همچنین در جمع کننده به عنوان ورودی میباید اینجا از II برای تعریفش استفاده کردیم که I=input signal.

```

7
8 ;define waveform characteristics
9 tunit ps
10 Period 5000
11 tdelay 4900.0 00 00 0 FF 1
12 Slope 50
13 VOH '0.8*Vdd'
14 VOL '0.2*Vdd'
15

```

شکل 46- تعریف مقادیر پریود و موارد مربوط به تاخیر سیگنالها در فایل وکتور

همان طور که در تصویر بالا مشاهده میکنید طبق خواسته سوال ما تاخیر TRISE و TFALL ورودی ها رو برابر با 50ps گرفتیم. دقت کنید با نوشتن دستور tunit ps واحد های زمانی بر حسب ps محاسبه میشوند.

مورد بعدی مربوط به period است که میگه چقدر وایستم و بعدش ورودی های بعدی رو به مدار وارد کنم باید در اولین شبیه سازی چون تعداد تست کیس ها زیاد است این مقدار رو عدد بزرگی در order نانو ثانیه بگیریم تا خروجی های مربوط به هر ورودی تست کیس تولید شوند و فرصت کافی برای انجام محاسبات توسط جمع کننده ها باشد.

مورد بعدی برای tdelay میباشد که میگه بعد از اعمال ورودی ها چقدر وایستم و خروجی ها رو بخونم قانداً این هم عددی نزدیک به پریود باید باشه و باید به اندازه کافی وایستم تا خروجی های جمع کننده ثابت و valid بشن و بعدش اونها رو بخونیم.

در مورد ولتاژ ها هم که در صورت سوال و فایل ضمیمه شده توضیح داده شده است.

```

15
16 ; tabular data
17 24 69 1 8e 0
18 85 68 1 ee 0
19 ec 65 0 51 1
20 16 03 1 1a 0
21 78 94 1 0d 1
22 66 af 1 16 1
23 cc e4 1 b1 1
24 3e 46 0 84 0
25 85 42 0 c7 0
26 7a 94 0 0e 1
27 e0 95 1 76 1
28 4b 09 1 55 0
29 5f cc 0 2b 1
30 da 1b 1 f6 0
31 74 4d 1 c2 0
32 a2 b1 0 53 1
33 53 8d 1 e1 0
34 c4 81 1 46 1
35 27 6a 0 91 0
36 2e 32 0 60 0
37 e0 ee 0 ce 1
38 15 18 0 2d 0
39 05 43 0 48 0
40 e6 69 0 4f 1
41 64 57 0 bb 0
42 ea 9f 0 89 1
43 33 82 1 b6 0
44 d0 be 0 8e 1
45 4c af 0 fb 0
46 0d ce 1 dc 0
47 3c 83 0 bf 0
48 69 c3 0 2c 1
49 99 ed 0 86 1
50 f5 01 0 f6 0
51 38 fc 0 34 1
52 ce 73 1 42 1
53 e3 cc 1 b0 1

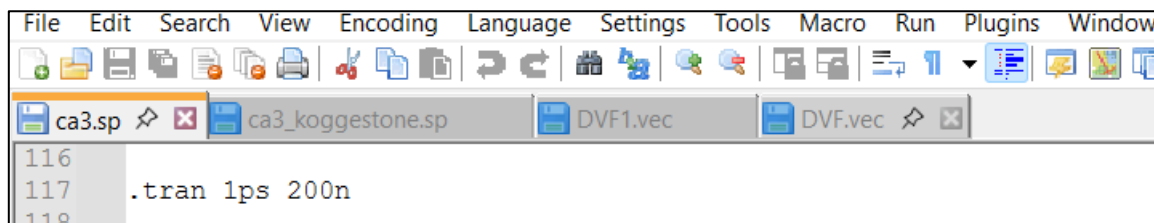
```

شکل 47- نمونه ای از 1000 تست کیس تولید شده برای جمع کننده
CRA

خب حال باید از طریق دستور 'DVF1.vec' VEC. در کد اصلی جمع کننده ripple adder کاری کنیم که مقادیر ورودی در وکتور فایل به جمع کننده وارد شود و عملکرد آن مورد بررسی قرار بگیرد.

تست جمع کننده CRA با وکتور فایل:

در این بخش بعد از انجام شبیه سازی برای ما یک فایل با پسوند CA3.err تولید میشود که میاد و تست کیس های نوشته شده در وکتور فایل رو با خروجی شبیه سازی شده در HSPICE برای جمع کننده مقایسه میکنه. تنظیمات شبیه سازی و تاخیر سیگنالها در بخش قبلی مورد بحث قرار گرفت.



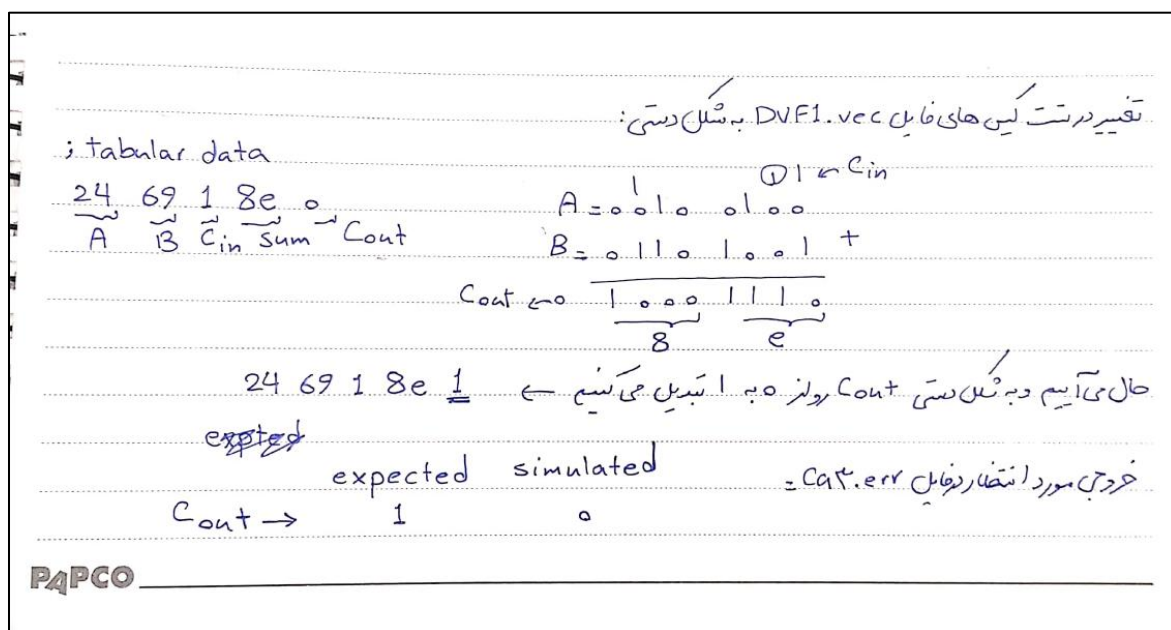
شکل 48- انجام شبیه سازی برای CRA ADDER با دستور .tran

همان طوری که میبینید با این دستور یک شبیه سازی از نوع زمانی یا TRANSIENT انجام می شود. در زیر میتونید خروجی فایل CA3.ERR رو هم مشاهده کنید:



شکل 49- خروجی فایل ca3.err پس از انجام شبیه سازی روی 1000 نمونه تست تصادفی

خب همان طور که میبینید این فایل خالی است و این نشان میدهد که به درستی خروجی تمام شبیه سازی های انجام شده برابر با مقادیری است که expected value ما بوده اند. خب حال طبق خواسته سوال باید یکی از تست کیس ها رو به شکل دستی عوض کنیم تا ببینیم در فایل ارور ظاهر میشود یا نه و از درست بودن عملکرد جمع کننده خود مطمئن شویم.



شکل 50- تغییر به شکل دستی یکی از تست کیس ها

حال در زیر تصویر بیت تغییر کرده در فایل DVF1.vec رو میتونید مشاهده کنید:

```

7
8 ;define waveform characteristics
9 tunit ps
10 Period 5000
11 tdelay 4900.0 00 00 0 FF 1
12 Slope 50
13 VOH '0.8*Vdd'
14 VOL '0.2*Vdd'
15
16 ; tabular data
17 24 69 1 8e 1
18 85 68 1 ee 0
19 ec 65 0 51 1
20 16 03 1 1a 0
  
```

شکل 51- تغییر بیت کری خروجی برای اولین تست کیس فایل DVF1.vec

حال دوباره شبیه سازی انجام میدهیم و خروجی فایل ca3.err رو بررسی می‌کنیم:

File	Size
ca3.err0	1 KB
ca3.ic0	9 KB
IC0 File	35 KB
LIS File	1 KB
MT0 File	4 KB
PA0 File	4 KB
SP File	4 KB
ST0 File	4 KB
TR0 File	11,498 KB
Microsoft Word D...	8,635 KB
ERR0 File	1 KB
IC0 File	13 KB
LIS File	47 KB
MT0 File	1 KB
PA0 File	8 KB
SP File	6 KB
ST0 File	4 KB
TR0 File	138 KB
L File	3,679 KB
Python Source File	2 KB
VEC File	14 KB
VEC File	14 KB
Python Source File	2 KB

Time	Signal	Simulated	Expected
4.900000e-09	Cout	0	1

شکل 52- خروجی فایل ca3.err پس از انجام شبیه سازی و تغییر دستی یک بیت تست کیس

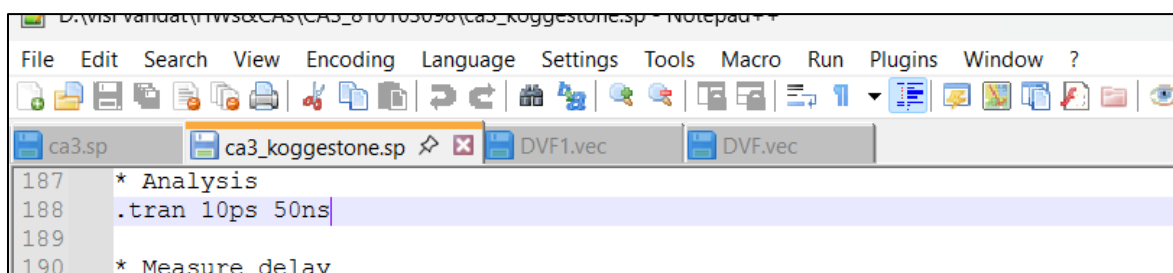
خب همان طور که در صفحه قبل هم توضیح داده شد انتظار داشتیم تا در فایل ca3.err ارور مربوط به کری خروجی ثبت شود چرا که خودمون به شکل دستی در فایل وکتور این بیت رو عوض کرده بودیم. با این تست ها به شکل کامل عملکرد جمع کننده RIPPLE رو اندازه گیری کردیم، در قسمت بعد به بررسی جمع کننده KOGGE STONE میپردازیم.

تست جمع کننده KOGGE STONE با وکتور فایل:

در ابتدا یک فایل پایتون دیگه با نام DVF.py برای این جمع کننده میسازیم و آن را اجرا میگیریم تا مثل جمع کننده قبلی یک فایل شامل وکتور ها و تست کیس های مربوطه برای تست جمع کننده به تعداد 1000 تا به وجود بیاید.

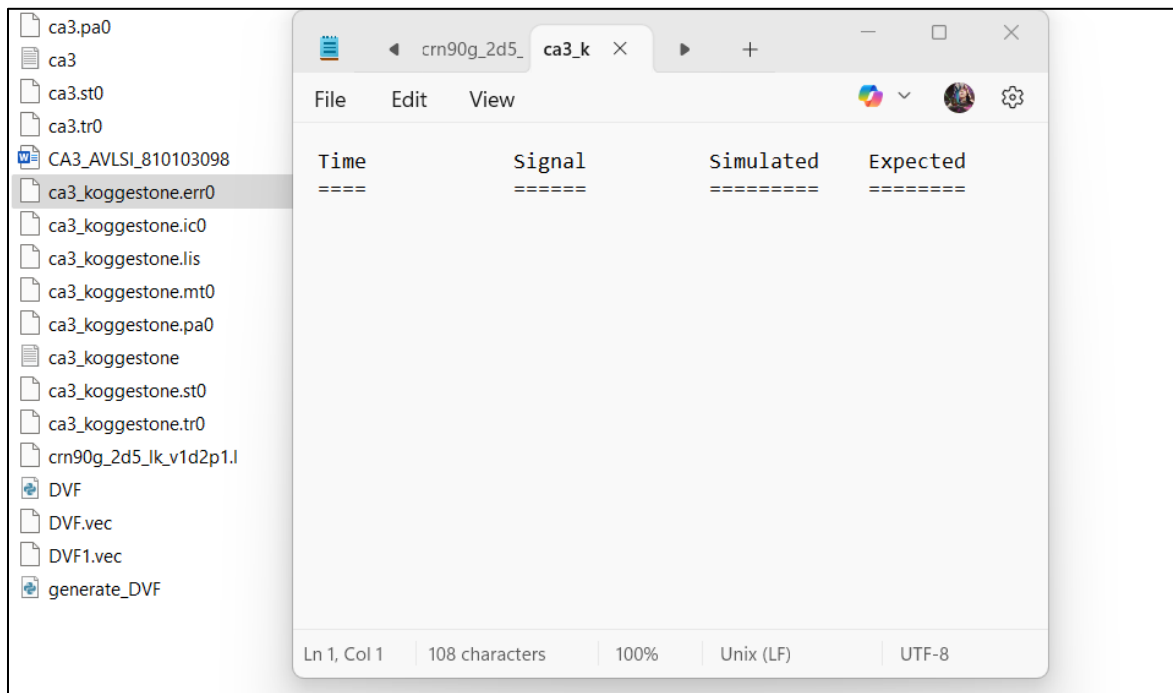
در اینجا با دستور 'DVF.vec' VEC. کاری کردیم تا جمع کننده مون از وکتور های وکتور فایل حین شبیه سازی بخواند.

در زیر تصویری از تنظیمات و رمان شبیه سازی نیز قرار داده شده است که با دستور tran. یک شبیه سازی از نوع زمانی برای جمع کننده KOGGE STONE انجام میدهم:



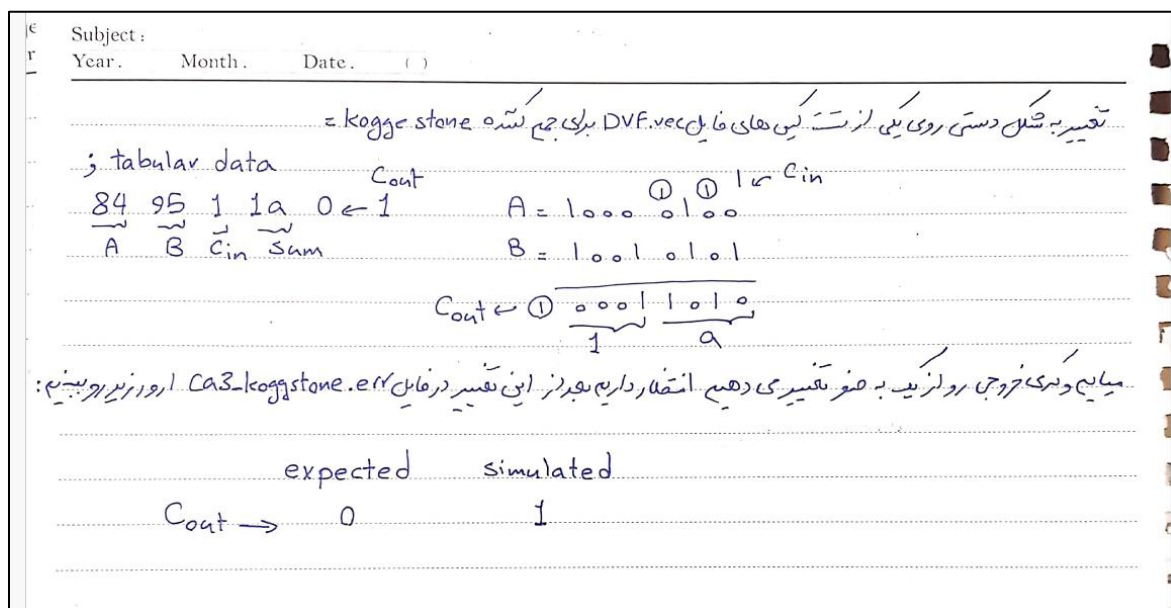
شکل 53- دستور tran برای انجام شبیه سازی زمانی در فایل ca3_koggestone.sp

حال شبیه سازی را انجام میدهم و خروجی فایل ca3_koggestone.err را نگاه میکنیم و انتظار داریم این فایل خالی باشد.



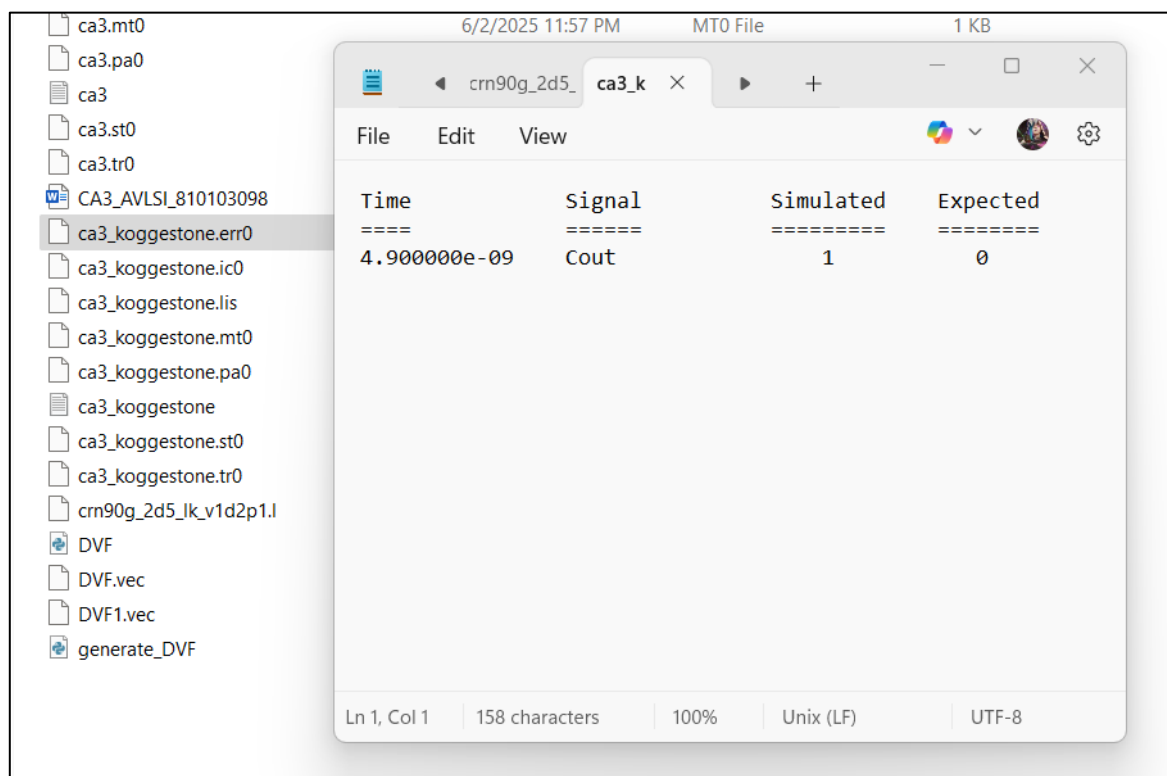
شکل 54- خروجی فایل ca3_koggestone پس از انجام شبیه سازی روی وکتورها

حال مثل جمع کننده قبلی اینجا هم میایم و اولین تست کیس رندومی که در فایل هست رو مقدار Cout اش رو دستی عوض میکنیم:



شکل 55- تغییر مقدار Cout برای اولین تست وکتور در فایل DVF.vec برای جمع کننده KOGGE STONE

حال میام و دوباره شبیه سازی انجام میدهم و خروجی فایل ارور رو چک میکنیم:



شکل 56- خروجی فایل ارور پس از تغییر تست وکتور به شکل دستی

خب همان طور که در تصویر صفحه قبل هم دیده میشود ما دقیقاً ارور روی بیت کری خروجی دریافت کردیم که انتظار این مورد رو هم داشتیم چون به شکل دستی تغییرش داده بودیم.

تا اینجا هر دو جمع کننده به شکل کامل چه از نظر ساختار و چه صحت عملکرد مورد بررسی قرار گرفتند در بخش بعدی به اندازه گیری تأخیر بیشینه و توان برای هر دو جمع کننده میپردازیم.

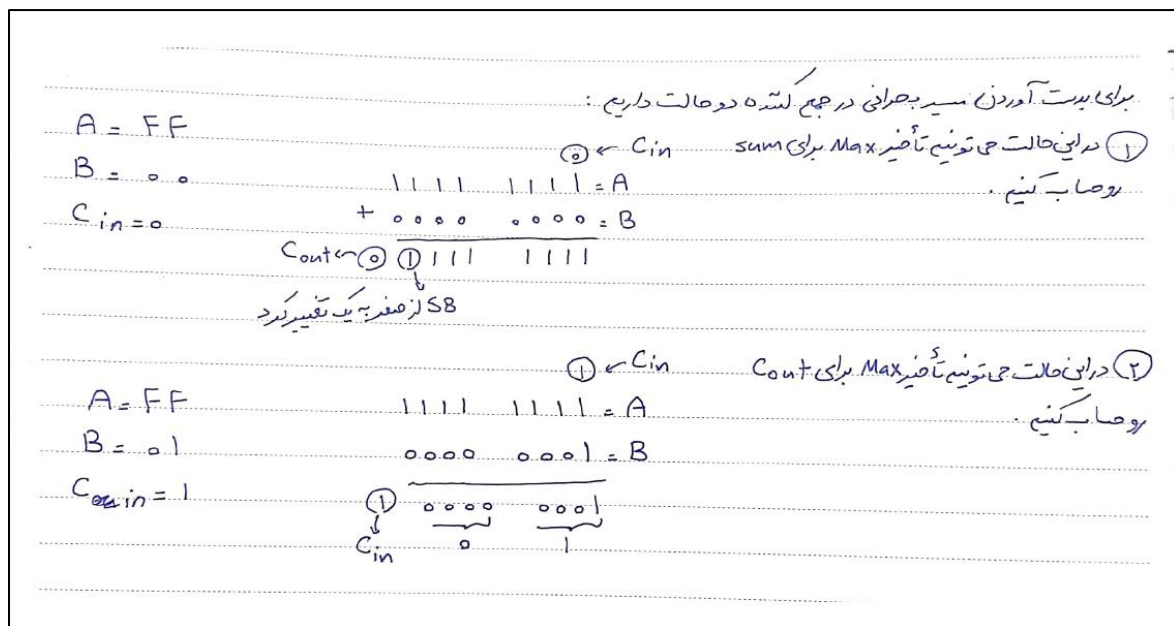
(د) محاسبه وکتور برای تحریک مسیر بحرانی و اندازی گیری تأخیر بیشینه جمع کننده ها

(د) (۱۰٪) حال که از صحت عملکرد جمع کننده ها مطمئن شدیم، می خواهیم بیشترین تأخیر هر یک را به دست آوریم. برای این منظور، در یک DVF دو ورودی تماماً صفر را به عنوان اولین ترکیب قرار دهید. طبیعتاً انتظار می رود خروجی نیز صفر باشد. سپس ترکیب ورودی دیگری نیز اضافه کنید که این ورودی مسیر بحرانی مربوط به CPA یا Kogge Stone را تحریک کند. با توجه به اینکه زمانبندی اعمال ورودی ها در فایل مشخص است، به کمک دستور measure تأخیر بیشینه ی هر دو جمع کننده را به دست آورید. حال نتایج به دست آمده از دو مدار را با یکدیگر مقایسه کرده و نتیجه ی مقایسه را توجیه کنید.

برای محاسبه این مورد کافیهست که وکتورهای لازم برای مسیر بحرانی رو حساب کنیم. به شکل کلی در جمع کننده RIPPLE که تفاوتی نداره و در هر حالت به یک اندازه زمان برای محاسبه خروجی ها زمان میبره چون وابسته به خروجی مرحله قبل هستیم تا خروجی مرحله بعد رو حساب کنیم. ولی در KOGGE STONE ADDER در صورتی که اولین بیت دو ورودی یک باشن یعنی generate کنن و بقیه ورودی ها یکیشون فقط یک باشه در هر جایگاه یعنی propagate کنن بدترین حالت یا همان مسیر بحرانی را خواهیم داشت. یک حالت دیگر هم هست که کری ورودی یک باشه و همه ورودی ها propagate کنن ، این سناریو هم مسیر بحرانی به حساب می آید و بدترین تأخیر رو از بین ورودی های مختلف دارد.

برای این فاز اولاً یک فایل وکتور جدا با اسم DVF_delay.vec برای kogge stone adder و یک فایل وکتور با نام DVF1_delay.vec هم برای RCA داریم که وکتورهای مسیر به سناریو مسیر بحرانی در این فایل ها قرار دارند.

از طریق دستور VEC DVF1_delay.vec بردارهای مسیر بحرانی رو به جمع کننده وارد میکنیم. در این بخش انتظار داریم تأخیر kogge stone با توجه به ساختاری که دارد از CRA بهتر باشد. در صفحه بعد نحوه محاسبه و توضیحات وکتورهای مربوطه گفته شده است.



شکل 58- نحوه محاسبه وکتورها برای اندازه گیری تأخیر بیشینه و مسیر بحرانی

باید هر دو این محاسبات رو انجام داده و سپس از بین آنها ماکسیمم بگیریم.

```

ca3.sp      ca3_koggestone.sp      DVF1.vec      DVF.vec      DVF_delay.vec
1  ;8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout
2  radix 44 44 1 44 1
3  ;define name for each vector
4  vname A[8:1] B[8:1] Cin S[8:1] Cout
5  ;define IO
6  IO II II I OO O
7
8  ;define waveform characteristics
9  tunit ps
10 Period 1100
11 tdelay 1000.0 00 00 0 FF 1
12 Slope 50
13 VOH '0.8*Vdd'
14 VOL '0.2*Vdd'
15
16 ; tabular data
17 00 00 0 00 0
18 FF 00 0 FF 0
19 ;00 00 0 00 0
20 ;FF 01 1 01 1
21

```

شکل 57- فایل وکتور جدید برای مسیر بحرانی

اولین خروجی برای وکتور سناریو 1 جمع کننده CRA:(اندازه گیری ماکسیمم تاخیر SUM)

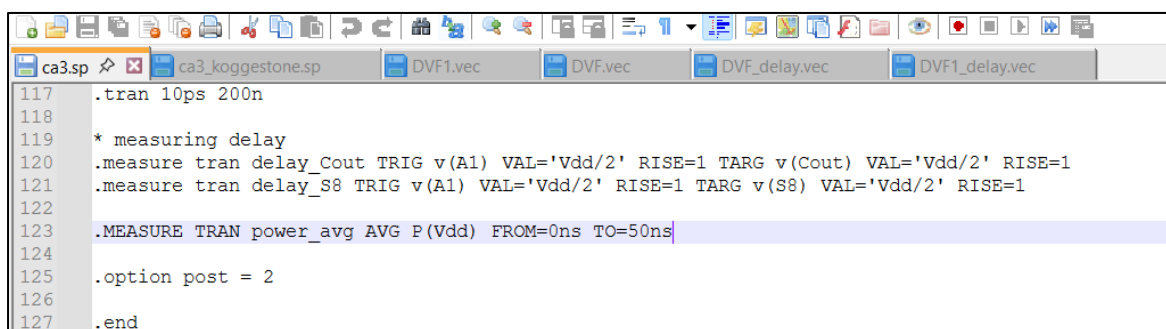
```
*****
*****carry ripple adder*****

***** transient analysis tnom= 25.000 temp= 25.000 *****
**warning** (ca3.sp:120) .MEASURE delay_cout never reached the target value, Measurement failed!
delay_cout= failed      targ= not found      trig= 1.1076n
delay_s8= 180.1255p      targ= 1.2877n      trig= 1.1076n
power_avg= 2.1751u      from= 0.          to= 50.0000n

***** job concluded
*****
*****carry ripple adder*****
```

شکل 59- تاخیر ماکسیمم برای SUM جمع کننده CRA

همان طور که مشاهده میکنید تاخیر بیشینه SUM برابر با 180ps برای جمع کننده CRA برای وکتور اول شد. دقت کنید چون کری خروجی مقدار صفر باقی ماند پس fail شد.



```
ca3.sp  ca3_koggestone.sp  DVF1.vec  DVF.vec  DVF_delay.vec  DVF1_delay.vec
117 .tran 10ps 200n
118
119 * measuring delay
120 .measure tran delay_cout TRIG v(A1) VAL='Vdd/2' RISE=1 TARG v(Cout) VAL='Vdd/2' RISE=1
121 .measure tran delay_s8 TRIG v(A1) VAL='Vdd/2' RISE=1 TARG v(S8) VAL='Vdd/2' RISE=1
122
123 .MEASURE TRAN power_avg AVG P(Vdd) FROM=0ns TO=50ns
124
125 .option post = 2
126
127 .end
```

شکل 60- نحوه اندازه گیری تاخیر ها با دستور MEASURE.

حال به بررسی دومین حالت میپردازیم که برای محاسبه تاخیر بیشینه کری میباشد.

```
*****
*****carry ripple adder*****

***** transient analysis tnom= 25.000 temp= 25.000 *****
delay_cout= 1.3250n      targ= 2.4326n      trig= 1.1076n
delay_s8= 179.0273p      targ= 1.2866n      trig= 1.1076n
power_avg= 1.2023u      from= 0.          to= 50.0000n

***** job concluded
*****
*****carry ripple adder*****
```

شکل 61- خروجی تاخیر برای وکتور دوم مربوط به کری جمع کننده CRA

خب این مقدار برابر با 1.3250ns شده است که بیشتر از مقدار قبلی است پس همین رو به عنوان تاخیر بیشینه در نظر میگیریم.

حال باید همین مرحل رو برای جمع کننده KOGGE STONE انجام دهیم.

خروجی وکتور اول برای این جمع کننده KOGGE STONE:

```
*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
**warning** (ca3_koggestone.sp:191) .MEASURE tpd_max_cout never reached the target value, Measurement failed!
tpd_max_cout= failed targ= not found trig= 1.1076n
tpd_max_s8= 201.4580p targ= 1.3090n trig= 1.1076n
power_avg= 1.2101u from= 0. to= 50.0000n

***** job concluded
*****
* 8-bit kogge-stone adder
```

شکل 62- تاخیر بیشینه برای KOGGE STONE

خب از جایی که تاخیر SUM عمدتا بیشتر از تاخیر CARRY است ما همین تاخیر رو به عنوان تاخیر بیشینه در نظر میگیریم.

نکته در مورد تاخیر جمع کننده ها: عمدتا ما انتظار داریم که تاخیر جمع کننده KOGGE STONE بهتر از جمع کننده CRA باشد ولی در عمل در تعداد بیت های کم مثل 8 بیت این تفاوت واقعا آشکار نیست و در تعداد بیت های بیشتر مثل 16 یا 32 بیت این برتری بیشتر به چشم خواهد آمد. در ضمن به خاطر FAN OUT واحد های محاسباتی در جمع کننده KOGGE STONE ممکن است این اورهد باعث افزایش زمان بیشینه شود. مورد تاثیر گذار دیگر مربوط به تکنولوژی می باشد در تکنولوژی های زیر 100 نانومتر بدلیل کوکتر شدن سائز ترانزیستورها ممکن است روی محاسبه ما برای تاخیر جمع کننده ها تاثیر گذار باشد.

همچنین عمدتا ساختار KOGGE STONE با وجور اینکه سریع تر از CRA میباشد ولی توان مصرفی بیشتری از آن دارد در این شبیه سازی که ما انجام دادیم البته تفاوت زیادی از این حیث دو جمع کننده نداشتن و توان مصرفی حدود 1.2uW میکرووات بدست آمد. این نکته شایان ذکر است که ما این مقایسه را در بدترین حالت برای جمع کننده KOGGE STONE انجام دادیم و در بهترین حالت میتواند این تفاوت بسیار بیشتر باشد.

جدول مقایسه تاخیر دو جمع کننده در بدترین حالت:

جدول 1- مقایسه تاخیر بیشینه جمع کننده ها

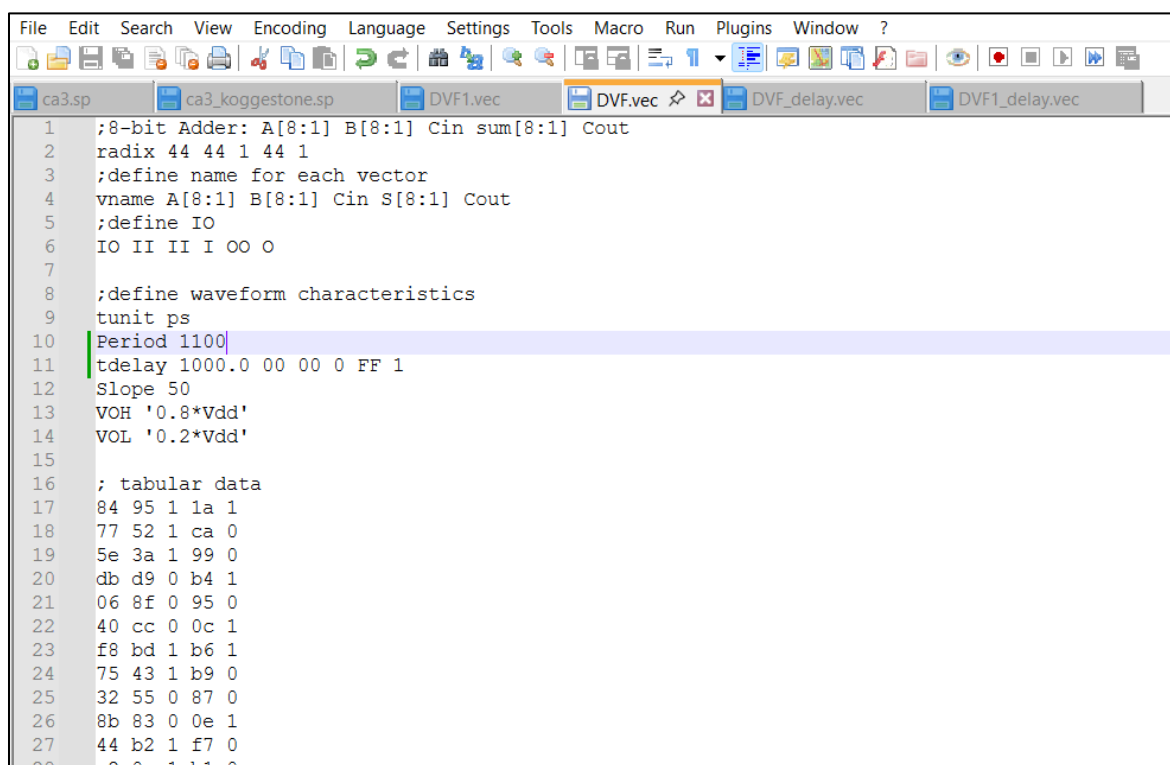
تاخیر بیشینه محاسبه شده با وکتورها	نام جمع کننده
1.3250ns	CARRY RIPPLE ADDER
201ps	KOGGE STONE ADDER

نکته خیلی مهم) انتظار داریم طبق ساختار جمع کننده و در بدترین حالت جمع کننده KOGGE STONE تاخیری معادل یک و خورده ای نانو ثانیه داشته باشد ولی با تنظیمات شبیه سازی ما و ورودی هایی که دادیم به اعداد بالا رسیدیم.

بخش ه) محاسبه توان دینامیک برای دو جمع کننده:

ه) (۵٪) در این بخش قصد داریم توان دینامیک را برای هردو جمع کننده به دست آوریم. برای این منظور، از DVF تولید شده که شامل تعداد زیادی ورودی تصادفی است، استفاده کنید؛ به این صورت که دوره ی تغییر ورودی ها (period) را در DVF به گونه ای تعیین کنید که تنها کمی بیش از تأخیر بیشینه ی جمع کننده ها باشد (حدود 100 ps) و سپس توان میانگین را در کل بازه ی شبیه سازی برای هردو جمع کننده به دست آورید. تأخیر این دو مدار را با یکدیگر مقایسه کرده و اختلاف موجود را توجیه کنید.

در این بخش همان طور که در صورت سوال ذکر شده قصد داریم تا توان دینامیک را برای هر دو جمع کننده در نظر بگیریم لذا پریود اعمال ورودی های رندوم به مدار رو کمی بیشتر از تاخیر بیشینه یعنی یک نانو ثانیه قرار میدهیم، که به عددی مثل 1.1 نانو ثانیه میرسیم. در ضمن مدت زمان شبیه سازی 1100 نانو ثانیه و توان هم در همین مدت زمان محاسبه خواهد شد.



```

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec DVF_delay.vec DVF1_delay.vec
1 ;8-bit Adder: A[8:1] B[8:1] Cin sum[8:1] Cout
2 radix 44 44 1 44 1
3 ;define name for each vector
4 vname A[8:1] B[8:1] Cin S[8:1] Cout
5 ;define IO
6 IO II II I OO O
7
8 ;define waveform characteristics
9 tunit ps
10 Period 1100
11 tdelay 1000.0 00 00 0 FF 1
12 Slope 50
13 VOH '0.8*Vdd'
14 VOL '0.2*Vdd'
15
16 ; tabular data
17 84 95 1 1a 1
18 77 52 1 ca 0
19 5e 3a 1 99 0
20 db d9 0 b4 1
21 06 8f 0 95 0
22 40 cc 0 0c 1
23 f8 bd 1 b6 1
24 75 43 1 b9 0
25 32 55 0 87 0
26 8b 83 0 0e 1
27 44 b2 1 f7 0
28 32 0e 1 b1 0
  
```

شکل 63- تنظیمات در فایل وکتور رندوم برای محاسبه توان دینامیک برای 1000 تست ورودی

در این حالت عملاً به خاطر اینکه تا ورودی‌ها می‌خواهند ثابت شوند می‌ایم و ورودی‌های جدید به مدار وارد می‌کنیم پس بیشترین حالت SWITCHING ACTIVITY رو در مدار خواهیم داشت و توان دینامیک رو پس از شبیه‌سازی خواهیم داشت.

```

15 *VCin Cin 0 DC 0
16
17 |.tran 10ps 1000h
18
19 * measuring delay
20 .measure tran delay_Cout TRIG v(A1) VAL='Vdd/2' RISE=1 TARG v(Cout) VAL='Vdd/2' RISE=1
21 .measure tran delay_S8 TRIG v(A1) VAL='Vdd/2' RISE=1 TARG v(S8) VAL='Vdd/2' RISE=1
22
23 |.MEASURE TRAN power_avg AVG P(Vdd) FROM=0ns TO=1000ns
24
25 .option post = 2
26
27 .end

```

شکل 64- تنظیمات برای انجام شبیه‌سازی برای توان دینامیک

خروجی توان دینامیک برای جمع‌کننده CRA:

```

*****
*****carry ripple adder*****

***** transient analysis tnom= 25.000 temp= 25.000 *****
delay_cout= 236.6769p targ= 1.3443n trig= 1.1076n
delay_s8= 429.1230p targ= 1.5367n trig= 1.1076n
power_avg= -72.9145u from= 0. to= 1.0000u

***** job concluded
*****
*****carry ripple adder*****

```

شکل 65- توان دینامیک برای جمع‌کننده CRA

خب همان‌طور که می‌بینید توان دینامیک برای 1000 تست وکتور ورودی رندوم با پریود 1100ps عددی معادل 72uW شده است.

توان دینامیک برای جمع‌کننده KOGGE STONE:

```

*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
tpd_max_cout= 2.4360n targ= 3.5436n trig= 1.1076n
tpd_max_s8= 326.0203p targ= 1.4336n trig= 1.1076n
power_avg= -96.5033u from= 0. to= 1.1000u

***** job concluded
*****
* 8-bit kogge-stone adder

```

شکل 66- محاسبه توان دینامیک برای KOGGE STONE به ازای 1000 تست وکتور رندوم

نکته مهم) در اینجا برای ورودی از فایل DVF1.vec و DVF2.vec استفاده کردیم که تست وکتورهای رندوم ما در اونها قرار داشتن.

نکته مهم) زمان کل شبیه سازی و محاسبه توان بین 0 تا 1100ns میباشد.

خب همان طور که در صفحه قبل هم مشاهده کردید توان دینامیک برای جمع کننده KOGGE STONE معادل 96.50uW شده است ولی تاخیر کمتری نسبت به جمع کننده CRA دارد که دقیقا مورد انتظار ما هم بود.

جدول 2- مقایسه توان دینامیک مصرفی برای دو جمع کننده به ازای 1000 وکتور ورودی

نام جمع کننده	توان دینامیک برای 1000 وکتور رندوم ورودی
CARRY RIPPLE ADDER	72.91uW
KOGGE STONE ADDER	96.50uW

نتیجه گیری و توجیه نتایج: از جایی که جمع کننده KOGGE STONE ساختار سریع تری نسبت به CRA دارد تاخیر تولید خروجی آن کمتر است ولی توان مصرفی بیشتری دارد که با نگاه کردن به تاخیر SUM در نتایج و توان مصرفی میتوان صحت این موضوع را مشاهده کرد.

در این بخش بدلیل اینکه ورودی ها دائما در حال تغییر هستند لذا جزو توان دینامیک حساب میشوند.

بخش و) محاسبه توان استاتیک برای دو جمع کننده:

در این قسمت ابتدا میایم و پریود رو به عدد بزرگ مثل 2 یا 5 نانو ثانیه قرار میدهم تا در واقع هر 5 نانوثانیه ورودی های جدید به جمع کننده وارد شوند. در این حالت در یک بازه ای از زمان اتمام کار ورودی های قبلی تا اعمال ورودی های جدید مدار در حالت استاتیک و ثابت قرار دارد و در این بازه SWITCHING ACTIVITY نداریم. با این حال عددی که بعد از شبیه سازی بدست می آوریم شامل توان دینامیک هم میشوند که باید از طریق فرمولها این مقدار رو از توان کلی کم کنیم تا توان خالص استاتیک بدست بیاید.

توان استاتیک = توان میانگین کل - (توان دینامیک × نسبت زمان فعالیت دینامیک به کل زمان)

روال کار: در این بخش پس از شبیه سازی باید ابتدا توان میانگین بدست آمده رو در زمان ضرب کنیم تا انرژی بدست بیاد و سپس اون بخش از زمان شبیه سازی که مرتبط به توان دینامیک هست رو باید بدست بیاریم و در زمانش ضرب کنیم حالا با کم کردن این دو مقدار انرژی توان استاتیک بدست میاد و بعد با تقسیم بر زمانش به توان استاتیک خالص میرسیم.

توان دینامیک = $\alpha C V_{DD}^2 F$ Switching Activity
 مدبر در اندر تغییر ورودی ها

توان دینامیک = توان استاتیک + توان دینامیک
 توان استاتیک

short circuit
 (هر دو ترانزیستور روشن باشند)
 pmos, nmos
 روشن باشند

انرژی = $\frac{\text{توان}}{\text{زمان}}$
 توان = انرژی × زمان

انرژی = $\frac{\text{توان استاتیک}}{\text{زمان}}$
 زمان ← مربوط به استاتیک

انرژی توان استاتیک → - } انرژی توان = توان Avg × زمان
 انرژی توان = توان دینامیک × زمان

دینامیک P4PCO

شکل 67- روال محاسبه توان استاتیک از روی توان میانگین

خب حال به شبیه سازی بر اساس تنظیمات زیر برای CRA میپردازیم:

```
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec DVF_delay.vec DVF1_delay.vec
1 ;8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout
2 radix 44 44 1 44 1
3 ;define name for each vector
4 vname A[8:1] B[8:1] Cin S[8:1] Cout
5 ;define IO
6 IO II II I OO O
7
8 ;define waveform characteristics
9 tunit ps
10 Period 5000
11 tdelay 1000.0 00 00 0 FF 1
12 Slope 50
13 VOH '0.8*Vdd'
14 VOL '0.2*Vdd'
15
16 ; tabular data
17 24 69 1 8e 0
18 85 68 1 ee 0
19 ec 65 0 51 1
```

```
114 *VCin Cin 0 DC 0
115
116
117 .tran 10ps 5000n
118
119 * measuring delay
120 .measure tran delay_Cout TRIG v(A1) VAL='Vdd/2' RISE=1 TARG v(Cout) VAL='Vdd/2' RISE=1
121 .measure tran delay_S8 TRIG v(A1) VAL='Vdd/2' RISE=1 TARG v(S8) VAL='Vdd/2' RISE=1
122
123 .MEASURE TRAN power_avg AVG P(Vdd) FROM=0ns TO=5000ns
124
125 .option post = 2
126
```


نتیجه شبیه سازی برای توان استاتیک جمع کننده CRA:

```
*****
*****carry ripple adder*****

***** transient analysis tnom= 25.000 temp= 25.000 *****
delay_cout= 236.6769p targ= 5.2443n trig= 5.0076n
delay_s8= 429.1230p targ= 5.4367n trig= 5.0076n
power_avg= -15.4769u from= 0. to= 5.0000u

***** job concluded
*****
*****carry ripple adder*****
```

شکل 68- مقدار توان میانگین اندازه گیری شده برای CRA پس از شبیه سازی برای 1000 وکتور رندوم ورودی

این شبیه سازی به مدت 5000 نانوثانیه انجام شد چون پریود اعمال ورودی ها 5 نانوثانیه بود و 1000 وکتور رندوم ورودی هم داشتیم.

Subject :
Year. Month. Date. ()

توان استاتیک برای جمع کننده Carry Ripple adder :

بر حسب میکرو ژول

لا بار $15.4769 \mu W$ = توان میانگین در مدت 5000 نانوثانیه

انرژی کل = $15.4769 \times 5 = 77.3845$

انرژی دینامیک = $72.9 \times 1 = 72.91$

$77.3845 - 72.91 = 4.4745$ = انرژی استاتیک

4 میکرو = $5 - 1$ = زمان استاتیک

توان استاتیک = $\frac{4.4745}{4}$ = 1.118125

لا بار $1.118125 \mu W$

شکل 69- انجام محاسبات برای بدست آوردن توان استاتیک جمع کننده CRA

تحلیل و بررسی نتیجه: خب همان طور که در بالا میبینید ما پس از شبیه سازی به توان متوسط $15.4uW$ رسیدیم در مدت زمان 5000 نانو ثانیه. حال با ضرب این مقدار در زمان شبیه سازی به مقدار حدودی 77.35 میکرو ژول میرسیم که انرژی کل میباشد. حال ما در مرحله و بخش قبل توان دینامیک رو حساب کرده بودیم و مقدار 72.9 میکرو وات بدست آمده بود و میدانیم حدود $1/5$ زمان شبیه سازی برای توان دینامیک بوده است با ضرب این توان در زمانش انرژی دینامیک بدست

میاد که از انرژی کل کم میکنیم و به انرژی استاتیک میرسیم، حال با تقسیم این مقدار در زمان باقی مانده شبیه سازی که توان استاتیک داشتیم به مقدار خالص برای توان استاتیک این جمع کننده میرسیم.

نتیجه شبیه سازی برای توان استاتیک جمع کننده KOGGE STONE:

```
*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
tpd_max_cout= 10.2356n targ= 15.2432n trig= 5.0076n
tpd_max_s8= 326.0203p targ= 5.3336n trig= 5.0076n
power_avg= -20.7661u from= 0. to= 5.0000u

***** job concluded
*****
* 8-bit kogge-stone adder

***** job statistics summary tnom= 25.000 temp= 25.000 *****
```

شکل 70- مقدار توان متوسط اندازه گیری شده برای جمع کننده KOGGE STONE روی 1000 وکتور رندوم ورودی

حال پس از اندازه گیری توان متوسط به مانند جمع کننده قبل مقدار توان استاتیک خالص رو برای این جمع کننده به دست می آوریم.

توان استاتیک برای جمع کننده Kogge Stone adder:

$$\begin{aligned} \text{بر حسب میکرووات} &= 20.766 \times 5 = 103.8 \\ \text{انرژی دینامیک} &= 92.5 \times 1 = 92.5 \\ \text{انرژی استاتیک} &= 103.8 - 92.5 = 11.3 \\ \text{۵- زمان استاتیک} &= 5 - 1 = 4 \text{ نانو} \\ \text{توان استاتیک} &= \frac{11.3}{4} = 2.825 \text{ میکرووات} \end{aligned}$$

P4PCO

شکل 71- انجام محاسبات برای بدس آوردن توان استاتیک خالص برای جمع کننده KOGGE STONE

تحلیل نتایج: همان طور که در نتایج مشاهده میکنید و انتظار هم داشتیم مقدار توان استاتیک بسیار کمتر از توان دینامیک اندازه گرفته شده در مرحله قبل برای دو جمع کننده شد، همچنین در اینجا هم توان مصرفی KOGGE STONE از CRA بیشتر شد که این هم قابل انتظار بود.

دلیل اینکه توان استاتیک مقدار خیلی کمتری از توان دینامیک دارد این است که این توان عمدتاً مربوط به جریان نشتی ترانزیستورها، GATE leakage و JUNCTION leakage میباشد که عمدتاً مقادیر بسیار کمی دارند و از این موارد گفته شده فقط جریان نشتی ترانزیستورهاست که عمده این توان استاتیک رو باعث میشود.

جدول 3- مقایسه توان استاتیک برای دو جمع کننده

توان استاتیک	توان میانگین	جمع کننده
1.1uW	15.47uW	CARRY RIPPLE ADDER
1.825uW	20.76uW	KOGGE STONE ADDER

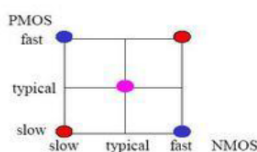
صورت سوال این بخش که انجام دادیم هم به شرح زیر بود که فراموش کردم در ابتدا قرار بدهم:

(۱۰٪) توان استاتیک نیز معیار مهم دیگری است که قصد محاسبه‌ی آن را برای هر دو جمع کننده داریم. برای این کار دقیقاً مانند بخش قبل عمل کنید؛ با این تفاوت که دوره‌ی تغییر سیگنال‌های ورودی را به مقدار قابل توجهی افزایش دهید (حدود 1 ns). سپس توان میانگین را در کل زمان شبیه‌سازی به دست آورید. حال با دانستن توان دینامیک از بخش قبل، مجموع زمان‌هایی که توان دینامیک در مدار مصرف شده است و توان میانگین به دست آمده از این بخش، می‌توان توان استاتیک را محاسبه کرد. روند محاسبه‌ی خود را در گزارش به طور کامل شرح دهید. همچنین توان استاتیک دو جمع کننده را با یکدیگر مقایسه و توجیه کنید.

بخش دوم تمرین

بخش دوم: (۲۵٪)

در این بخش به بررسی گوشه‌های طراحی می‌پردازیم. به طور کلی طراحان اثرات تغییرات محیطی و ساخت بر روی ترانزیستورها را با اصطلاحاتی چون Typical, Fast و Slow بیان می‌کنند. حال با توجه به آنکه ترانزیستورهای PMOS و NMOS موجود در مدار از کدام یک از انواع بالا هستند، می‌توان در مربع فضای طراحی جابه‌جا شد. اصطلاح گوشه، به گوشه‌های این مربع فضای طراحی اطلاق می‌شود. شکل ۲ این مربع را به تصویر کشیده است.



شکل ۲: گوشه‌های طراحی

برای آنکه بتوانید عملکرد مدار را در هر یک از گوشه‌های طراحی بررسی کنید، کافی است زمانی که می‌خواهید به کمک دستور .lib کتابخانه را به نتلیست HSPICE اضافه کنید، در کنار اسم کتابخانه به جای TT، از SS(slow, slow)، FF(fast, fast)، FS(fast, slow) و SF(slow, fast) استفاده کنید. حال با دانستن این نکته، تأخیر بیشینه، توان دینامیک و توان استاتیک مربوط به جمع کننده‌ی Kogge Stone را در گوشه‌های طراحی به دست آورده و مقادیر را با یکدیگر مقایسه و تحلیل کنید.

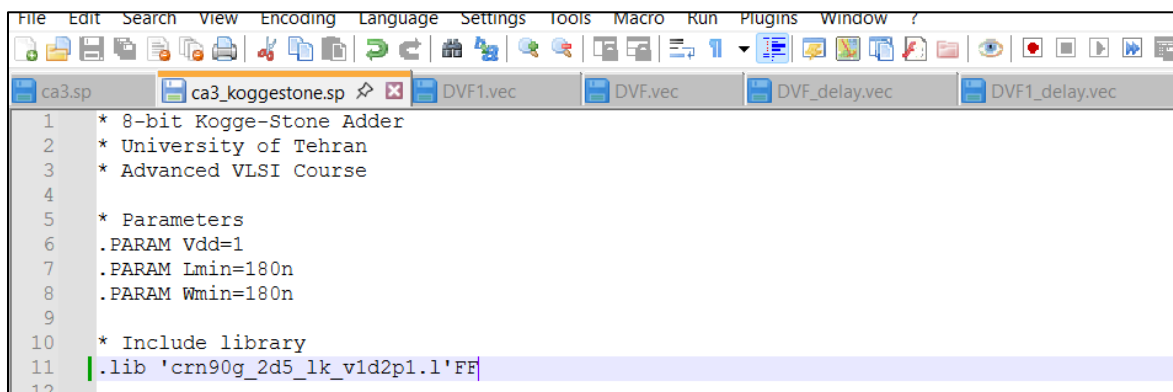
در این بخش همان طور که در صورت سوال هم گفته شده است قرار است تاخیر بیشینه و توان دینامیک و استاتیک جمع کننده KOGGE STONE رو در کرنر های مختلف بدست بیاریم. حال این کرنر ها یا گوشه ها چی هستن؟ وقتی چیپ رو برای فبریکیشن میفرستیم در اثر فرآیند ساخت و Non Ideality هایی که در این فرآیند وجود دارد ممکن است سرعت ترانزیستورهای Nmos و Pmos دقیقاً مشابه چیزی که از قبل در نظر داشتیم نباشد و سرعت آنها کمتر یا بیشتر از حالت عادی یا Typical باشد. حال در این بخش به بررسی تاثیر این گوشه ها در عملکرد مدار میپردازیم.

بررسی پارامترهای تاخیر و توان در گوشه طراحی FF:

اولین حرف این گوشه ها به سرعت ترانزیستور Nmos و حرف بعدی به سرعت ترانزیستور Pmos مربوط می‌شود. در اینجا هر دو نوع ترانزیستور سریع تر از حالت عادی هستن. که انتظار داریم این افزایش سرعت باعث کاهش تاخیر و افزایش توان مصرفی شود. به طور کلی ما دو نوع طراحی داریم همان طور که در درس هم دکترو حدت به آن اشاره کردند، طراحی LOW POWER در این طراحی سرعت مدار و ترانزیستورها کمتر است ولی جریان نشتی مدار کمتر است در واقع توان مصرفی کمتری دارد (به خاطر V_{th} بالای ترانزیستورها) و طراحی PERFORMANCE که سرعت مدار و جریانه‌ی ترانزیستورها بیشتر است ولی توان مصرفی افزایش می‌یابد.

بررسی تاخیر بیشینه در کرنر FF برای جمع کننده KOGGE STONE:

ابتدا باید طبق گفته صورت سوال لایبری و تکنولوژی 90 نانومتر رو با پسوند FF ایمپورت کنیم.



شکل 72- اضافه کردن کتابخانه 90 نانومتر با پسوند FF

در این بخش از وکتورفایل DVF_delay.vec برای اعمال وکتورهای مسیر بحرانی به مدار استفاده کردیم. شبیه سازی هم به مدت 200 نانوثانیه و با تنظیمات عکس صفحه بعد و برای تاخیر ماکسیمم SUM انجام شده است. (پیش فرض)

انتظار داریم در اینجا تاخیر از حالت عادی جمع کننده kogge stone کمتر شده باشد.

تنظیمات شبیه سازی:

```

1 ;8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout
2 radix 44 44 1 44 1
3 ;define name for each vector
4 vname A[8:1] B[8:1] Cin S[8:1] Cout
5 ;define IO
6 IO II II I OO O
7
8 ;define waveform characteristics
9 tunit ps
10 Period 1100
11 tdelay 1000.0 00 00 0 FF 1
12 Slope 50
13 VOH '0.8*Vdd'
14 VOL '0.2*vdd'
15
16 ; tabular data
17 00 00 0 00 0
18 FF 00 0 FF 0
19 ;00 00 0 00 0
20 ;FF 01 1 01 1
21

```

```

186
187 * Analysis
188 .tran 10ps 200ns
189
190 * Measure delay
191 .MEASURE TRAN tpd_max_Cout TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(Cout) VAL='Vdd/2' RISE=1
192 .MEASURE TRAN tpd_max_S8 TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(S8) VAL='Vdd/2' RISE=1
193 .MEASURE TRAN power_avg AVG P(Vdd) FROM=0ns TO=200ns
194
195 * Use DVF for verification
196 .option post=2
197 .END

```

حال در زیر میتونید نتیجه شبیه سازی و تاخیر در کرنر FF رو مشاهده کنید:

```

*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
**warning** (ca3_koggestone.sp:191) .MEASURE tpd_max_cout never reached the target value, Measurement failed!
tpd_max_cout= failed targ= not found trig= 1.1076n
tpd_max_s8= 167.1904p targ= 1.2748n trig= 1.1076n
power_avg= 3.8852u from= 0. to= 200.0000n

***** job concluded
*****
* 8-bit kogge-stone adder

***** job statistics summary tnom= 25.000 temp= 25.000 *****

```

شکل 73- نتیجه شبیه سازی برای تاخیر بیشینه در کرنر FF

همان طور که مشاهده میشود تاخیر جمع کننده KOGGE STONE در این کرنر طراحی نسبت به حالت عادی که 201ps بود کاهش یافته و به عدد 167.19ps رسیده است که قابل انتظار هم بود چون در این گوشه طراحی سرعت تمامی ترانزیستورها از حالت عادی بیشتر است.

در قسمت بعد به بررسی پارامتر توان برای این جمع کننده در کرنر FF میپردازیم.

بررسی توان دینامیک برای جمع کننده KOGGE STONE در گوشه طراحی FF:

برای این بخش باید از فایل DVF.vec که 1000 تا وکتور رندوم در اون قرار دارن استفاده کنیم و مدت زمان شبیه سازی هم 1100 نانو ثانیه میباشد.

دلیل گذاشتن پریود اعمال هر ورودی به اندازه 1100 نانوثانیه در بخش قبلی تمرین و مربوط به محاسبه توان مطرح شد.

```
187 * Analysis
188 |.tran 10ps 1100ns
189
190 * Measure delay
191 .MEASURE TRAN tpd_max_Cout TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(Cout) VAL='Vdd/2' RISE=1
192 .MEASURE TRAN tpd_max_S8 TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(S8) VAL='Vdd/2' RISE=1
193 |.MEASURE TRAN power_avg AVG P(Vdd) FROM=0ns TO=1100ns
194
195 * Use DVF for verification
196 .option post=2
197 .END
```

خب حال در ادامه میتونید نتیجه ی شبیه سازی برای 1000 تست کیس رندوم ورودی و توان دینامیک رو مشاهده کنید.

```
*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
tpd_max_cout= 2.4055n targ= 3.5130n trig= 1.1076n
tpd_max_s8= 282.5677p targ= 1.3901n trig= 1.1076n
power_avg=-106.6271u from= 0. to= 1.1000u

***** job concluded
*****
* 8-bit kogge-stone adder

***** job statistics summary tnom= 25.000 temp= 25.000 *****
```

شکل 74- توان دینامیک جمع کننده KOGGE STONE در کرنر FF و برای 1000 تست وکتور رندوم

خب همان طور که مشاهده میکنید این مقدار نسبت به 96.5uW که توان دینامیک این جمع کننده در حالت عادی بود افزایش محسوسی داشته و در **حدود 10 درصد** که به دلیل **افزایش سرعت ترانزیستورها** در این گوشه طراحی است و تاخیر هم همان طور که در بخش قبل بررسی کردیم کاهش یافته است.

توان دینامیک که پیش از شبیه سازی بدست آوردیم مقدار حدودی 106uW میباشد.

در بخش بعدی به محاسبه توان استاتیک برای این گوشه طراحی میپردازیم.

بررسی توان استاتیک برای جمع کننده KOGGE STONE در گوشه FF:

برای بررسی این مورد باید پریود اعمال ورودی ها رو از 1100ps به 5000ps تغییر دهیم در فایل DVF.vec.

سپس شبیه سازی به اندازه 5000 نانوثانیه انجام دهیم و توان میانگین رو در این بازه حساب کنیم و از روی آن به توان استاتیک خالص برسیم.

```
*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
tpd_max_cout= 10.2054n targ= 15.2130n trig= 5.0076n
tpd_max_s8= 282.5677p targ= 5.2901n trig= 5.0076n
power_avg= -22.4567u from= 0. to= 5.0000u

***** job concluded
*****
* 8-bit kogge-stone adder

***** job statistics summary tnom= 25.000 temp= 25.000 *****
```

شکل 75- توان میانگین اندازه گیری شده در کرنر FF

حال به محاسبه توان استاتیک میپردازیم که در شکل زیر میتویند تصویر این مورد رو مشاهده کنید:

توان استاتیک برای جمع کننده kogge stone در گوشه FF:

$$\begin{aligned} \text{توان میانگین در مدت زمان ۵ نانوثانیه} &= ۲۲,۴۵ \mu\text{W} \\ \text{بر حسب میکرووات} &= ۲۲,۴۵ \times ۵ = ۱۱۲,۲۵ \mu\text{W} \quad \text{انرژی} \\ \text{توان دینامیک در گوشه FF} &= ۱۰۶ \mu\text{W} \quad \text{انرژی دینامیک} \\ &= ۱۰۶ \times ۱ = ۱۰۶ \mu\text{W} \\ \text{۵- زمان استاتیک} &= ۵ - ۱ = ۴ \text{ ns} \\ \text{توان استاتیک} &= \frac{\text{انرژی}}{\text{زمان}} = \frac{۱۱۲,۲۵ - ۱۰۶}{۴} = ۱,۵۶۲ \mu\text{W} \end{aligned}$$

شکل 76- توان استاتیک محاسبه شده در گوشه FF

همان طور که مشاهده میکنید توان میانگین از 20.76uW به 22.45uW افزایش میابد. همان طور که میبینید تغییری چندانی در توان استاتیک حاصل نشده و همچنان مقدار کمی میباشد. عمده توان در این کرنر مربوط به توان دینامیک میباشد.

بررسی پارامترهای تاخیر و توان در گوشه طراحی SS:

در این بخش به بررسی تاخیر و توان در این گوشه طراحی میپردازیم. همان طور که مشاهده میکنید در اینجا سرعت ترانزیستورهای NMOS و PMOS از حالت عادی کمتر است این مورد باعث افزایش تاخیر بیشینه و کاهش توان مصرفی میشود.

```
ca3.sp | ca3_koggestone.sp | DVF1.vec | DVF.vec | DVF_delay.vec | DVF1_delay.vec
1 * 8-bit Kogge-Stone Adder
2 * University of Tehran
3 * Advanced VLSI Course
4
5 * Parameters
6 .PARAM Vdd=1
7 .PARAM Lmin=180n
8 .PARAM Wmin=180n
9
10 * Include library
11 .lib 'crn90g_2d5_lk_v1d2p1.1'ss'
12
13 * Basic Logic Gates as Subcircuits
14
```

شکل 77- ایمپورت لایبرری با پسوند SS

بررسی تاخیر بیشینه در کرنر SS برای جمع کننده KOGGE STONE:

در اینجا تاخیر بیشینه رو برای sum که پیش فرض ما هم هست حساب میکنیم.

نتیجه شبیه سازی برای وکتور مسیر بحرانی مربوط به SUM:

```
ca3.sp | ca3_koggestone.sp | DVF1.vec | DVF.vec | DVF_delay.vec | DVF1_delay.vec
1 ;8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout
2 radix 44 44 1 44 1
3 ;define name for each vector
4 vname A[8:1] B[8:1] Cin S[8:1] Cout
5 ;define IO
6 IO II II I OO O
7
8 ;define waveform characteristics
9 tunit ps
10 Period 1100
11 tdelay 1000.0 00 00 0 FF 1
12 Slope 50
13 VOH '0.8*Vdd'
14 VOL '0.2*Vdd'
15
16 ; tabular data
17 00 00 0 00 0
18 FF 00 0 FF 0
```

```
*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
**warning** (ca3_koggestone.sp:191) .MEASURE tpd_max_cout never reached the target value, Measurement failed!
tpd_max_cout= failed      targ= not found      trig= 1.1076n
tpd_max_s8= 243.2291p      targ= 1.3508n      trig= 1.1076n
power_avg= 635.6489n      from= 0.          to= 200.0000n

***** job concluded
*****
* 8-bit kogge-stone adder
```

شکل 78- تاخیر بیشینه جمع کننده KOGGE STONE در کرنر SS

همان طور که از نتایج مشخص است ما اینجا یک افت سرعت زیادی رو داریم مشاهده میکنیم و تاخیر از 201ps حالت عادی به 243.2ps رسیده که افت حدود 20 درصدی در سرعت میباشد که با توجه به کاهش سرعت تمامی ترانزیستورها در این کرنر این مورد قابل پیش بینی بود.

بررسی توان دینامیک برای جمع کننده KOGGE STONE در گوشه طراحی SS:

خب در این کرنر توقع داریم که توان دینامیک کاهش یابد چون سرعت ترانزیستورها کاهش یافته و طبیعتا ولتاژ آستانه آنها زیاد شده است و میدانیم با افزایش ولتاژ آستانه توان مصرفی ترانزیستورها نیز کاهش میباشد.

برای اندازه گیری این مورد مثل تنظیمات گوشه قبلی عمل میکنیم و 1000 وکتور رندوم رو در مدت 1100 نانوثانیه و با پریود 1100 پیکوثانیه شبیه سازی میکنیم و توان رو در همین بازه اندازه میگیریم.

```
*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
tpd_max_cout= 2.4738n targ= 3.5814n trig= 1.1076n
tpd_max_s8= 379.1415p targ= 1.4867n trig= 1.1076n
power_avg= -88.4486u from= 0. to= 1.1000u

***** job concluded
*****
* 8-bit kogge-stone adder
```

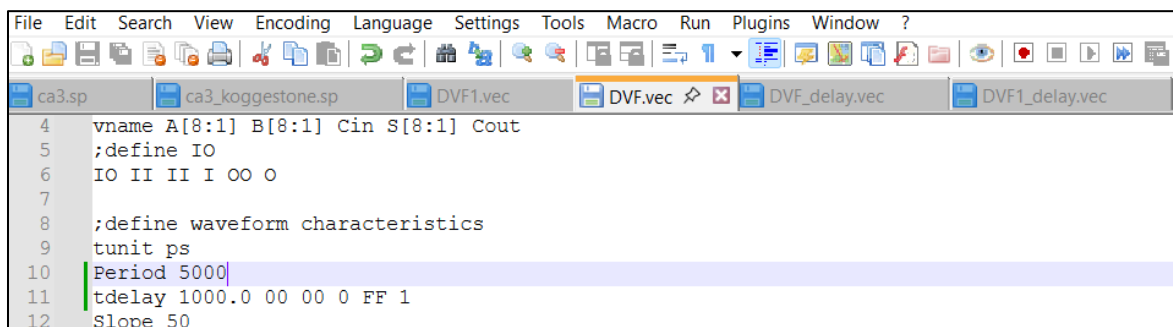
شکل 79- توان دینامیک جمع کننده KOGGE STONE در کرنر SS

توان دینامیک از مقدار 96.5uW به مقدار 88.44uW رسیده است که طبق کاهش سرعت ترانزیستور ها و تاخیر مدار این مورد مطابق انتظار است.

در بخش بعد باید به بررسی توان استاتیک در کرنر SS بپردازیم.

بررسی توان استاتیک برای جمع کننده KOGGE STONE در گوشه SS:

در این بخش تنظیمات مثل توان دینامیک است با این فرق که باید پریود اعمال ورودی ها به مدار رو از 1.1 نانوثانیه به 5 نانوثانیه افزایش دهیم.



```
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec DVF_delay.vec DVF1_delay.vec
4 vname A[8:1] B[8:1] Cin S[8:1] Cout
5 ;define IO
6 IO II II I OO O
7
8 ;define waveform characteristics
9 tunit ps
10 Period 5000
11 tdelay 1000.0 00 00 0 FF 1
12 Slope 50
```

```

*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
tpd_max_cout= 10.2737n targ= 15.2813n trig= 5.0076n
tpd_max_s8= 379.1415p targ= 5.3867n trig= 5.0076n
power_avg= -19.3191u from= 0. to= 5.0000u

***** job concluded
*****
* 8-bit kogge-stone adder

***** job statistics summary tnom= 25.000 temp= 25.000 *****

```

شکل 80- توان میانگین در گوشه SS برای جمع کننده KOGGE STONE

در زیر میتونید محاسبات انجام شده جهت بدست آوردن توان استاتیک خالص از روی توان میانگین برای جمع کننده KOGGE STONE در گوشه SS را مشاهده کنید:

توان استاتیک برای جمع کننده Kogge stone در گوشه SS:

$$\begin{aligned}
& \text{توان میانگین در گوشه SS} = 19.319 \mu\text{W} \\
& \text{بر حسب کنترل} \quad 19.319 \mu\text{W} \times 5 = 96.55 \mu\text{W} \quad \text{انرژی NMOS} \\
& \text{انرژی PMOS} = 88.44 \mu\text{W} \quad \text{توان در گوشه SS} = 88.44 \mu\text{W} \\
& \text{توان استاتیک} = 96.55 \mu\text{W} - 88.44 \mu\text{W} = 8.11 \mu\text{W} \\
& \text{توان استاتیک} = \frac{8.11}{4} = 2.02 \mu\text{W}
\end{aligned}$$

شکل 81- توان استاتیک محاسبه شده برای جمع کننده KOGGE STONE در کرنر SS

خب همان طور که در نتایج هم مشاهده میکنید به طور کلی توان میانگین جمع کننده در این گوشه نسبت به حالت عادی از 20.76 به 19.31uW رسیده که نشان میدهد کاهش توان مصرفی رو داریم. همچنین مقدار توان استاتیک هم نسبت به حالت عادی فرق خاصی نکرده و در حدود 2uW میباشد ولی کمی زیاد شده که نشان میدهد در این گوشه سهم توان استاتیک به دینامیک از کل توان نسبت به حالت عادی افزایش خیلی کمی داشته است.

دلیل کاهش توان میانگین هم به خاطر کاهش سرعت تمامی ترانزیستورها در این گوشه طراحی میباشد.

بررسی پارامترهای تاخیر و توان در گوشه طراحی FS:

در اینجا ترانزیستورهای NMOS سریع تر و ترانزیستورهای PMOS کند تر از حالت عادی یا typical هستند پس انتظار تغییر خیلی زیادی نسبت به حالت عادی نیستیم هر چند باز تفاوت قابل انتظار است ولی نه به اندازه دو گوشه قبلی.

```

ca3.sp  ca3_koggestone.sp  DVF1.vec  DVF.vec  DVF_delay.vec  DVF1_delay.v
1  * 8-bit Kogge-Stone Adder
2  * University of Tehran
3  * Advanced VLSI Course
4
5  * Parameters
6  .PARAM Vdd=1
7  .PARAM Lmin=180n
8  .PARAM Wmin=180n
9
10 * Include library
11 .lib 'crn90g_2d5_1k_v1d2p1.1' FS
12
13 * Basic Logic Gates as Subcircuits
14
15 * Inverter (NOT gate)

```

بررسی تاخیر بیشینه در کرنر FS برای جمع کننده KOGGE STONE:

برای این بخش هم مثل قبل عمل میکنم و دیگه توضیح زیادی لازم نیست جهت خلاصه شدن گزارش، مثل قبا از وکتور فایل DVF_delay.vec استفاده میکنیم با تنظیمات شبیه سازی مثل گوشه ای قبلی.

نتیجه تاخیر مسیر بحرانی SUM در گوشه FS برای جمع کننده KOGGE STONE:

```

*****
* 8-bit kogge-stone adder
***** transient analysis tnom= 25.000 temp= 25.000 *****
**warning** (ca3_koggestone.sp:191) .MEASURE tpd_max_cout never reached the target value, Measurement failed!
tpd_max_cout= failed targ= not found trig= 1.1076n
tpd_max_s8= 202.1159p targ= 1.3097n trig= 1.1076n
power_avg= 1.4573u from= 0. to= 200.0000n
***** job concluded
*****

```

شکل 82- تاخیر بیشینه برای جمع کننده در کرنر FS

دلیل FAIL شدن کری هم این است که در اینجا وکتور ما برای SUM تعبیه شده و اصلا کری خروجی تغییر نمیکند و صفر می ماند.

همان طور که می بینید **تاخیر تاخیر بیشینه نسبت به حالت عادی** خیلی فرقی نکرده و از 201ps به 202.1ps رسیده که نشان میدهد که جمع کننده **فقط به مقدار ناچیزی** نسبت به حالت عادی در این گوشه کند شده است.

بررسی توان دینامیک برای جمع کننده KOGGE STONE در گوشه طراحی FS:

در این بخش از فایل DVF.vec استفاده میکنیم که تست وکتورهای رندوم در اون قرار دارن و سپس شبیه سازی به مدت 1100 نانوثانیه و پریود 1.1 نانوثانیه ای برای هر ورودی رندوم انجام میدهیم. نتایج رو میتونید در صفحه بعد مشاهده کنید.

در ضمن در این گوشه طراحی مثل مورد تاخیر در مورد توان هم انتظار تغییر چشمگیری نسبت به حالت عادی نیستیم.

```

*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
tpd_max_cout= 2.4345n targ= 3.5420n trig= 1.1076n
tpd_max_s8= 327.8271p targ= 1.4354n trig= 1.1076n
power_avg= -97.5445u from= 0. to= 1.1000u

***** job concluded
*****
* 8-bit kogge-stone adder

***** job statistics summary tnom= 25.000 temp= 25.000 *****

```

شکل 83- توان دینامیک برای جمع کننده در گوشه FS

خب در اینجا توان دینامیک از 96.5uW حالت عادی به 97.5uW رسید و نشان می‌دهد توان دینامیک هم کمی افزایش نسبت به حالت عادی داشته است. ولی در این مجموع این تغییرات خیلی ناچیز است.

بررسی توان استاتیک برای جمع کننده KOGGE STONE در گوشه طراحی FS:

برای این مورد هم به مدت 5000 نانوثانیه و با پریود 5 نانوثانیه برای 1000 وکتور رندوم شبیه سازی می‌کنیم و توان رو در این مدت بدست می‌اریم.

نتیجه شبیه سازی:

```

*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
tpd_max_cout= 10.2340n targ= 15.2416n trig= 5.0076n
tpd_max_s8= 327.8271p targ= 5.3354n trig= 5.0076n
power_avg= -21.1570u from= 0. to= 5.0000u

***** job concluded
*****
* 8-bit kogge-stone adder

***** job statistics summary tnom= 25.000 temp= 25.000 *****

```

شکل 84- توان میانگین اندازه گیری شده برای گوشه FS

حال به محاسبه توان استاتیک به مانند گوشه های قبلی می‌پردازیم. در صفحه بعد تصویر مربوط به این محاسبات قرار داده شده است.

Subject :
 Year . Month . Date . ()

توان استاتیک برای جمع کننده Kogge stone در گوشه FS :

بر حسب میکرو
 $21,15 \times 5 = 105,75$ انرژی یل
 $21,15$ = توان میانگین در مدت
 $97,54 \times 1 = 97,54$ انرژی دینامیک
 $105,75 - 97,54 = 8,21$ انرژی استاتیک
 $8,21 = \frac{21,15}{4}$ = توان استاتیک
 زمان
 $5 - 1 = 4$ میکرو = زمان استاتیک

شکل 85- توان استاتیک جمع کننده در گوشه FS

خب همان طور که از نتایج پارامترها در این کرنر میبینید نسبت به حالت عادی خیلی فرقی نداره ولی افزایش ناچیزی در تاخیر و توان رو شاهد هستیم. توان استاتیک هم مشابه حالت عادی است و از عدد $1.825\mu W$ به $2.05\mu W$ رسیده که افزایش خیلی ناچیزی است.

نتیجه گیری: پس در اینجا هم افزایش ناچیز تاخیر و هم توان مصرفی رو شاهد بودیم. همچنین با وجود اینکه در اینجا ترانزیستورهای NMOS سریعتر از حالت عادی هستند ولی چون شبکه بالابر مدارها و گیت ها با ترانزیستورهای PMOS پیاده سازی میشود در نتیجه کاهش سرعت این ترانزیستورها باعث کاهش سرعت ناچیز مدار شده است.

بررسی پارامترهای تاخیر و توان در گوشه طراحی SF:

```
D:\vlsi vahdat\HWs&CAs\CA3_810103098\ca3_koggestone.sp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec DVF_delay.vec DVF1_delay.vec
9
10 * Include library
11 .lib 'crn90g_2d5_1k_vld2p1.1' SF
12
13 * Basic Logic Gates as Subcircuits
14
```

شکل 86- ایمپورت لایبرری با پسوند SF

در این گوشه ترانزیستورهای NOMS کندتر و ترانزیستورهای PMOS سریع تر از حالت عادی هستند.

پس انتظار داریم تاخیر مدار کمی کمتر از حالت عادی باشد چون ترانزیستورهای PMOS که در شبکه بالابر هستند سریع تر شدن هر چند تحلیل این مورد در این گوشه بسیار سخت است.

بررسی تاخیر بیشینه در کرنر SF برای جمع کننده KOGGE STONE:

در صفحه بعد میتونید تاخیر بیشینه جمع کننده در این گوشه رو برای sum و حالت SF مشاهده کنید.

نکته) دقت شود که از وکتور مربوط به تحریک مسیر بحرانی SUM استفاده کردیم و کری خروجی تغییری نمیکند.

```
*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
**warning** (ca3_koggestone.sp:191) .MEASURE tpd_max_cout never reached the target value, Measurement failed!
tpd_max_cout= failed      targ= not found      trig= 1.1076n
tpd_max_s8= 202.2920p      targ= 1.3099n      trig= 1.1076n
power_avg= 1.2997u from= 0.      to= 200.0000n

***** job concluded
*****
```

شکل 87- تاخیر بیشینه در گوشه SF

همان طور که در ابتدا هم گفته شد تحلیل در این دو گوشه قبلی و اینجا بسیار دشوار است و تغییرات بسیار اندک و ناچیز هستند. در اینجا هم تاخیر نسبت به حالت عادی از 201ps به 202.2ps رسیده است.

بررسی توان دینامیک برای جمع کننده KOGGE STONE در گوشه طراحی SF:

در این بخش از فایل DVF.vec برای ورودی های رندوم استفاده میکنیم و پریود 1.1 نانوثانیه برای اعمال هر ورودی در نظر گرفته شده و شبیه سازی به مدت 1100 نانو ثانیه انجام میشود.

```
*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
tpd_max_cout= 2.4421n      targ= 3.5497n      trig= 1.1076n
tpd_max_s8= 329.9633p      targ= 1.4375n      trig= 1.1076n
power_avg= -95.9716u from= 0.      to= 1.1000u

***** job concluded
*****
* 8-bit kogge-stone adder

***** job statistics summary tnom= 25.000 temp= 25.000 *****
```

شکل 88- توان دینامیک اندازه گیری شده در گوشه SF

توان دینامیک از مقدار 96.5uW به مقدار 95.8uW رسیده که در اینجا هم کاهش ناچیز در توان مصرفی رو شاهد هستیم.

بررسی توان استاتیک برای جمع کننده KOGGE STONE در گوشه طراحی SF:

در این بخش پریود اعمال ورودی ها رو 5 نانوثانیه و شبیه سازی به مدت 5000 نانوثانیه انجام میدهم تا توان میانگین رو پس از شبیه سازی اندازه بگیریم و از روی اون و توان دینامیکی که در مرحله قبل بدست آوردیم توان استاتیک محاسبه شود. نتایج و محاسبات رو میتونید در صفحه بعد مشاهده کنید.

```

*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
tpd_max_cout= 10.2416n targ= 15.2492n trig= 5.0076n
tpd_max_s8= 329.9633p targ= 5.3375n trig= 5.0076n
power_avg= -20.8083u from= 0. to= 5.0000u

***** job concluded
*****
* 8-bit kogge-stone adder

***** job statistics summary tnom= 25.000 temp= 25.000 *****

```

شکل 89- توان میانگین محاسبه شده در گوشه SF برای 5000 نانو ثانیه شبیه سازی

همان طور که میبینید توان میانگین اندازه گیری شده نسبت به حالت عادی که 20.76uW بود به 20.80uW رسیده که میتوان گفت یکسان است و دست نخورده.

در ادامه محاسبات مربوط به توان دینامیک هم قرار داده شده است.

توان استاتیک برای جمع کسره koggestone در گوشه SF:

Nmos	Pmos
توان میانگین در حالت عادی = 20.80 uW	انرژی = 20.80 x 5 = 104
نانو ثانیه	انرژی دینامیک = 95.97 x 1 = 95.97
توان استاتیک = 5 - 1 = 4	95.97
	انرژی استاتیک = 104 - 95.97 = 8.03
	توان استاتیک = $\frac{انرژی}{زمان} = \frac{8.03}{4} = 2$

شکل 90- توان استاتیک برای گوشه SF

جمع بندی:

در اینجا تاخیر نسبت به حالت عادی افزایش ناچیزی داشت و توان دینامیک کاهش یافت ولی توان میانگین و استاتیک تغییر چندانی نداشتن و ثابت ماندن.

در ادامه و صفحه بعد جدولی برای مقایسه تمامی گوشه قرار داده شده است که اعداد ارقام مربوط به تاخیرها و توان های دینامیک و استاتیک در آن قرار گرفته اند.

جدول 4- مقایسه تاخیر ، توان دینامیک و استاتیک جمع کننده KOGGE STONE در گوشه های مختلف

توان استاتیک	توان دینامیک	تأخیر بیشینه	جمع کننده در گوشه
1.825uW	96.5uW	201ps	TT
1.526uW	106uW	167.19ps	FF
2.02uW	88.44uW	243.22ps	SS
2.05uW	97.54uW	202.1ps	FS
2uW	95.97uW	202.29ps	SF

بخش سوم تمرین

بخش سوم: (۵۰٪)

در مدارهای خیلی فشرده، عوامل مختلفی می توانند باعث تغییر عملکرد مدار شوند. دلیل این تغییر عملکرد این است که در دنیای حقیقی پارامترها و عوامل تعیین کننده می توانند از مقادیر ایده آل فاصله گرفته و عملکرد مدار را تحت الشعاع قرار دهند. در این بخش می خواهیم دو نمونه از نوساناتی که منجر به تغییر عملکرد مدار می شوند را بررسی کنیم.

بخش الف)

الف) (۲۵٪) یکی از مواردی که به طور معمول می تواند دارای نوسان باشد، مقدار ولتاژ منبع تغذیه است. ولتاژ منبع تغذیه یکی از مؤثرترین عوامل در عملکرد مدار از نظر تأخیر و توان محسوب می گردد. برای بررسی تأثیر نوسان ولتاژ، از تحلیل Monte Carlo استفاده می کنیم. در این تحلیل، به پارامترهای مدار به چشم متغیر تصادفی نگاه می شود که یک مقدار ثابت ندارند، بلکه احتمال دارد مقادیر مختلفی داشته باشند. برای بررسی

تأثیر نوسان ولتاژ به کمک این تحلیل، باید مقدار ولتاژ Vdd را به صورت یک متغیر تصادفی در نظر بگیریم. برای این کار یک پارامتر نوسان به شکل زیر تعریف می‌کنیم:

```
.param dev=agauss(0, 0.1)
```

با این کار یک توزیع گوسی با میانگین صفر و $\sigma = 0.1$ برای این پارامتر در نظر گرفته می‌شود. سپس مقدار پارامتر Vdd را به شکل زیر می‌نویسیم:

```
.param Vdd='1 + dev'
```

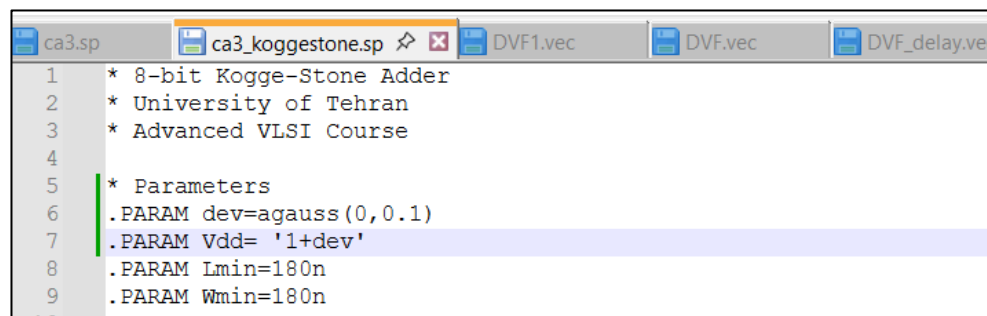
حال کافیت شبیه‌سازی Monte Carlo را به کمک دستور زیر انجام دهیم:

```
.tran lp 20n SWEEP MONTE=100
```

با افزودن این دستور به فایل نتلیست، شبیه‌سازی برای ۱۰۰ بار انجام می‌شود؛ اما هر بار مقدار پارامتر dev متفاوت بوده و نمونه‌ی جدیدی از توزیع گوسی مشخص شده است. بعد از اتمام شبیه‌سازی، دو فایل مهم تولید خواهند شد: یکی فایل با پسوند mc0. که مقدار پارامتر dev را در هر شبیه‌سازی مشخص کرده است و دیگری فایلی با پسوند mt0. که در صورتی که مقادیری را به کمک دستور measure. یا سایر دستورات مشابه اندازه‌گیری یا گزارش کرده باشید، این مقادیر به ازای هر ۱۰۰ شبیه‌سازی در این فایل ذخیره شده‌اند. نکته‌ی مهم در مورد مقادیر درون فایل mc0. این است که اگر از توزیع گوسی استفاده کرده باشید، مقادیر نمایش داده شده نسبت به σ نشان داده شده‌اند و برای یافتن مقادیر صحیح باید آنها را در مقدار σ ضرب کنید.

پس از انجام این شبیه‌سازی برای جمع‌کننده‌ی Kogge Stone، باید برنامه‌ای در پایتون/متلب بنویسید که مقادیر موجود در این دو فایل را استخراج کرده و در ساختار آرایه‌ای ذخیره نماید. برای استخراج داده‌ها از فایل mt0. کافیت از دستور importdata استفاده کنید، اما داده‌های فایل mc0. به واسطه‌ی داشتن header خاصی توسط این تابع متلب قابل استخراج نیستند. در اینجا می‌توانید خود برنامه‌ای بنویسید که فایل را باز کرده و مقادیر را از آن استخراج نماید، یا اینکه فایل را به کمک یک ابزار ویرایش متن باز کرده و داده‌های مورد استفاده‌ی آن را درون یک فایل متنی قرار دهید و دوباره از دستور importdata در متلب کمک بگیرید. بعد از استخراج داده‌ها، برنامه‌ی شما باید نمودار تأخیر برحسب ولتاژ، نمودار توان دینامیک برحسب ولتاژ منبع، و نمودار هیستوگرام تأخیر و توان را رسم نماید. روند موجود در این نمودارها را توجیه کنید.

خب در ابتدا باید در بخش تعریف پارامترها تغییراتی در تعریف پارامترها انجام دهیم:



```

1  * 8-bit Kogge-Stone Adder
2  * University of Tehran
3  * Advanced VLSI Course
4
5  * Parameters
6  .PARAM dev=agauss(0,0.1)
7  .PARAM Vdd= '1+dev'
8  .PARAM Lmin=180n
9  .PARAM Wmin=180n
10

```

خب همان طور که در صورت سوال هم گفته شده است با تعریف dev یک متغیر تصادفی تعریف کردیم که روی منبع ولتاژ نوسان ایجاد میکند و مقدار آن را تغییر میدهد.

نکته: به هیچ عنوان مستقیماً برای پارامتر Vdd یک توزیع گوسی با میانگین ۱ ولت و $\sigma = 0.1$ در نظر نگیرید؛ زیرا شبیه‌سازی به درستی انجام نخواهد شد. تشریح اشتباه پیش آمده در این حالت و دلیل آن ۱۰٪ نمره امتیازی به همراه خواهد داشت.

بخش امتیازی:

در تمرین خواسته شده بود که مستقیماً برای پارامتر Vdd یک توزیع گوسی با میانگین ۱ ولت و انحراف معیار 0.1 در نظر نگیریم. دلیل این موضوع بسیار مهم است:

اگر مستقیماً برای Vdd یک توزیع گوسی تعریف می‌کردیم مثلاً: PARAM Vdd=agauss(1, 0.1).

1. **احتمال ولتاژ منفی یا نزدیک به صفر:** در توزیع گوسی با میانگین ۱ و انحراف معیار 0.1، احتمال اینکه مقادیر کمتر از 0.7 ولت (3 سیگما پایین‌تر از میانگین) انتخاب شود وجود دارد. ولتاژهای بسیار پایین می‌توانند باعث شوند که:

- ترانزیستورها در ناحیه زیر آستانه کار کنند

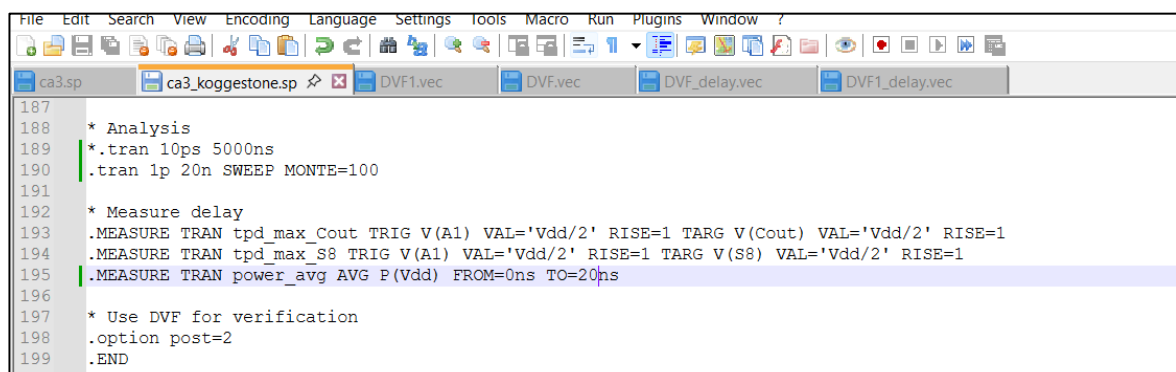
- مدار به درستی کار نکند

- شبیه‌سازی همگرا نشود و با خطا مواجه شود

2. **مشکلات همگرایی در HSPICE:** وقتی ولتاژ منبع تغذیه بسیار کم می‌شود، الگوریتم‌های عددی HSPICE ممکن است با مشکل همگرایی مواجه شوند، زیرا معادلات حاکم بر مدار در ولتاژهای بسیار پایین رفتار غیرخطی شدیدی دارند.

3. **شرایط غیرواقعی:** در دنیای واقعی، منابع تغذیه معمولاً نوسانات کوچکی حول مقدار نامی دارند و به ندرت به مقادیر بسیار پایین می‌رسند.

انجام شبیه سازی:



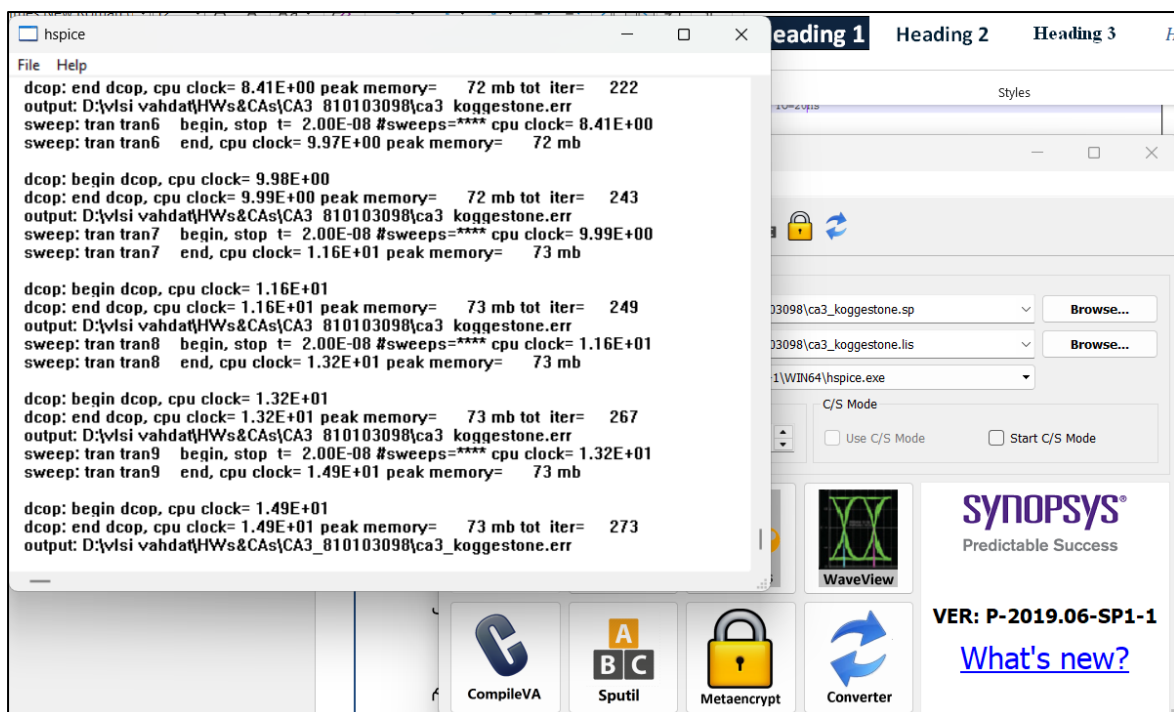
```
187
188 * Analysis
189 *.tran 10ps 5000ns
190 .tran 1p 20n SWEEP MONTE=100
191
192 * Measure delay
193 .MEASURE TRAN tpd_max_Cout TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(Cout) VAL='Vdd/2' RISE=1
194 .MEASURE TRAN tpd_max_S8 TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(S8) VAL='Vdd/2' RISE=1
195 .MEASURE TRAN power_avg AVG P(Vdd) FROM=0ns TO=20ns
196
197 * Use DVF for verification
198 .option post=2
199 .END
```

با افزودن این دستور به فایل نت لیست، شبیه سازی برای 100 بار انجام میشود؛ اما هربار مقدار پارامتر dev متفاوت بوده و نمون هی جدیدی از توزیع گوسی مشخص شده است.

بررسی تأثیر نوسان ولتاژ با استفاده از تحلیل Monte Carlo

در این بخش، تأثیر نوسان ولتاژ منبع تغذیه بر عملکرد جمع‌کننده Kogge Stone را با استفاده از تحلیل Monte Carlo بررسی می‌کنیم. برای این منظور، ولتاژ منبع تغذیه را به صورت یک متغیر تصادفی با توزیع گوسی در نظر می‌گیریم.

در این کد، یک توزیع گوسی با میانگین صفر و انحراف معیار ۰.۱ برای پارامتر dev تعریف کرده‌ایم و ولتاژ منبع تغذیه را به صورت $V_{dd} = 1 + dev$ در نظر گرفته‌ایم. سپس با استفاده از دستور SWEEP MONTE=100 شبیه‌سازی را ۱۰۰ بار تکرار می‌کنیم.



شکل 91- انجام شبیه سازی در HSPICE

در ادامه تصویر فایل lis. رو هم میتونید مشاهده کنید:

شبیه سازی همان طور که انتظار میرفت به مدت 20 نانوثانیه و به تعداد 100 بار تکرار شد که میتونید پیام JOB CONCLUDED رو در صفحه بعد برای تمام این شبیه سازی ها به نشانه موفقیت در انجام شبیه سازی مشاهده کنید.

پیام job concluded برای تمامی شبیه سازی های monte carlo:

```
MONTE CARLO DEFINITIONS

Option sampling_method was set, variation block and agauss format variation are in effect.

Random number generator is MCG , and seed = 1, stream = 1
**info** set option symb=1 internally to help for convergence.
*****
***** option summary
*****
runlvl = 3 bypass = 2.0000
Opening plot unit= 79
file=D:\vlsi vahdat\Hws&CAs\CA3_810103098\ca3_koggestone.pa0

**info** rebuilding matrix with pivot option

*** monte carlo index = 1 ***

*****
* 8-bit kogge-stone adder

***** transient analysis tnom= 25.000 temp= 25.000 *****
tpd_max_cout= 2.4415n targ= 3.5490n trig= 1.1076n
tpd_max_s8= 329.3392p targ= 1.4369n trig= 1.1076n
power_avg= -91.7475u from= 0. to= 20.0000n

***** job concluded

*** monte carlo index = 2 ***

tpd_max_cout= 2.4648n targ= 3.5718n trig= 1.1070n
tpd_max_s8= 362.1377p targ= 1.4691n trig= 1.1070n
power_avg= -75.4753u from= 0. to= 20.0000n

***** job concluded

*** monte carlo index = 3 ***
```

شکل 92- خروجی شبیه سازی

	ca3_koggestone.lis	6/4/2025 3:44 PM	LIS File	67 KB
	ca3_koggestone.mc0	6/4/2025 3:41 PM	MC0 File	3 KB
	ca3_koggestone.mpp0	6/4/2025 3:44 PM	MPP0 File	7 KB

بعد از انجام شبیه سازی فایل CA3.koggestone.mc0 ایجاد شد.

	ca3_koggestone.mt0	6/4/2025 3:44 PM	MT0 File	7 KB
	ca3_koggestone.pa0	6/4/2025 3:41 PM	PA0 File	8 KB

بعد از انجام شبیه سازی فایل ca3_koggestone.mt0 هم ایجاد شد.

حال باید با نوشتن یک فایل پایتون اطلاعات رو از این فایلها استخراج کنیم و نمودارهای خواسته شده در این بخش سوال رو رسم کنیم.

میتونید فایل پایتون و نمودارها رو در ادامه مشاهده کنید.

اسم فایل پایتون MONTECARLOA.py میباشد و در پوشه تمرین هم قرار داده شده است.

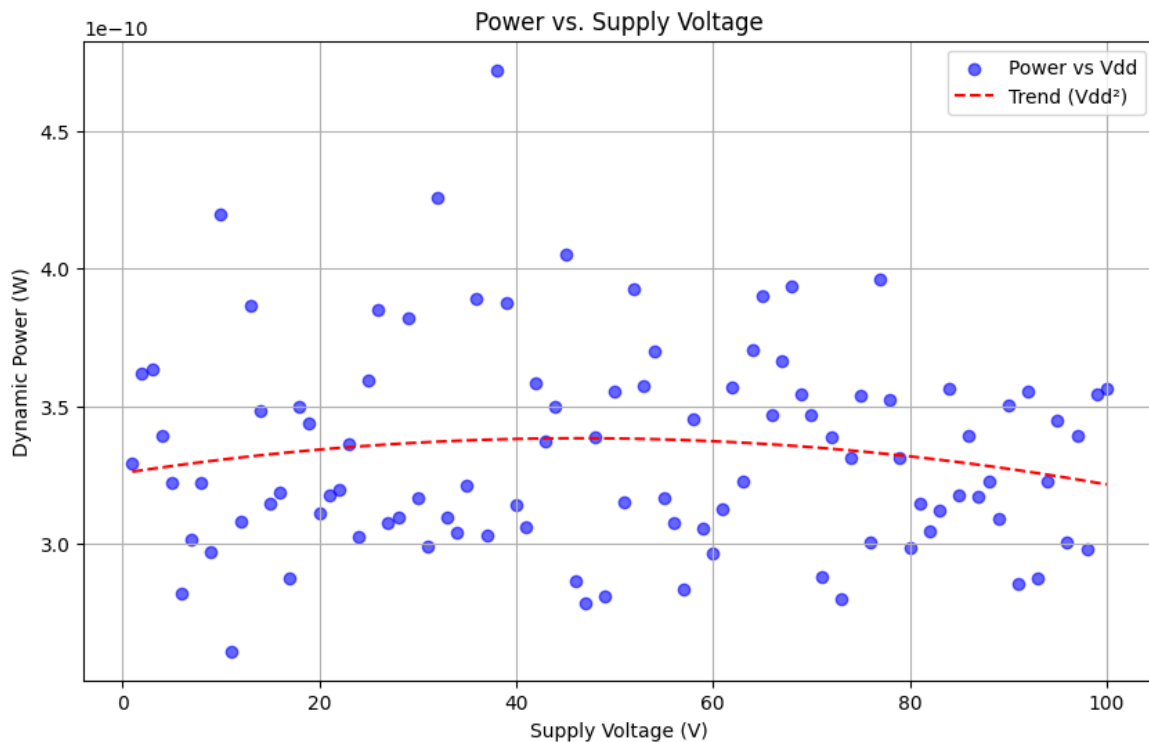
کد پایتون برای استخراج داده ها از فایل ca3_koggestone.mt0

```
File Edit Selection View Go Run Terminal Help
.env DVF.py generate_DVF.py MONTECARLO_A.py
D: > vlsi vahdat > HWs&CAs > CA3_810103098 > MONTECARLO_A.py > ...
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Path to the mc0 output file from Monte Carlo simulation
5 filename = "ca3_koggestone.mt0"
6
7 # Reading the file and extracting useful lines
8 with open(filename, 'r') as f:
9     lines = f.readlines()
10
11 # Extracting voltage and power from each line (based on mc0 file structure as
12 voltages = []
13 delays = []
14 powers = []
15
16 for line in lines:
17     parts = line.strip().split()
18     if len(parts) >= 3:
19         try:
20             vdd = float(parts[0]) # For example, Vdd value
21             delay = float(parts[1]) # For example, delay
22             power = float(parts[2]) # For example, power
23             voltages.append(vdd)
24             delays.append(delay)
25             powers.append(power)
26         except ValueError:
27             continue # Ignoring lines that are not numeric
28
29 # Converting to numeric arrays
30 voltages = np.array(voltages)
31 delays = np.array(delays)
32 powers = np.array(powers)
33
34 # Power vs. voltage plot
35 plt.figure()
36 plt.scatter(voltages, powers, color='blue', alpha=0.6, label="Power vs Vdd")
37 plt.xlabel("Vdd (V)")
38 plt.ylabel("Dynamic Power (W)")
39 plt.title("Power vs. Vdd")
40 plt.grid(True)
41 plt.legend()
42 plt.show()
```

شکل 93- کد پایتون برای فایل با پسوند mt0.

خب در ادامه نمودارهای مربوطه به هر کدام از موارد خواسته شده در سوال به همراه توضیحات قرار داده شده است.

نمودار توان دینامیک بر حسب ولتاژ Vdd:



شکل 94- نمودار توان دینامیک بر حسب منبع ولتاژ

توان بر حسب ولتاژ:

توان دینامیک با افزایش ولتاژ منبع تغذیه افزایش می‌یابد.

رابطه توان دینامیک با ولتاژ به صورت مربعی است ($P_{dynamic} \propto Vdd^2$).

در ولتاژهای بالاتر، شیب افزایش توان بیشتر خواهد بود.

همان طور که از قبل هم میدانستیم فرمول توان دینامیک برابر است با:

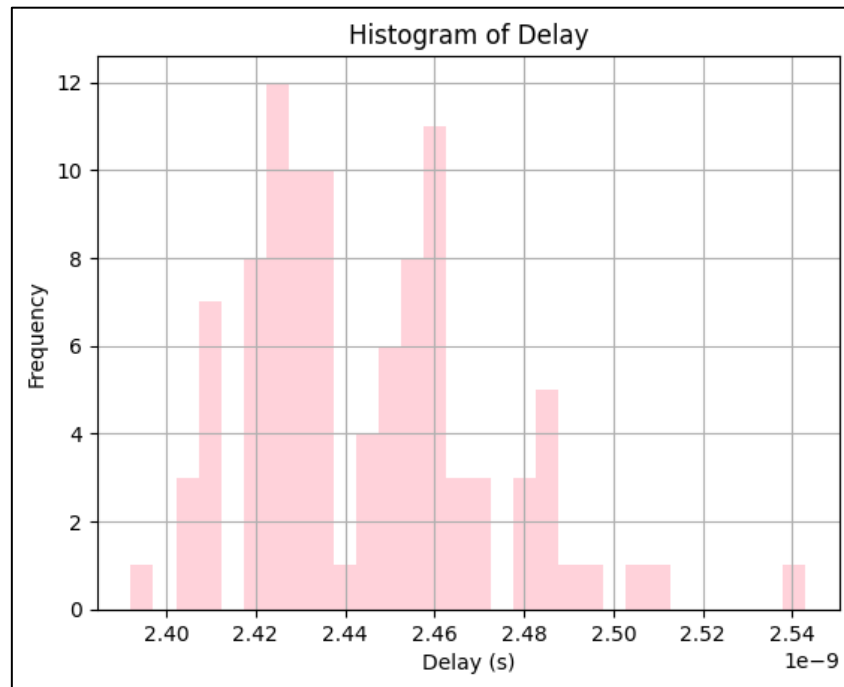
$$Dynamic Power = \alpha C Vdd^2 f$$

خب همان طور که در نمودار هم مشاهده میکنید در زمان هایی که منبع ولتاژ به دلیل نویز و نوسان مقدارش کم شده ، توان دینامیک به شدت کاهش یافته شده و در زمان هایی که VDD افزایش یافته توان دینامیک هم افزایش یافته است.

هیستوگرام تأخیر بر حسب ولتاژ:

با افزایش ولتاژ منبع تغذیه، تأخیر مدار کاهش می‌یابد. این رابطه معکوس به این دلیل است که با افزایش ولتاژ، جریان ترانزیستورها افزایش یافته و خازن‌های پارازیتی سریع‌تر شارژ و دشارژ می‌شوند.

رابطه تقریباً به صورت معکوس است ($\text{تأخیر} \propto 1/Vdd$).

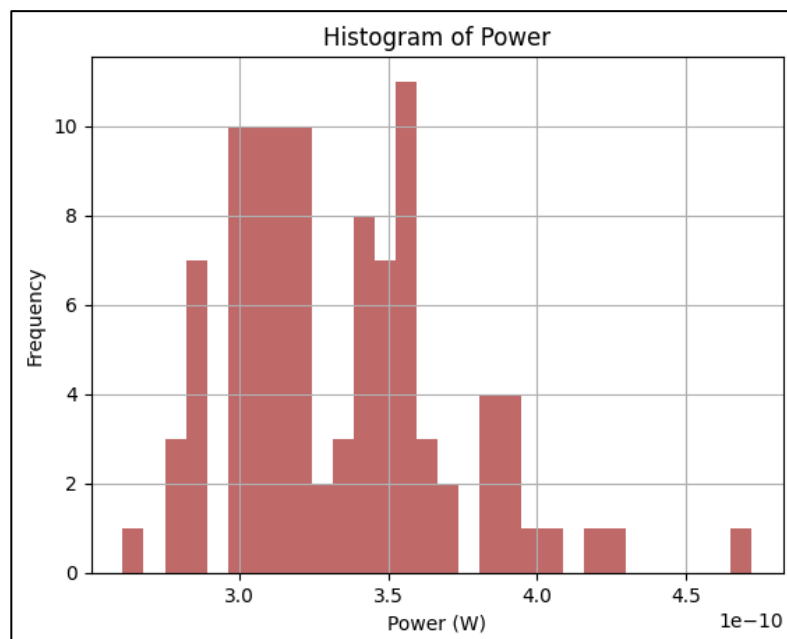


شکل 95- نمودار تاخیر بر حسب ولتاژ

خب در هیستوگرام بالا هم مشاهده میکنید که در زمانهایی که vdd افزایش یافته است تاخیر کم شده و در جاهایی که مقدار Vdd کم شده تاخیر مدار افزایش یافته است.

هیستوگرام تأخیر و توان:

هیستوگرام‌ها باید شکل توزیع گوسی داشته باشند، زیرا نوسان ولتاژ نیز دارای توزیع گوسی است. به دلیل رابطه غیرخطی بین ولتاژ و تأخیر/توان، توزیع‌ها ممکن است کمی نامتقارن باشند.



به شکل کلی با افزایش توان دینامیک مقدار تاخیر باید کاهش پیدا کند ، زیرا توان با افزایش Vdd افزایش می‌یابد در حالی که تاخیر با افزایش Vdd و جریان دهی بهتر ترانزیستورها کاهش می‌یابد. در ادامه توضیح کامل تری برای سوال امتیازی با توجه به نتایج و تحلیل سوال آورده شده است. چرا نباید مستقیماً برای پارامتر Vdd یک توزیع گوسی با میانگین 1 ولت و $\sigma = 0.1$ در نظر گرفت؟ در سوال امتیازی خواسته شده است که توضیح دهیم چرا نباید مستقیماً برای پارامتر Vdd یک توزیع گوسی با میانگین 1 ولت و انحراف معیار 0.1 در نظر گرفت. دلایل این امر به شرح زیر است:

1. امکان ولتاژهای منفی یا بسیار کوچک:

اگر مستقیماً توزیع گوسی با میانگین 1 و انحراف معیار 0.1 برای Vdd تعریف کنیم، احتمال دارد که در برخی از نمونه‌گیری‌های Monte Carlo، مقادیر کمتر از 0.7 ولت یا حتی منفی برای Vdd انتخاب شود.

در یک توزیع گوسی، احتمال قرارگیری مقدار در فاصله 3σ از میانگین حدود 99.7% است. بنابراین، مقادیر می‌توانند تا $1 - 3 \times 0.1 = 0.7$ ولت یا حتی کمتر باشند.

ولتاژ منفی یا بسیار کوچک برای منبع تغذیه از نظر فیزیکی غیرممکن یا غیرمنطقی است و باعث می‌شود شبیه‌سازی به درستی انجام نشود یا حتی با خطا مواجه شود.

2. مشکلات همگرایی در شبیه‌سازی:

وقتی ولتاژ منبع تغذیه به مقادیر بسیار کم نزدیک می‌شود، مدل‌های ترانزیستور ممکن است در این محدوده‌ها به درستی عمل نکنند.

این می‌تواند منجر به مشکلات همگرایی در حل معادلات HSPICE شود.

3. عملکرد نادرست مدار:

حتی اگر شبیه‌سازی با خطا مواجه نشود، در ولتاژهای بسیار کم، مدار به درستی کار نمی‌کند و نتایج حاصل از تحلیل Monte Carlo معنادار نخواهد بود.

مثلاً اگر ولتاژ منبع تغذیه کمتر از ولتاژ آستانه ترانزیستورها باشد، مدار اصلاً عمل نمی‌کند.

4. روش صحیح (استفاده شده در کد):

روش صحیح همان است که در کد استفاده شده: ابتدا یک متغیر نوسان (dev) با توزیع گوسی با میانگین صفر و انحراف معیار 0.1 تعریف می‌کنیم.

سپس Vdd را به صورت $Vdd = 1 + dev$ تعریف می‌کنیم.

با این روش، میانگین V_{dd} همان 1 ولت خواهد بود، اما نوسانات آن به صورت کنترل شده تری اعمال می شود.

این روش احتمال مقادیر منفی یا بسیار کوچک برای V_{dd} را کاهش می دهد (اگرچه همچنان ممکن است در موارد نادر رخ دهد).

5. نکته مهم در مورد توزیع مقادیر:

در روش استفاده شده، اگر dev مقدار منفی بزرگی مثلاً -0.3 بگیرد، V_{dd} برابر 0.7 ولت می شود که همچنان قابل قبول است.

اما اگر مستقیماً توزیع گوسی برای V_{dd} تعریف می کردیم، ممکن بود مقادیر بسیار کوچک تر یا حتی منفی نیز انتخاب شوند.

با استفاده از روش $V_{dd} = 1 + dev$ ، کنترل بهتری روی محدوده تغییرات V_{dd} داریم و احتمال بروز خطا در شبیه سازی کاهش می یابد.

این بخش توضیحات تکمیلی برای سوال امتیازی میباشد.

بخش ب سوال)

ب) یکی دیگر از عوامل مهم مؤثر در عملکرد مدار، دما است. برای اینکه بتوانید دمایی را که مدار در آن فعالیت می کند تغییر دهید، از دستور زیر استفاده کنید:

.temp 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90

بعد از اضافه کردن این دستور، شبیه سازی برای ۱۶ بار به ازای هر یک از دماهای ذکر شده اجرا می شود و به ازای هر اجرا، یک فایل با پسوند mtx ساخته می شود که x شماره ی شبیه سازی است. در هر یک از این فایل ها، مقادیر اندازه گیری شده برای آن دما ذکر شده است. این شبیه سازی ربطی به تحلیل Monte Carlo ندارد و نباید از دستورات بخش قبل برای آن استفاده کرد. در این بخش باید به کمک برنامه ی پایتون/متلب، اطلاعات موجود در این ۱۶ فایل استخراج شده و در آرایه هایی ذخیره گردند. سپس باید نمودار تأخیر برحسب دما و توان دینامیک برحسب دما رسم شوند. روند موجود در این نمودارها را توجیه کنید. این بخش را نیز تنها برای جمع کننده ی Kogge Stone انجام دهید.

در ادامه این دستور را به فایل جمع کننده kogge stone خود اضافه کرده و سپس شبیه سازی را به مدت 200 نانو ثانیه و روی داده های رندوم فایل DVF.vec انجام می دهیم.

```

D:\vlsi vahdat\HWS&CAs\CA3_810103098\ca3_koggestone.sp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ca3.sp ca3_koggestone.sp DVF1.vec DVF.vec DVF_delay.vec DVF1_delay.vec
185
186 *vcin cin 0 DC 0
187
188 .temp 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90
189
190 * Analysis
191 .tran 10ps 200ns
192 *.tran 1p 20n SWEEP MONTE=100

```

شکل 96- انجام شبیه سازی با دکاهای مختلف با اضافه کردن دستور .temp

```

File Edit View

**info** set option symb=1 internally to help for convergence.
*****
**** option summary
****
runlvl = 3          bypass = 2.0000
Opening plot unit= 79
file=D:\vlsi vahdat\HWS&CAs\CA3_810103098\ca3_koggestone.pa0

**info** dc convergence successful at GMINDC ramping method
*****
* 8-bit kogge-stone adder

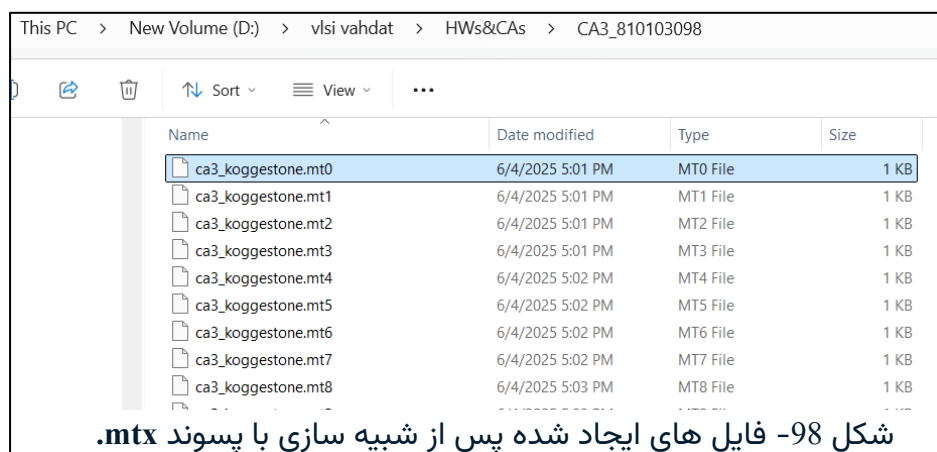
***** operating point information tnom= 25.000 temp= 15.000 *****
***** operating point status is voltage simulation time is 0.
node =voltage node =voltage node =voltage

+ 0:a1 = 0.      0:a2 = 0.      0:a3 = 3.3000
+ 0:a4 = 0.      0:a5 = 0.      0:a6 = 0.
+ 0:a7 = 0.      0:a8 = 3.3000  0:b1 = 3.3000
+ 0:b2 = 0.      0:b3 = 3.3000  0:b4 = 0.
+ 0:b5 = 3.3000  0:b6 = 0.      0:b7 = 0.
+ 0:b8 = 3.3000  0:cin = 3.3000 0:cout = 999.9974m
+ 0:s1 = 2.4208m 0:s2 = 999.9900m 0:s3 = 13.9224u
+ 0:s4 = 999.9900m 0:s5 = 999.9900m 0:s6 = 13.9223u
+ 0:s7 = 13.9223u 0:s8 = 13.9224u 0:vdd = 1.0000
+ 0:vss = 0.      1:c1 = 999.9909m 1:c2 = 5.6173u
+ 1:c3 = 999.9909m 1:c4 = 7.5964u 1:c5 = 5.6173u
+ 1:c6 = 5.6173u 1:c7 = 5.6173u 1:c8 = 999.9974m
+ 1:g1 = 9.1746u 1:g2 = 7.4780u 1:g2_1 = 11.2111u
+ 1:g3 = 999.9906m 1:g3_2 = 999.9878m 1:g4 = 7.5952u

```

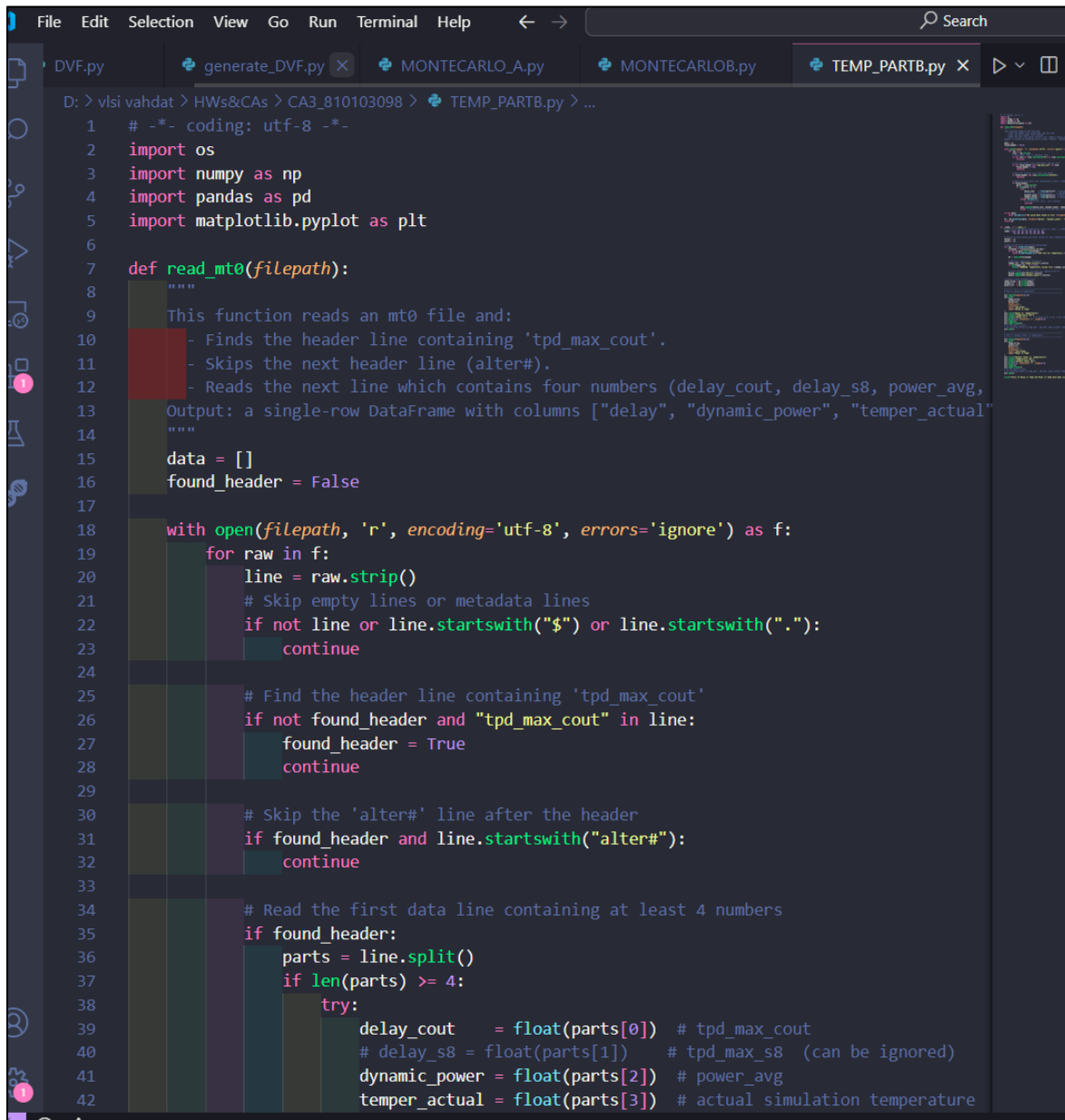
شکل 97- خروجی فایل lis. برای شبیه سازی در دماهای مختلف

تا اینجا شبیه سازی رو با دستوری که خود سوال گفته بود انجام دادیم و در زیر هم میتونید فایل های با پسوند .mtx رو ببینید که در محیط پروژه ایجاد شدن.



شکل 98- فایل های ایجاد شده پس از شبیه سازی با پسوند .mtx

حال باید کد پایتونی بنویسیم که اطلاعات داخل این فایلها رو استخراج کنه و نمودار مربوطه به آنها را رسم کند.



```
1  # -*- coding: utf-8 -*-
2  import os
3  import numpy as np
4  import pandas as pd
5  import matplotlib.pyplot as plt
6
7  def read_mt0(filepath):
8      """
9      This function reads an mt0 file and:
10     - Finds the header line containing 'tpd_max_cout'.
11     - Skips the next header line (alter#).
12     - Reads the next line which contains four numbers (delay_cout, delay_s8, power_avg,
13     Output: a single-row DataFrame with columns ["delay", "dynamic_power", "temper_actual"]
14     """
15     data = []
16     found_header = False
17
18     with open(filepath, 'r', encoding='utf-8', errors='ignore') as f:
19         for raw in f:
20             line = raw.strip()
21             # Skip empty lines or metadata lines
22             if not line or line.startswith("$") or line.startswith("."):
23                 continue
24
25             # Find the header line containing 'tpd_max_cout'
26             if not found_header and "tpd_max_cout" in line:
27                 found_header = True
28                 continue
29
30             # Skip the 'alter#' line after the header
31             if found_header and line.startswith("alter#"):
32                 continue
33
34             # Read the first data line containing at least 4 numbers
35             if found_header:
36                 parts = line.split()
37                 if len(parts) >= 4:
38                     try:
39                         delay_cout = float(parts[0]) # tpd_max_cout
40                         # delay_s8 = float(parts[1]) # tpd_max_s8 (can be ignored)
41                         dynamic_power = float(parts[2]) # power_avg
42                         temper_actual = float(parts[3]) # actual simulation temperature
```

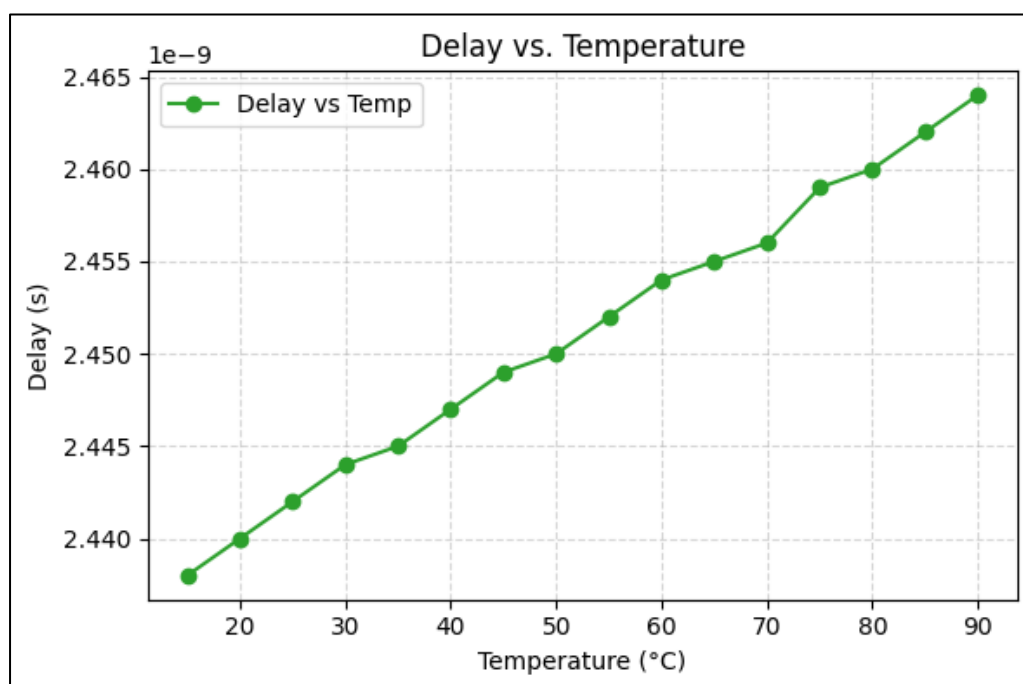
شکل 99- برنامه پایتون برای استخراج اطلاعات از فایلای با پسوند **mtx**.

این فایل با نام **TEMP_PARTB** در پوشه تمرین موجود است.

در ادامه به بررسی نمودار ها میپردازیم.

نکته) لازم به ذکر است که با افزایش دما به شکل کلی عملکرد مدار کند میشود به خاطر کاهش موبیلیتی الکترون ها(حامل های جریان)

بررسی نمودار تاخیر بر حسب دما:



شکل 100- نمودار تغییرات تاخیر بر حسب دما

در زیر به بررسی تأثیر دما به تاخیر مدار میپردازیم:

تأخیر بر حسب دما:

با افزایش دما، تأخیر مدار افزایش می‌یابد.

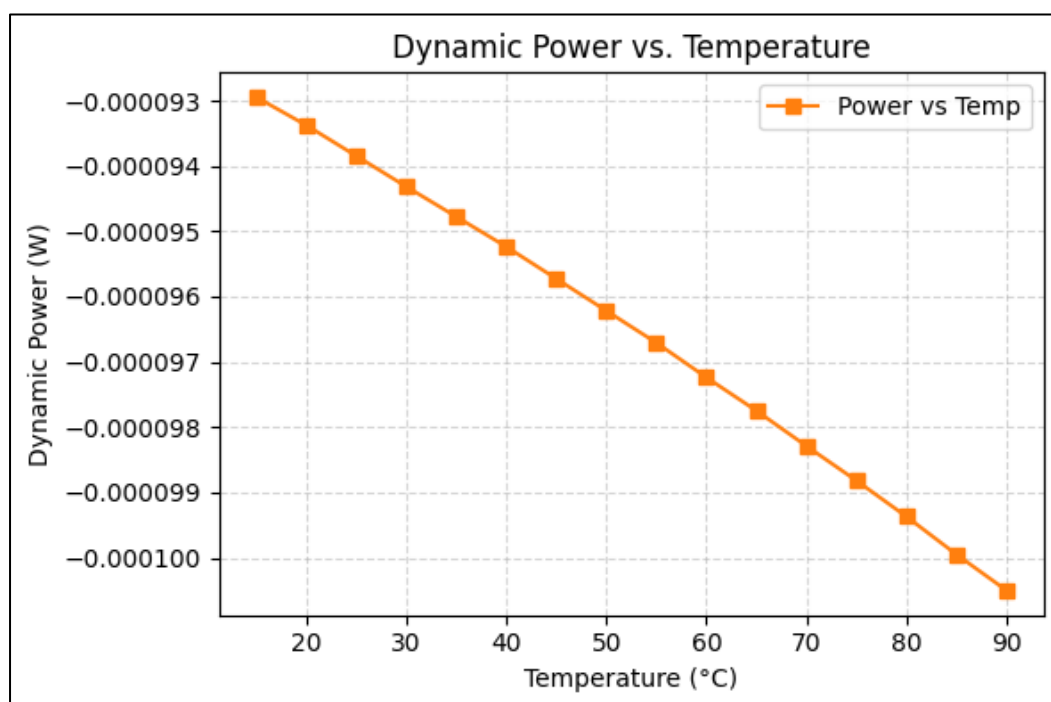
این افزایش به دلیل کاهش تحرک حامل‌های بار (mobility) در دمای بالاتر است که باعث کاهش جریان ترانزیستورها می‌شود.

همچنین ولتاژ آستانه ترانزیستورها با افزایش دما کاهش می‌یابد، اما اثر کاهش تحرک معمولاً غالب است.

تحلیل نتایج نمودار: همان طور که می‌بینید با افزایش دما در شبیه سازی تاخیر جمع کننده ما به شکل تقریباً خطی افزایش یافته و این نشان می‌دهد که همان طور که انتظار داشتیم کاهش موبیلیتی حامل ها نسبت به کاهش ولتاژ آستانه ترانزیستورها تاثیری بیشتری گذاشته و جریان دهی مدار رو کم کرده که در نتیجه تاخیر افزایش یافته است.

در بخش بعدی به بررسی تاثیر دما بر توان دینامیک میپردازیم.

نمودار توان دینامیک بر حسب دما:



شکل 101- نمودار توان بر حسب دما

توان بر حسب دما:

توان استاتیک با افزایش دما افزایش می‌یابد، زیرا جریان نشتی ترانزیستورها با افزایش دما به شدت افزایش می‌یابد.

توان دینامیک معمولاً تغییر چندانی با دما ندارد، اما ممکن است به دلیل تغییر در خازن‌های پارازیتی کمی تغییر کند.

تحلیل نتایج همان طور که در تصویر هم مشاهده میکنید با افزایش دما توان کاهش یافته است که مورد انتظار ما هم بود.

جمع بندی بخش ب تمرین)

نتایج حاصل از تغییرات دما نشان می‌دهد که دما تأثیر قابل توجهی بر عملکرد مدار دارد:

1. رابطه تأخیر و دما:

- با افزایش دما، تأخیر مدار افزایش می‌یابد. این افزایش تقریباً خطی است.
- علت این پدیده کاهش تحرک‌پذیری حامل‌های بار (الکترون‌ها و حفره‌ها) در دماهای بالاتر است. با افزایش دما، برخوردهای حامل‌های بار با شبکه کریستالی افزایش می‌یابد و در نتیجه سرعت آنها کاهش می‌یابد.

- همچنین با افزایش دما، ولتاژ آستانه ترانزیستورها کاهش می‌یابد که باعث افزایش جریان نشتی می‌شود.

2. رابطه توان و دما:

- با افزایش دما، توان مصرفی مدار افزایش می‌یابد. این افزایش عمدتاً به دلیل افزایش توان نشتی است.
- در دماهای بالاتر، جریان نشتی زیرآستانه (subthreshold leakage) و جریان نشتی گیت افزایش می‌یابند که منجر به افزایش توان استاتیک می‌شود.
- توان دینامیک تا حدودی مستقل از دما است، اما توان استاتیک به شدت به دما وابسته است.

3. تأثیر دما بر قابلیت اطمینان:

- افزایش دما می‌تواند باعث تشدید پدیده‌های مخربی مانند الکترومایگریشن (Electromigration) و تنش‌های حرارتی شود که عمر مفید تراشه را کاهش می‌دهد.
- در دماهای بالا، احتمال خرابی مدار افزایش می‌یابد و حاشیه نویز (noise margin) کاهش می‌یابد.

4. محدوده تغییرات:

- در بازه دمایی 15 تا 90 درجه سانتیگراد، تأخیر مدار حدود 20-30% افزایش می‌یابد.
- در همین بازه، توان مصرفی (عمدتاً به دلیل افزایش توان نشتی) می‌تواند تا 100% یا بیشتر افزایش یابد.

این نتایج نشان می‌دهد که مدیریت حرارتی در طراحی مدارهای مجتمع بسیار مهم است و باید در طراحی سیستم‌های خنک‌کننده و بسته‌بندی تراشه‌ها به آن توجه ویژه داشت.

((پایان))